Spring Boot

The Illustrated Book

From Spring core and REST to JPA, transactions and security — every topic of the course as a picture

Companion to the ABIS course text «Spring Boot for BNP» (1618_03b) and the `JpaSolution` project

Contents

SPRING FOUNDATIONS

1 Introduction to Spring 3

2 Getting Started with Spring Boot 9

3 Testing in Spring 16

4 Spring Core 24

5 Aspect-Oriented Spring 39

REST AND WEB SERVICES

6 Introduction to REST and JSON 47

7 Spring REST: basics and more functionality 57

8 Creating REST Clients 70

9 Exception Handling and Validation 77

10 Additional Topics 89

PERSISTENCE WITH JPA

11 Introduction to ORM and (Spring) JPA 92

12 Configuration 99

13 Basic OR Mapping and JPA Repositories 106

14 Mapping Associations 116

15 Querying in Spring JPA 128

16 Transaction Management and Exception Handling 140

SECURITY

17 Securing REST APIs 152

APPENDIX

A The Exercises and the JpaSolution Project 168

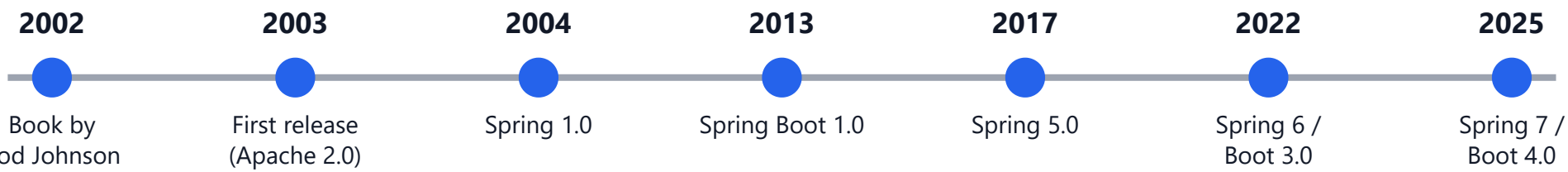
CHAPTER 1

Introduction to Spring

Goals of the framework · the Spring landscape · Spring vs. Spring Boot

1.1 Some history

Spring began as a **reaction against the complexity of EJBs**. Rod Johnson (and Juergen Hoeller) described a lighter way of building enterprise Java in the book *Expert One-on-One J2EE Design and Development* (October 2002). The framework was first called *Interface21*; the name «Spring» stands for a fresh start after the winter of J2EE. First release under the Apache 2.0 license: **June 2003**.



1.2 Design goals

Mission: simplify the development of enterprise Java applications, from front to back. Four key strategies:

POJO
lightweight & minimally invasive

DI
loose coupling & interface orientation

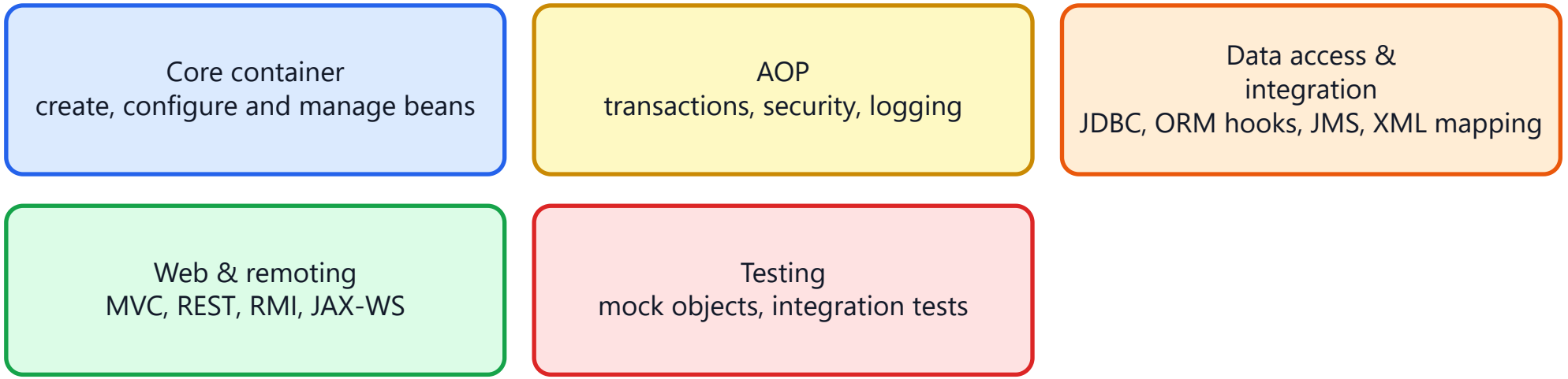
AOP
declarative programming

Templates
boilerplate reduction

Strategy	What it means in practice
Lightweight & minimally invasive (POJOs)	Your classes do not extend Spring classes; Spring's API does not litter your code.
Loose coupling (DI + interfaces)	Code becomes easier to understand, test and reuse. «Whack-a-mole» bugs are avoided.
Declarative programming (aspects)	Cross-cutting concerns such as logging, transactions and security are kept out of the business code.
Boilerplate reduction (templates)	Commonly repeated code is encapsulated (DRY), so you concentrate on business logic.

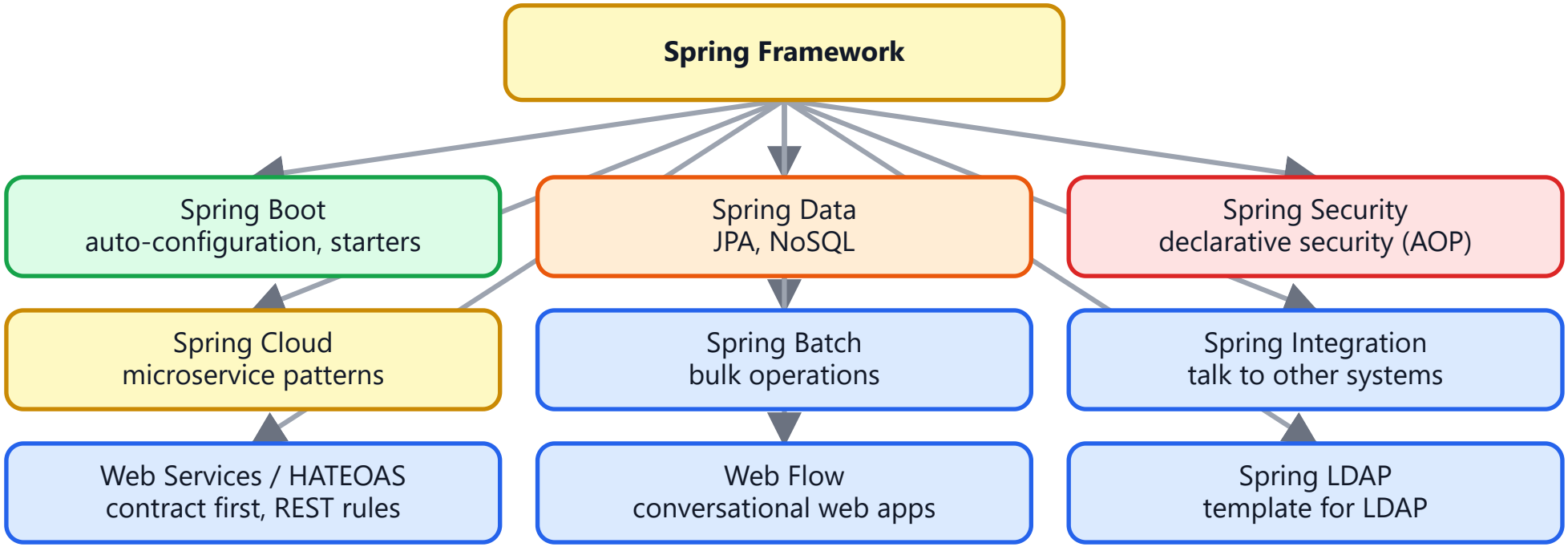
1.3 The Spring Framework landscape

The core framework consists of about 20 jar files, grouped in 6 categories. You are free to pick only the modules your application needs.



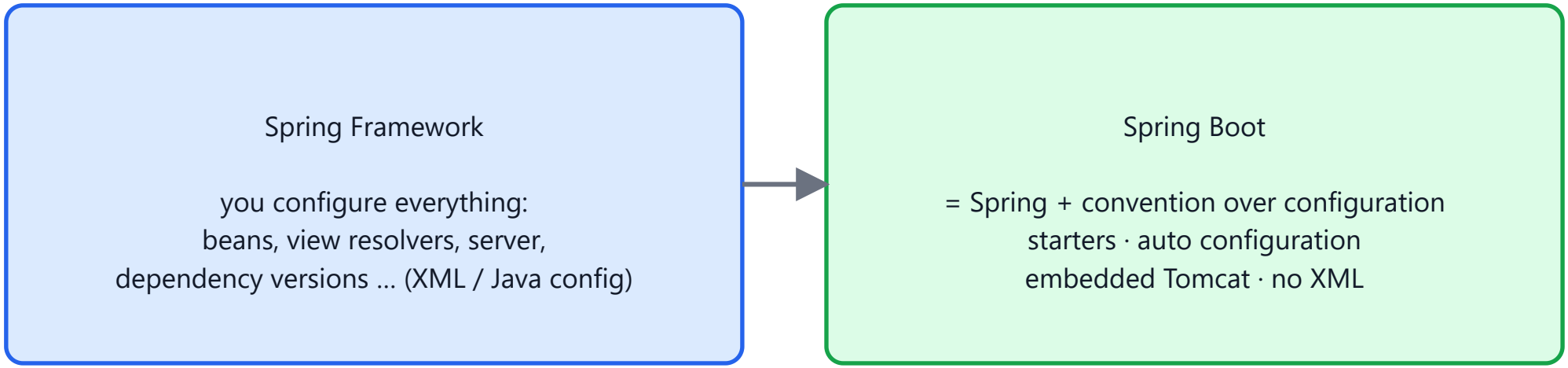
The Spring portfolio

Projects built on the core framework and on each other (see `spring.io/projects`):



1.4 Spring vs. Spring Boot

Spring Boot is an extension of the Spring framework that removes the boilerplate configuration needed to set up an application. It follows *convention over configuration*: easier to understand and develop, more productive, shorter development time, no XML configuration needed.



Versions

Spring Boot	Spring Framework	Notes
1.0 (2013)	4.0	
2.0 (2018)	5.0	
3.0 (Nov 2022)	6.0	needs JDK 17; latest 3.5.x (course uses 3.5)
4.0 (Nov 2025)	7.0	modularised code base → **some Maven dependencies changed**

The four main features of Spring Boot

Starters

Aggregate common groups of dependencies. Based on transitive dependencies; version compatibility is taken care of. 30+ starters.

Auto configuration

Spring Boot guesses the beans you need from what is on the classpath.

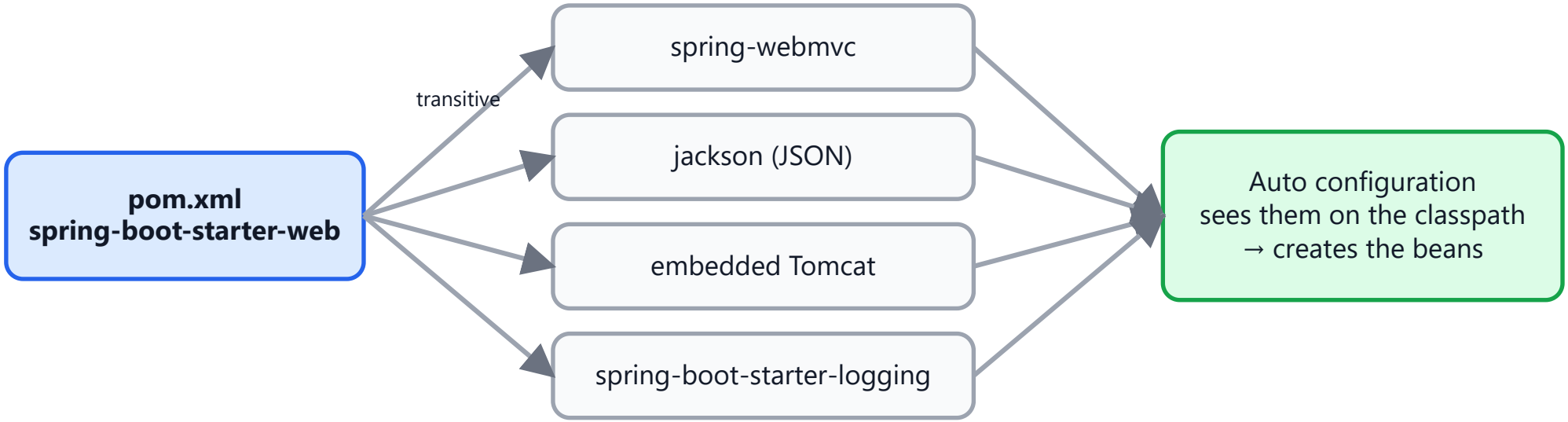
CLI

Command-line interface (Groovy based) that simplifies things even further.

Actuator

Adds management features (health, metrics, ...) to a running application.

1.5 Starter dependencies and auto configuration



> **Watch out for side effects!**
> Auto configuration makes assumptions. Adding a dependency can switch on beans you did not ask for (e.g. adding `spring-boot-starter-web` automatically adds a ViewResolver, resource handlers and message converters).

Running the class as a **Spring Boot App** bootstraps an embedded Tomcat server, deploys your application and maps the URLs. Typical starters: `-web`, `-aop`, `-jdbc`, `-data-jpa`, `-ws` (REST is included in `-web`).

CHAPTER 2

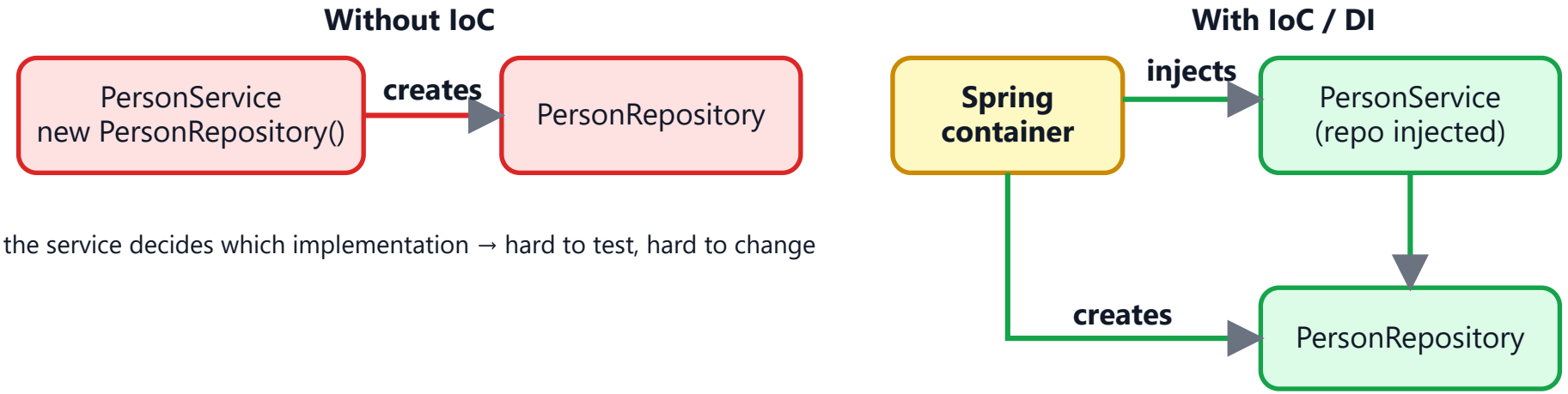
Getting Started with Spring Boot

Philosophy · ingredients · tools · a Hello World application

2.1 Philosophy

- **Design:** loose coupling, coding to interfaces, *dependency injection* and *inversion of control* (IoC). Avoid «lasagna» (too many layers) and «ravioli» (many small, tangled objects) code.
- **DRY:** avoid boilerplate → templates, configuration outside the code, AOP.
- **Easy to test:** because of the design, plus extra features on top of JUnit.

Inversion of control means: objects do **not** create the objects they rely on. They get them from an outside source.



2.2 Ingredients



Ingredient	Description
Java Beans	Reusable components that follow a convention: default constructor, private fields with getters/setters, serializable.
Services	**Repository/DAO** (CRUD methods) and a **Service layer** decoupled from the model.
Configuration	XML (deprecated), annotations, Java based configuration, `application.properties`.
Container	The `ApplicationContext` that creates and wires the beans.

2.3 Tools

IDE

IntelliJ, Eclipse with the STS plugin, or Spring Tool Suite.

Maven

Build automation. The **POM** describes the project, its dependencies and the build order; libraries are downloaded from Maven Central.

Server

No external application server needed: **embedded Tomcat** is provided.

2.4 Hello World

The recipe

1. Create a new project with **Spring Initializr** (IDE or start.spring.io).
2. Check the Spring Boot version in `pom.xml` and add the dependencies you need.
3. Spring creates `SpringHelloApplication` and `SpringHelloApplicationTests` in the **base package**.
4. Put your model and service classes in **sub-packages** of the base package — only there will Spring Boot look for registered classes.
5. The service layer = an **interface + implementation**; the class is registered with `@Component`.
6. In the test class, wire in the **interface** with `@Autowired`; Spring finds the implementation.

src/main/java/be/abis/hellospring

- HellospringApplication.java ← base package (@SpringBootApplication)
- model/Person.java
- service/HelloService.java (interface)
- service/AbisHelloService.java (@Component)

src/test/java/be/abis/hellospring

- HellospringApplicationTests.java

pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.7</version>
</parent>
<properties>
  <java.version>21</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Application class, model, service, test

```
@SpringBootApplication
public class HellospringApplication {
    public static void main(String[] args) {
        SpringApplication.run(HellospringApplication.class, args);
    }
}

// starts the app, registers, instantiates
// and wires all classes found in sub-packages

public class Person {
    private String firstName;

    public Person(String firstName) {
        ...
    }
```

```
    public String getFirstName() {
        ...
    }
}
```

```
public interface HelloService {
    Person findPerson(int id);
    void sayHelloTo(Person person);
}

@Component
public class AbisHelloService implements HelloService {
    public Person findPerson(int id) {
        return new Person("John");
    }

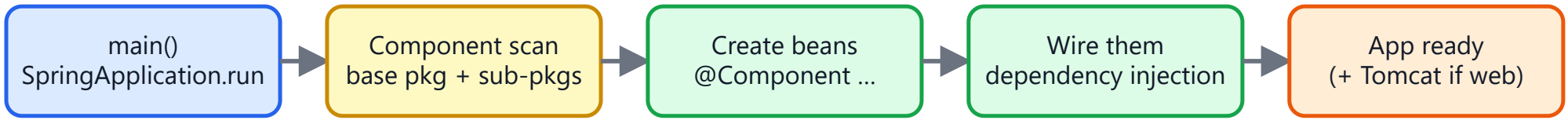
    public void sayHelloTo(Person p) {
        System.out.println("Welcome to Spring, " + p.getFirstName());
    }
}
```

```java
@SpringBootTest
public class HellospringApplicationTests {
    @Autowired private HelloService helloService;  // interface, not the class

    @Test
    void contextLoads() {}  // are all beans registered correctly?

    @Test
    void personNameIsJohn() {
        assertEquals("John", helloService.findPerson(1).getFirstName());
    }
}
```

What happens when the application starts



CHAPTER 3

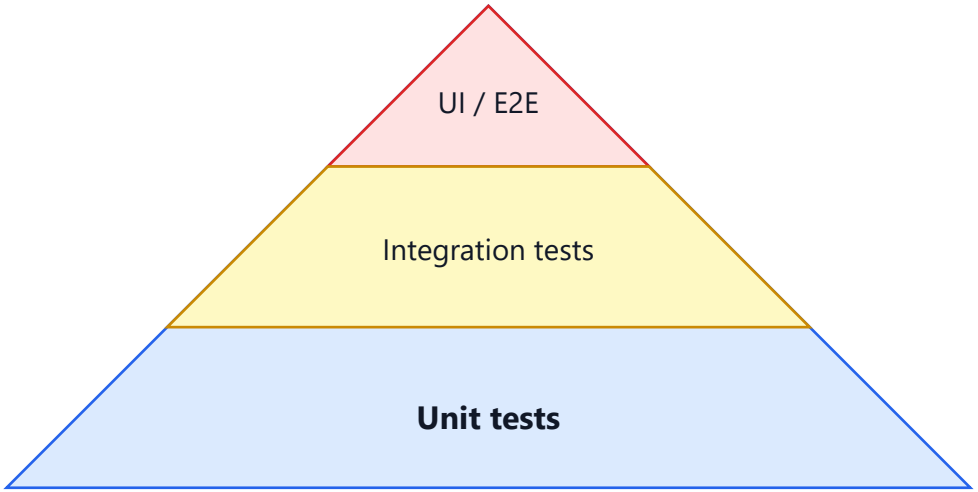
Testing in Spring

Testing principles · JUnit 5 · Mockito · Spring integration testing · test slices · MockMvc

3.1 Testing principles

Types of tests: **code testing** (unit test, integration test), regression testing, performance testing, defect tracking. Tests are based on test plans / scenarios derived from use cases.

Regression testing:
re-run all tests
after every change



few, slow, expensive

many, fast, cheap

Test Driven Design: Red – Green – Refactor



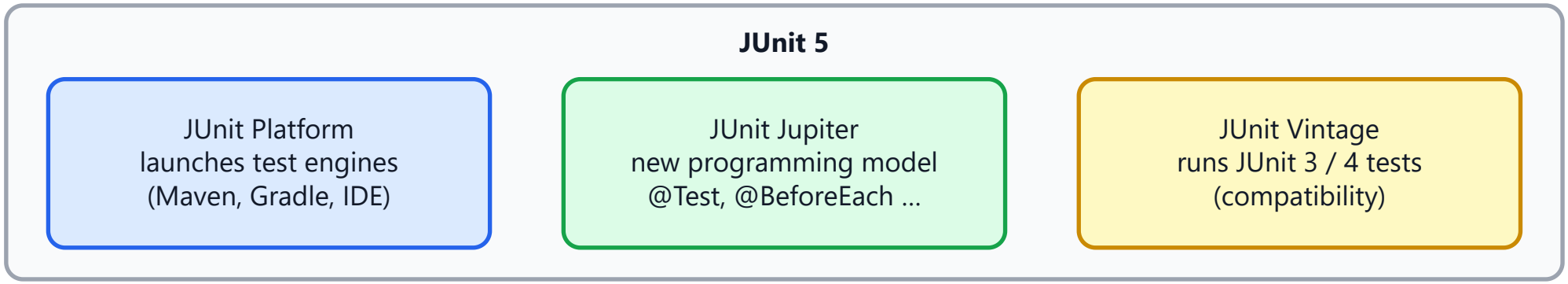
3.2 Spring testing at a glance

Everything is delivered by `spring-boot-starter-test`: JUnit Jupiter, AssertJ, Hamcrest, Mockito and more. `@SpringBootTest` starts the entire application: it creates the application context, wires all beans and loads the configuration. Spring adds features on top of JUnit and Mockito to test HTTP endpoints, test database calls and isolate application layers.

3.3 JUnit

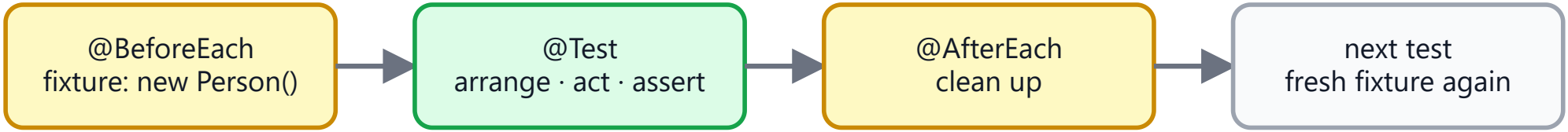
JUnit (`junit.org`) is the open source framework for writing and running unit tests in Java. A **unit test** is low-level: it investigates the behaviour of a single component (a class, repository, controller ...) based on its specification, ideally written *before* the component. A framework makes tests easier to write, run and re-run, and gives consistency, maintainability and automation.

JUnit 5 = Platform + Jupiter + Vintage



- **Platform**: foundation for launching test frameworks on the JVM; defines the `TestEngine` API; Maven/Gradle plugins and console launcher.
- **Jupiter**: new programming model + extension model for writing tests.
- **Vintage**: engine for running JUnit 3 / 4 tests.
- New in 5: lambdas, test interfaces with default methods, nested tests, conditional execution, parameterized tests, custom extensions, repeated and dynamic tests. *Watch out: some annotation names have changed.*

The life of a test: Arrange – Act – Assert




```java
@Test
void nameOfPerson1IsJohn() {
    Person p = new Person("John","Travolta");  // arrange
    String firstName = p.getFirstName();  // act
    assertEquals("John", firstName);  // assert  (static import of Assertions.*)
}
```

Two kinds of reports: **failures** (an anticipated condition is false, e.g. `expected:<Joh[a]n> but was:<Joh[]n>`) and **errors** (an unexpected exception).

Assertions

Type of test	Normal	Negated
condition is true	`assertTrue(boolean)`	`assertFalse(boolean)`
object does not exist	`assertNull(Object)`	`assertNotNull(Object)`
same instance	`assertSame(a, b)`	`assertNotSame(a, b)`
objects are equal	`assertEquals(a, b)` (overloaded for objects, booleans, longs, doubles ...)	`assertNotEquals(a, b)`
exception is thrown	`assertThrows(Ex.class, () -> ...)`	`assertDoesNotThrow(...)`
fail unconditionally	`fail()`	

Test classes and extensions

One test class per class under test; `@Test` before each test method; name methods after the goal (`nameOfPerson1IsJohn()`). The **fixture** is set up and torn down with `@BeforeEach` / `@AfterEach` so tests have no side effects on each other.

Annotation	Purpose
`@Timeout`	fail after a certain time
`@Disabled("reason")`	skip a test method or class
`@TestMethodOrder` + `@Order`	fix the execution order (**disadvised** — tests should be independent)
`@Tag`	group tests for filtering
`@DisplayName`	readable name in the report

3.4 Mockito

Test classes should not depend on the execution of other classes. **Mockito** (MIT license) creates *test doubles*: a mock is a fake implementation of a class or interface whose behaviour you script.



The mock lets you control every answer, including the unusual ones (exceptions, edge values).

```
@SpringBootTest
class HelloServiceTests {
    @Autowired private HelloService helloService;
```

```
    @MockitoBean private Person mockPerson;  // replaces the bean by a mock

    @Test
    void sayHelloToJohn() {
        when(mockPerson.calculateHeightInInches()).thenReturn(70.0);
        when(mockPerson.getFirstName()).thenReturn("John");
        helloService.sayHelloTo(mockPerson);
    }

    @Test
    void tooSmallThrowsException() {
        when(mockPerson.calculateHeightInInches()).thenReturn(62.9);
        assertThrows(NotBigEnoughException.class,
                () -> helloService.sayHelloTo(mockPerson));
    }
}
```

3.5 Spring integration testing

Integration tests exercise application components **together**, without deploying to an external server: databases, messaging, REST APIs and Spring infrastructure. Support comes from `spring-boot-starter-test`, the Spring TestContext Framework, JDBC/transaction support, MVC/WebFlux test support and Testcontainers.

Goals

- **Manage context caching** between tests (load via `@SpringBootTest`, `@ContextConfiguration` or a slice; reuse across tests for speed).
- **Dependency injection** into test classes (`@Autowired`; constructor injection works with Jupiter).
- **Transaction management**: transactional tests roll back by default.

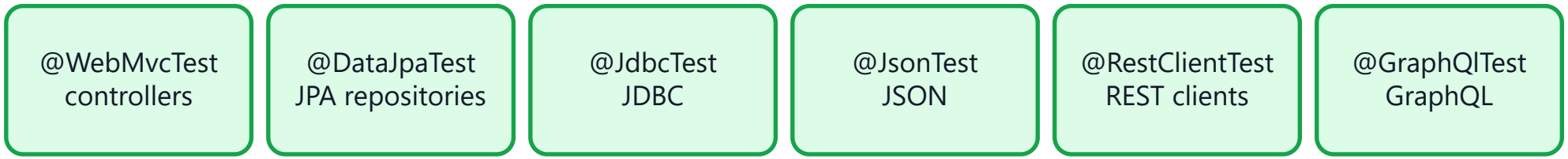


TestContext framework annotations

Area	Annotations
Context loading & caching	`@SpringBootTest` (full context, optional `webEnvironment` to start a web server) · `@ContextConfiguration` (which config classes / XML / initializers) · `@DirtiesContext` (context was changed → reload; slows tests if overused)
Dependency injection	`@Autowired` (constructor injection recommended in Boot 3) · `@MockitoBean` (replace a bean by a mock; reset between tests) · `@MockitoSpyBean` (spy: partial mock, real behaviour kept)
Transactions	`@Transactional` (test runs in a transaction, **rolled back** afterwards) · `@Rollback(false)` · `@Commit` (keep the data)
Profiles & other	`@ActiveProfiles` (e.g. `application-test.properties`), SQL script execution, property source customisation, test execution listeners

Test slices

Slices load only part of the context: faster startup, smaller context, better isolation.



each loads only the beans of that slice

Testing web layers

- **MockMvc**: built on Servlet API mocks, no running container; fast; verifies requests/responses, JSON, form submission and validation.
- **WebTestClient**: for reactive and servlet apps, can test live servers.
- **Selenium / browser testing**: end-to-end UI and workflow tests.

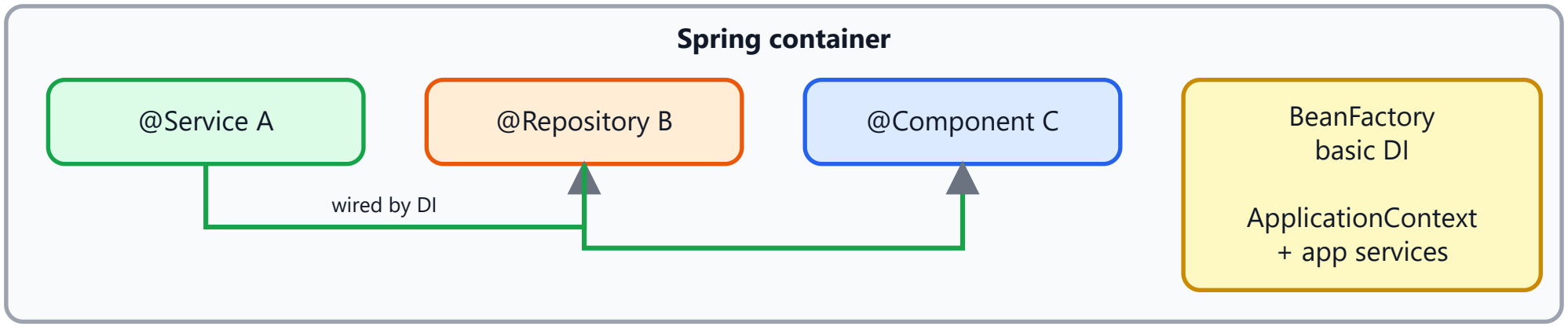
CHAPTER 4

Spring Core

The container · bean lifecycle · declaring and wiring beans · ambiguity · profiles · runtime value injection

4.1 The container

Objects live inside the Spring **container**. The container creates the objects, wires them together, configures them and manages their complete lifecycle, using dependency injection.



- **BeanFactory** (`org.springframework.beans.factory.BeanFactory`): basic, low-level DI support.
- **ApplicationContext** (extends BeanFactory): adds application-framework services; several implementation classes exist. A Spring Boot application **creates one automatically** behind the scenes.

4.2 Bean lifecycle and scopes

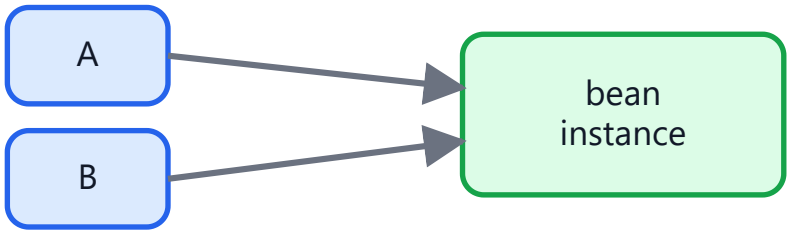
By default Spring Boot uses **eager loading**: all beans are created when the ApplicationContext is created (when you run the application or a test).



Bean scopes

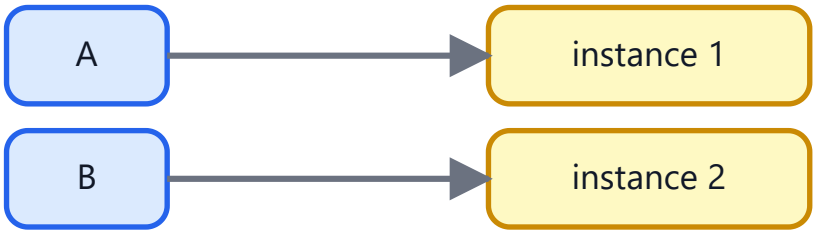
By default beans are **singletons** — the same instance is returned every time, matching the service-oriented nature of Spring.

singleton (default)



same instance for A and B

prototype



new instance for every injection

Scope	Meaning
singleton default	one instance, returned each time
prototype	a new instance every time the bean is injected or retrieved
request / session	web apps: one instance per HTTP request / session
application	one instance for the life of a ServletContext
websocket	one instance during the life of a WebSocket
globalSession	web apps (portlet context only)

4.3 Declaring and wiring beans

Beans managed by Spring must be declared. They are always defined as **interface + implementation**. Historically this was XML; nowadays it is Java based configuration (the Application class in Spring Boot) combined with annotations for automatic discovery. In tests the configuration is loaded by `@SpringBootTest`; different configurations can be combined.

Java based configuration

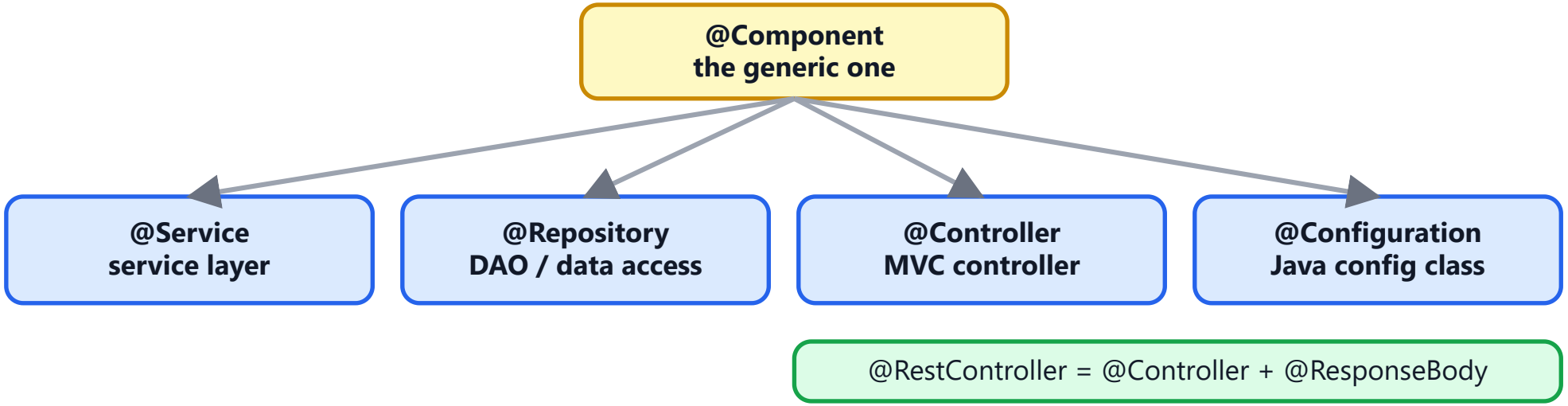
- A class annotated `@SpringBootApplication` (or `@SpringBootTest`) in the base package.
- Beans are discovered through annotations in the base package or its sub-packages, **or** defined explicitly in the configuration class with `@Bean` — both can be combined: *autodiscovery* for your own classes, `@Bean` for **3rd-party** classes.

```java
@SpringBootApplication
public class SpringRestTemplateClientApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringRestTemplateClientApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.build();
    }
}
```

Annotations for autodiscovery & stereotypes

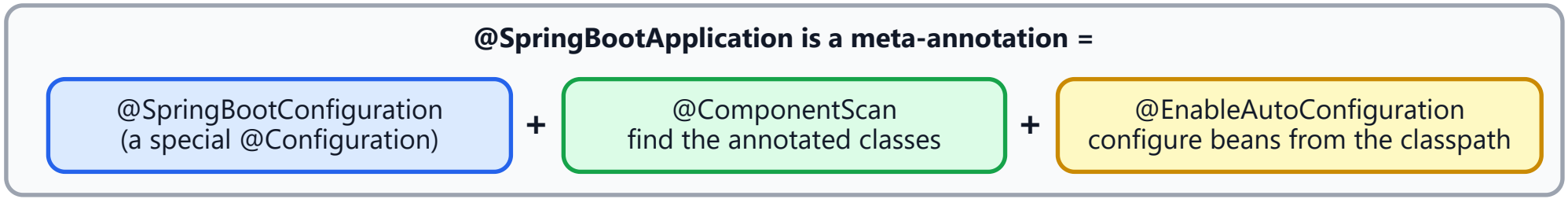
Explicit configuration is verbose, so classes get a **stereotype** annotation (package `org.springframework.stereotype`) and are wired with `@Autowired`. Spring Boot scans the current package and sub-packages; for another base package add `@ComponentScan(basePackages="be.abis.otherpackage")`. (CDI annotations `@Named`/`@Inject` also work but are rarely used.)



Stereotype	Use
`@Component`	most generic; found by classpath scanning and registered as a bean
`@Service`	beans in the service layer (alias for @Component)
`@Repository`	DAO/repository classes (database access layer); has automatic **persistence exception translation**
`@Controller`	MVC controller, usually combined with `@RequestMapping` (`@RestController` for REST)

Meta-annotations

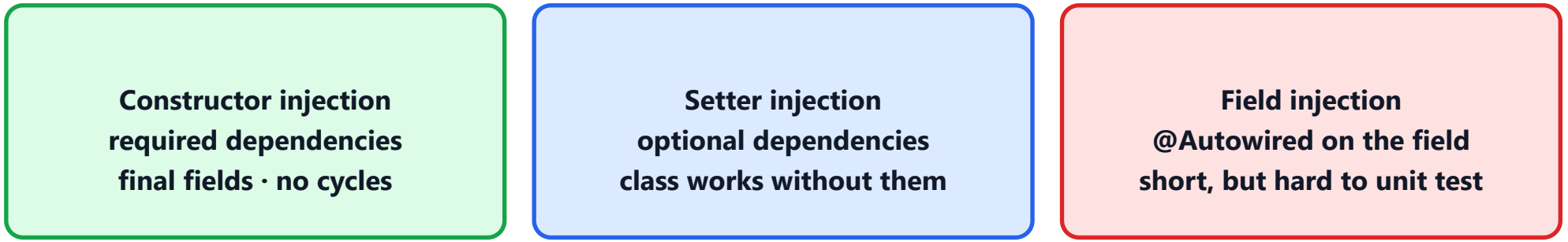
A meta-annotation combines annotations that are often used together and can be applied to other annotations (state the *Target*, the *Retention policy*, add the annotations and imports, then declare `public @interface MyAnnotation`). The best-known example:



Wiring beans with annotations

- `@Value` wires basic properties (String, int, ...); `@Autowired` wires bean properties — **wire the interface, not the implementation class**. It can be put on a constructor, setter, field or any method.
- Alternatives: `@Inject` (CDI, JSR-330) and `@Resource(name="myBean")` (JSR-250, also for JNDI resources).
- `@PostConstruct` / `@PreDestroy` mark the init and destroy methods.

Which type of injection?



recommended in Spring Boot 3: constructor injection (implicit when there is a single constructor)

Type	Use when	Remarks
Constructor	required dependencies	works with `final` fields, avoids circular dependencies; since Spring 4.3 **implicit** if there is only one constructor
Setter	optional / changeable dependencies	the class must still work when the dependency is omitted
Field	shortest code	tightly coupled to the DI container; cannot be instantiated without reflection (e.g. in plain unit tests) — avoid in most cases

Example: a reception with coffee

```
@Component
public class AbisHelloService implements HelloService {
    private String helloMessage;

    @Value("Welcome to Abis")
    public void setHelloMessage(String m) {
        ...
    }
```

```
    public void sayHelloTo(Person p) {
        System.out.println(helloMessage + ", " + p.getFirstName() + ".");
    }
}

@Service
public class AbisCoffeeService implements CoffeeService {
    @PostConstruct
    public void init() {
        System.out.println("preparing coffee");
    }

    public Coffee serveCoffeeToVisitor(Person p) {
        return new Coffee(p.getCoffeePreference());
    }

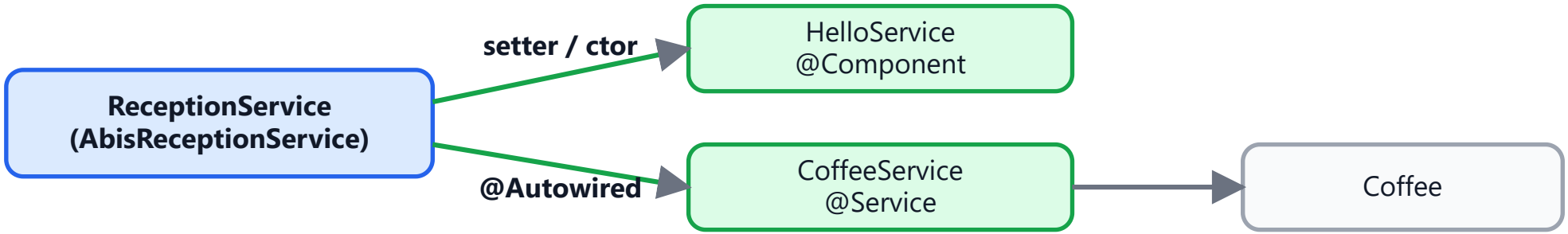
    @PreDestroy
    public void destroy() {
        System.out.println("cleaning coffee things");
    }
}
```

```
@Service
public class AbisReceptionService implements ReceptionService {
    private HelloService helloService;
    @Autowired private CoffeeService coffeeservice;

    // alternative: constructor injection
```

```
    // @Autowired
    // public AbisReceptionService(HelloService h)
    //         { this.helloService = h; }

    public void welcomePersonAndGetCoffee(Person p) {
        helloService.sayHelloTo(p);
        Coffee c = coffeeservice.serveCoffeeToVisitor(p);
        System.out.println("Here is your " + c.getFlavour() + " coffee.");
    }
}
```



4.4 Explicit definition and wiring with @Bean

Instead of annotating the classes, you can define every bean in the configuration class: the **method name** is the bean name, you create the instance with `new` as in plain Java, and you wire beans by calling the methods of previously defined beans (setter or constructor injection). Use `@Bean(initMethod="init", destroyMethod="destroy")` for init/destroy (they must be in the interface too). Take all Spring annotations out of the classes.

```
@SpringBootApplication
public class JavaconfigApplication {
    @Bean
    public HelloService helloService() {
        AbisHelloService ahs = new AbisHelloService();
```

```
        ahs.setHelloMessage("Welcome to Abis");
        return ahs;
    }

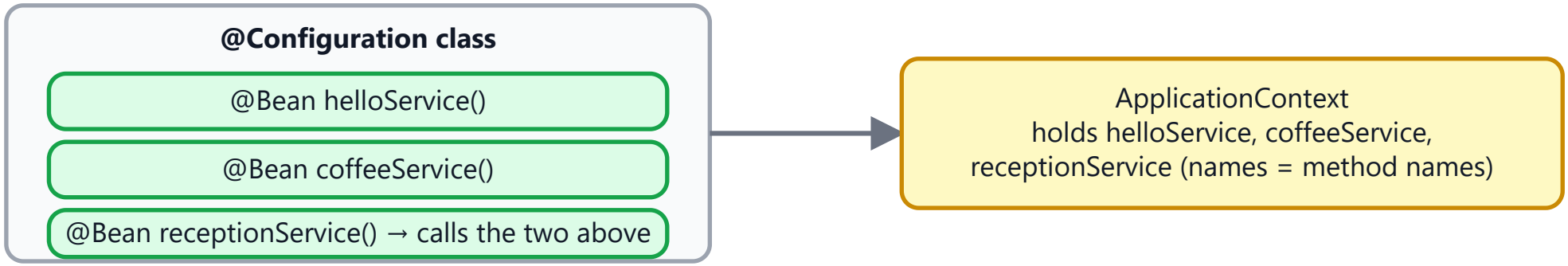
    @Bean(initMethod="init", destroyMethod="destroy")
    public CoffeeService coffeeService() {
        return new AbisCoffeeService();
    }

    @Bean
    public ReceptionService receptionService() {
        // constructor injection
        ReceptionService rs = new AbisReceptionService(helloService());
        // setter injection
        ((AbisReceptionService) rs).setCoffeeservice(coffeeService());
        return rs;
    }
}
```

In a test you can wire the bean with `@Autowired`, or fetch it explicitly: build the context in a `@BeforeAll` method with `new AnnotationConfigApplicationContext(JavaconfigApplication.class)` and call `context.getBean("receptionService")` (by name → cast needed) or `getBean(ReceptionService.class)` (by interface).

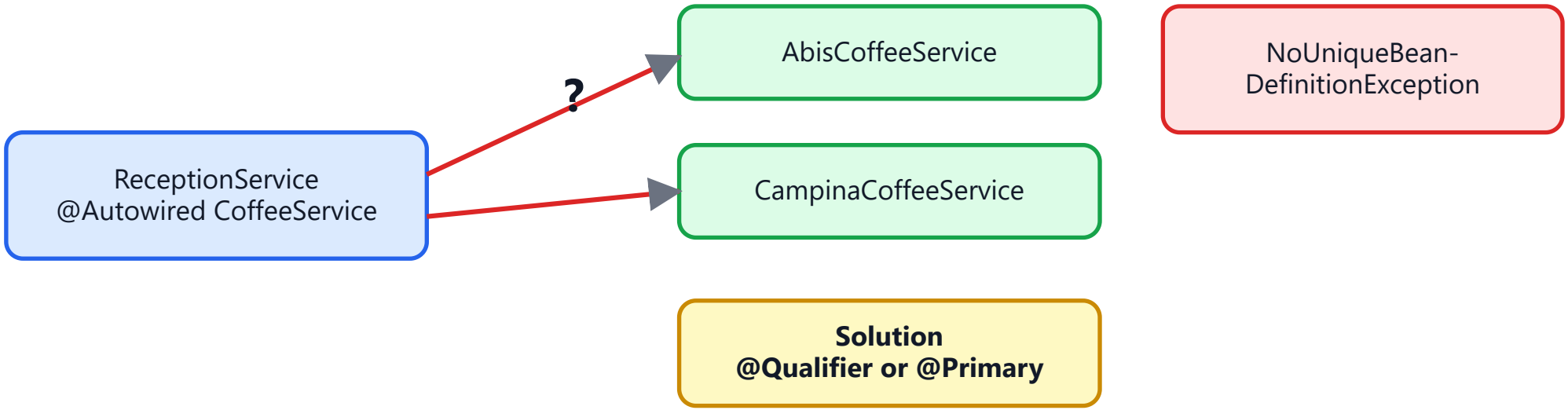
Multiple configuration files

If a Java config class gets too big, split it: extra classes annotated `@Configuration` are discovered automatically by Spring Boot.



4.5 Addressing ambiguity

`@Autowired` uses the **type** to find the implementation. With more than one implementation of the interface Spring throws a `NoUniqueBeanDefinitionException`. Two ways out:



```
// @Qualifier: the injection point chooses
@Service
@Qualifier("campina")
class CampinaCoffeeService implements CoffeeService {
    ..
}

@Service
@Qualifier("Abis")
class AbisCoffeeService implements CoffeeService {
    ..
}

@Autowired
@Qualifier("campina")
private CoffeeService coffeeservice;
```

```
// @Primary: the bean itself is the favourite
@Service
class CampinaCoffeeService implements CoffeeService {
    ..
}

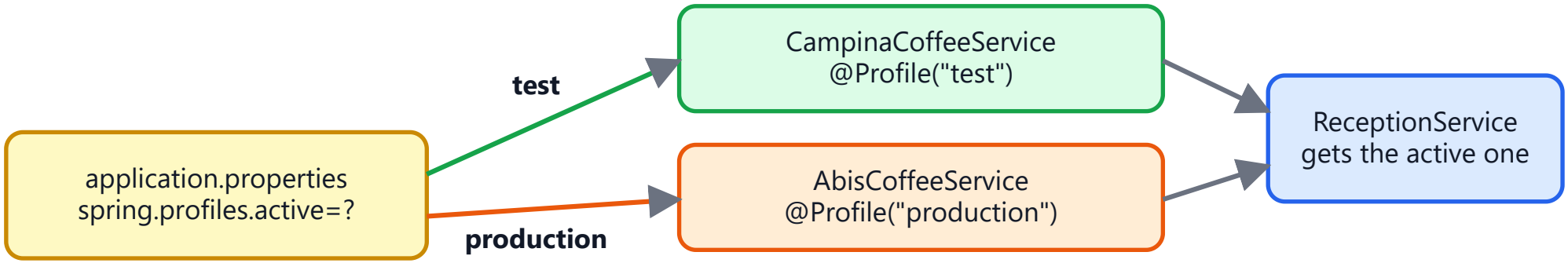
@Service
@Primary
class AbisCoffeeService implements CoffeeService {
    ..
}

@Autowired private CoffeeService coffeeservice;  // gets Abis
```

4.6 Profiles

A **profile** is a named, logical group of bean definitions. The environment decides which profiles are active, so the application **does not need to be rebuilt per environment**.

- in-memory datasource in development vs. JNDI lookup in QA/production;
- monitoring infrastructure only in a performance environment;
- customised beans for customer A vs. customer B.



same code, different beans per environment — no rebuild

Configure / activate	How
On a bean	`@Profile("development")` on a class or `@Bean` method; `@Profile("default")` is used when nothing is active (otherwise `NoSuchBeanDefinitionException`)
Declaratively	`spring.profiles.active=test` in `application.properties` or on the command line
In a test	`@ActiveProfiles("production")` — overrides application.properties
Programmatically	through the `Environment` object; more than one profile can be active at once

4.7 Runtime value injection

To avoid hard-coded values, determine them at runtime through **property placeholders**.




```
# application.properties
spring.profiles.active=production
hello.company=Abis

@Service
public class GeneralHelloService implements HelloService {
    @Value("Welcome to ${hello.company}")
    public void setHelloMessage(String m) {
        ...
    }
}

// a separate file my.properties (application.properties has precedence over it)
@SpringBootApplication
@PropertySource("classpath:my.properties")
public class RuntimevalueinjectionApplication {
```

```
    ...
}

// resolve a property programmatically
@Autowired private Environment env;
...
env.getProperty("hello.company");
```

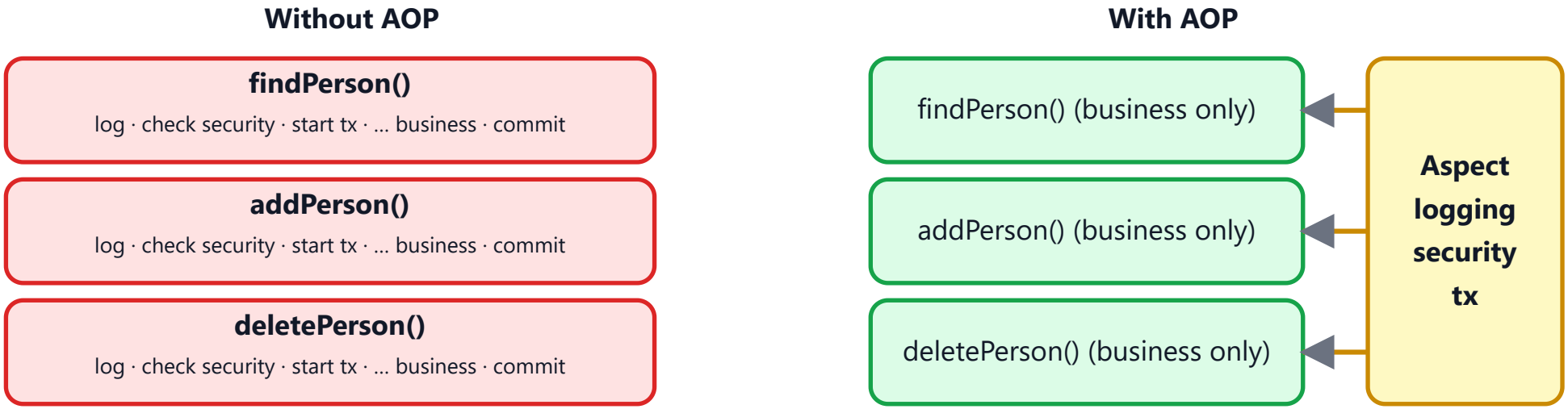
CHAPTER 5

Aspect-Oriented Spring

What is AOP · terminology · Spring's AOP support · pointcuts · advice types

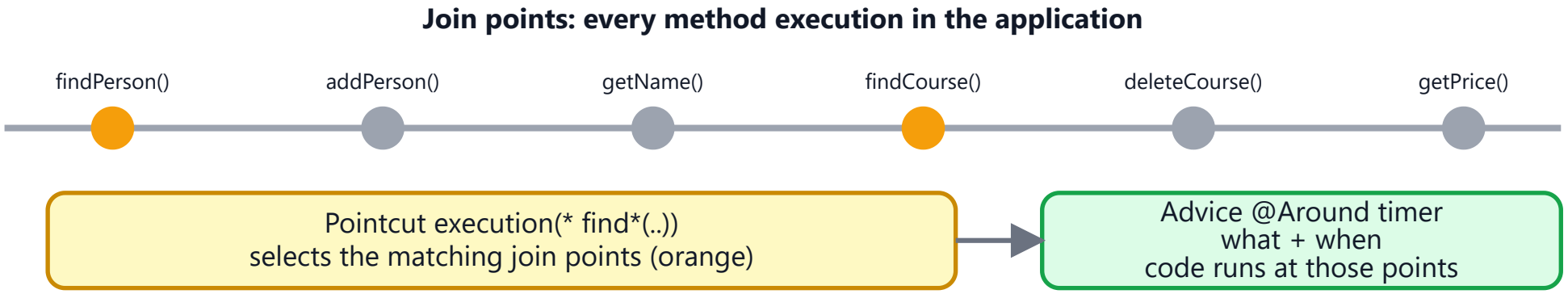
5.1 What is AOP?

Aspect-oriented programming separates *cross-cutting concerns* (logging, transactions, security …) from the business logic. You define **declaratively** how and where the extra functionality is applied; the concern is modularised in a special class — an **aspect**.



Terminology

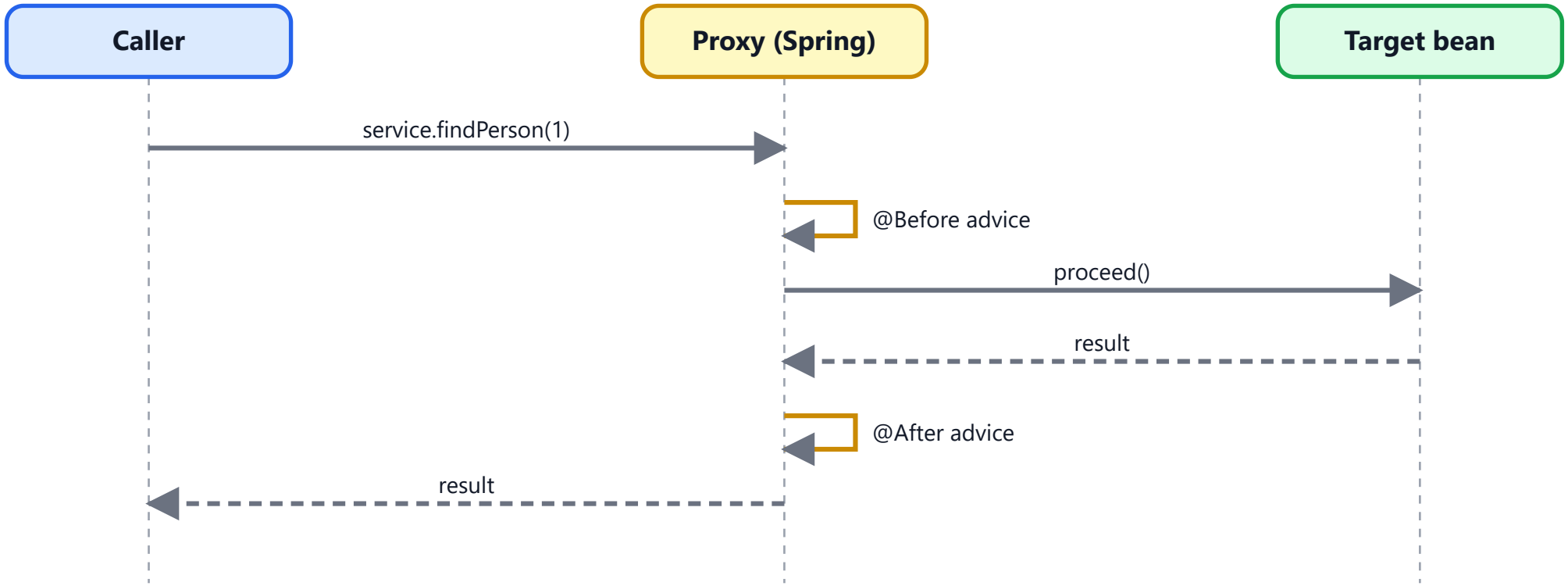
Term	Meaning
Aspect	The general functionality that has to be handled (e.g. timing, logging)
Advice	The *what* and *when* of an aspect. *What:* the code to execute. *When:* before, after, after-returning, after-throwing, around
Join point	A point in the execution where an aspect could be plugged in
Pointcut	Matches the join points where the advice is applied
Weaving	Applying aspects to a (proxied) object — at compile time, class-load time or runtime



Aspect = advice + pointcut together, in one class

Spring's AOP support

- Three major players: **AspectJ**, JBoss AOP, **Spring AOP** (which borrows a lot from AspectJ).
- Advice is written in Java: the aspect is a class, the advices are methods.
- Spring advises objects **at runtime**: weaving is **proxy-based**. Consequences: only methods defined in the interface can be advised, and **methods calling methods in the same class are not advised**.
- Spring only supports **method** join points.
- Advice should not influence the behaviour of the original method!


Caller
Proxy (Spring)
Target bean
service.findPerson(1)
@Before advice
proceed()
result
@After advice
result

5.2 Writing pointcuts

Spring uses the AspectJ **pointcut expression language** (a subset of designators: `args()`, `@args()`, `execution()`, `this()`, `target()`, `@target()`, `within()`, `@within()`, `@annotation()`, plus Spring's `bean()`).

Designator	Example → selects
`within(com.xyz.service.*)`	any method within the service package
`within(com.xyz.service..*)`	...or a sub-package
`target(com.xyz.service.AccountService)`	any method on a target implementing AccountService
`bean(myHelloBean)` / `bean(*Service)`	methods of the bean with that (id-based) name / names matching the wildcard
`args(name)`	pass arguments to the advice (used together with `execution()`)
`execution(...)`	see below — the most used one

execution(public * be.abis.service.*.find* (..))

modifiers
public (optional)

return type
* = any

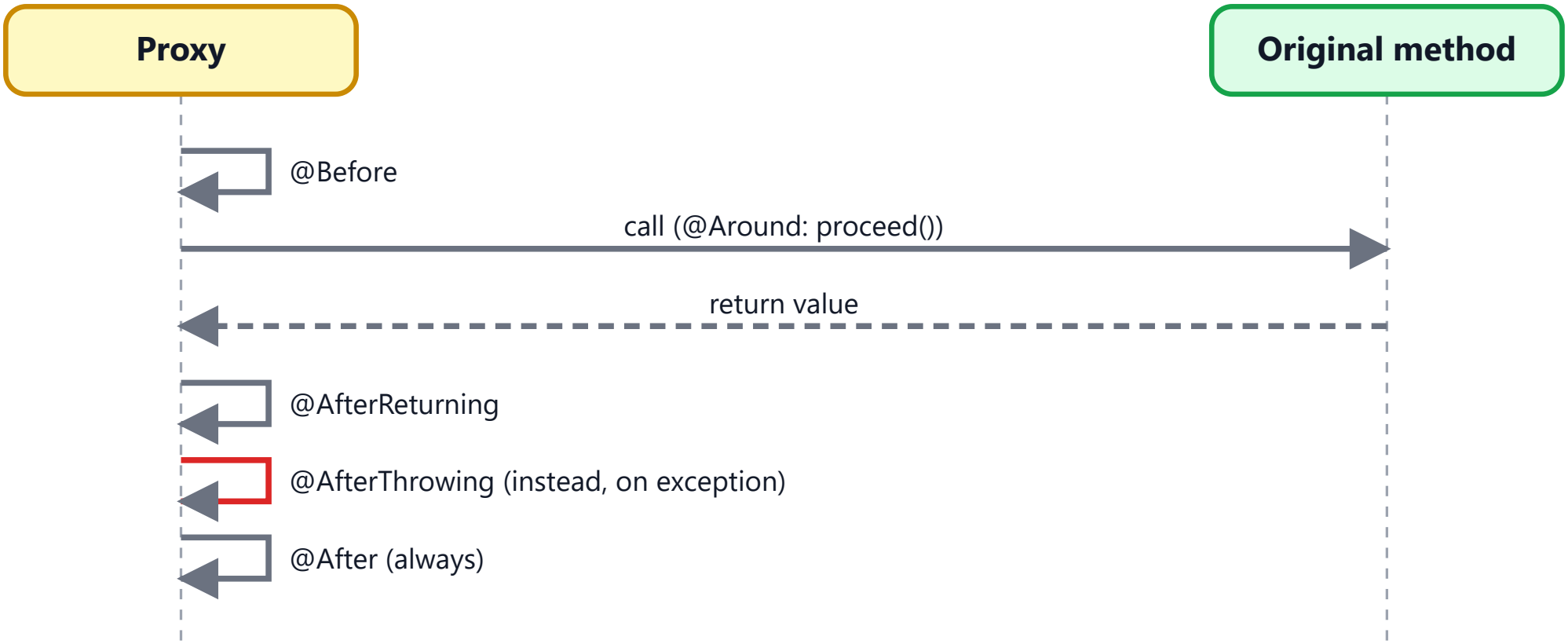
declaring type + name
service package, find*

parameters
(..) = any number

param-pattern	Meaning	Example	Selects
`()`	no arguments	`execution(public * *(..))`	any public method
`(..)`	any number of arguments	`execution(* set*(..))`	methods starting with *set*
`(*)`	one argument of any type	`execution(* Instrument.play(..))`	play method of Instrument
`(*,String)`	two arguments, 2nd a String	`execution(* com.xyz.service.*.*(..))`	any method in the service package

5.3 Defining aspects and advices

Spring uses AspectJ's annotation-oriented model: `@Aspect` turns a class into an aspect, `@Pointcut` defines a pointcut. Add the dependency `spring-boot-starter-aop`; the aspect class itself must also be a bean (`@Component`).



- **@Before** / **@After**: run before / after the original method.
- **@AfterReturning**: after the method returned normally. **@AfterThrowing**: after it threw an exception (and gives access to that exception).
- These advice methods return `void`. A `JoinPoint` can give information about the advised method/class, and has to be the **first** argument.

- **@Around**: one method for before and after — useful when information must be shared. Return type `void` or `Object` (the value of the original method). Uses `ProceedingJoinPoint`; you must call `pjp.proceed()` explicitly.

Pointcuts can be **named and combined**: `@Pointcut("inService() && execPerform()")`, and used from advice as `@Before("MyPointcuts.execPerform()")`.

Example

```
@Component
@Aspect
public class MyAspect {
    @Before("execution(public * be.abis.demo.service.*.welcome*(..))")
    public void beforeWelcome() {
        System.out.println("Receptionist stands up:");
    }

    @AfterReturning("AopPointcuts.execWelcome()")
    public void afterWelcomeOK(JoinPoint jp) {
        System.out.println("Guest is guided to lounge.");
    }

    @AfterThrowing(pointcut="AopPointcuts.execWelcome()", throwing="exc")
    public void afterNotWelcome(GuestNotWelcomeException exc) {
        System.out.println("Escorted back to the door " + exc.getMessage());
    }

    @After("AopPointcuts.execGet()")
    public void afterGetting(JoinPoint jp) {
        System.out.println("finished calling getter: " + jp.getSignature()
                .getName());
    }
```

```
    @Around("AopPointcuts.findThings()")
    // a timer for all find* methods
    public Object timing(ProceedingJoinPoint pjp) {
        Object o = null;
        try {
            long start = System.nanoTime();
            o = pjp.proceed();  // call the original method
            long end = System.nanoTime();
            System.out.println("time taken by " + pjp.getSignature().getName()
                    + " : " + ((end-start)/1000000.0) + " ms");
        } catch (Throwable e) {
            e.printStackTrace();
        }
        return o;
    }
}
```

```
public class AopPointcuts {
    @Pointcut("execution(* welcome*(..))")
    public void execWelcome() {}

    @Pointcut("execution(* be.abis.demo.service.*.get*(..))")
    public void execGet() {}

    @Pointcut("execution(* be.abis.demo.service.*.find*(..))")
    public void findThings() {}
}
```



One @Around advice = @Before + @After in a single method, so the two parts can share data (here: the start time).

Forget pjp.proceed() and the original method is never executed!

Exercise ideas

1. A `LoginAspect` that prints a message after a login, and another one after a *wrong* login (`@AfterReturning` / `@AfterThrowing`). 2. A timer for all `find*` methods in the repository package (`@Around`).

CHAPTER 6

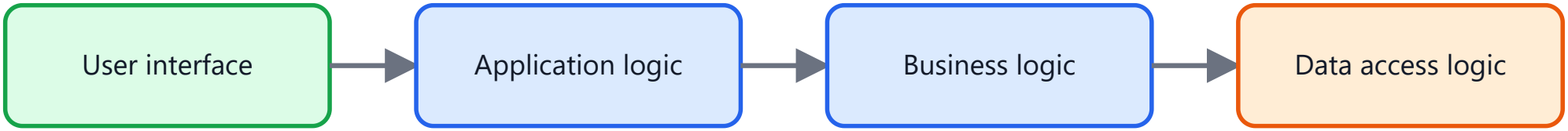
Introduction to REST and JSON

Layers & distributed computing · RESTful web services · the REST constraints · resources and URIs · XML vs. JSON

6.1 Application development and distributed computing

Layers

An application is structured in layers. Think further than just this one application: several interfaces may share the same business layer / domain model. Modular applications avoid «spaghetti code», allow reuse (even of code written in other languages), are easy to maintain and extensible.

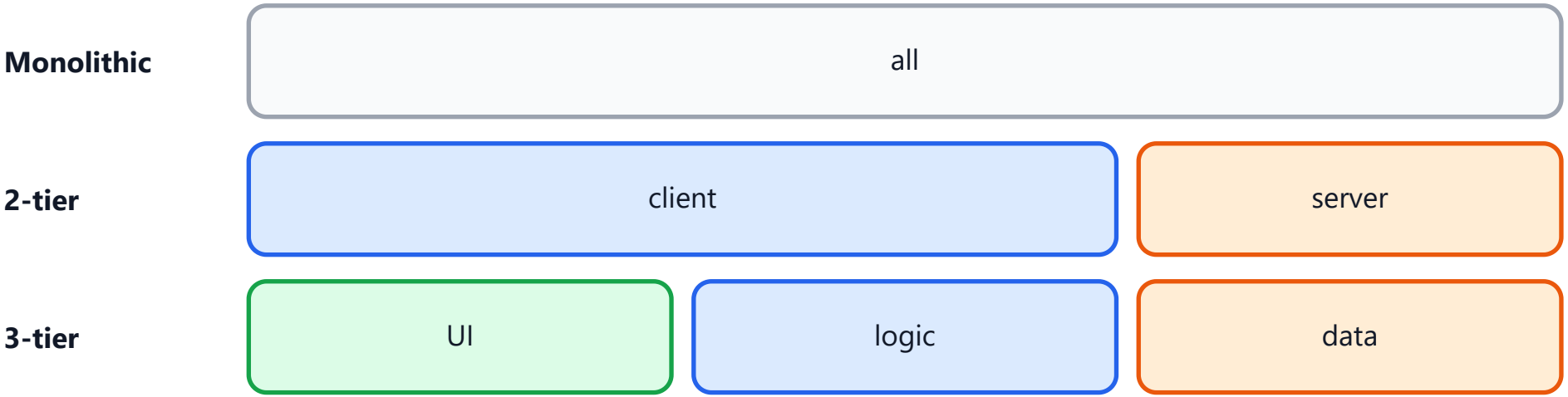


Distributed computing

«The components and objects of an application can be located on different computers connected to a network.»

- **Reuse** (multiple clients)
- **Location transparency** (standard lookup)
- **Reliability / availability** (clustering)
- **Scalability** (powerful servers)
- **Performance** (parallel processing)
- **Extensibility** (dynamic configuration)
- **Productivity** (small programs)
- **Interoperability**, less vendor dependence

Layers vs. tiers — where does each layer run?



Evolution: monolithic → client/server (2-tier, 3-tier, e.g. Java + Oracle) → n-tier distributed components (CORBA, Java RMI, MS DCOM) → fully distributed, heterogeneous systems with independent services over standard internet technology (web / cloud / mobile / IoT).

Communicating objects & data types

Remote communication (remote procedure call) passes a request and a (complex) data structure between two applications: the sender **marshals / serializes / encodes** it, the receiver **unmarshals / decodes** it.



Protocols over time: CORBA (IIOP), MS DCOM, Java RMI → WDDX, XML-RPC, SOAP → **REST**, JSON-RPC.

6.2 RESTful web services / APIs

Before: classic web services with **SOAP and WSDL** — well structured and governable, but complex. From roughly 2010 a shift to more lightweight *APIs*: easier access to services outside the enterprise, more agile, less ceremony; often REST + JSON instead of SOAP + XML; better suited for consumer apps, new channels and mashups (Web 2.0).

What is REST?

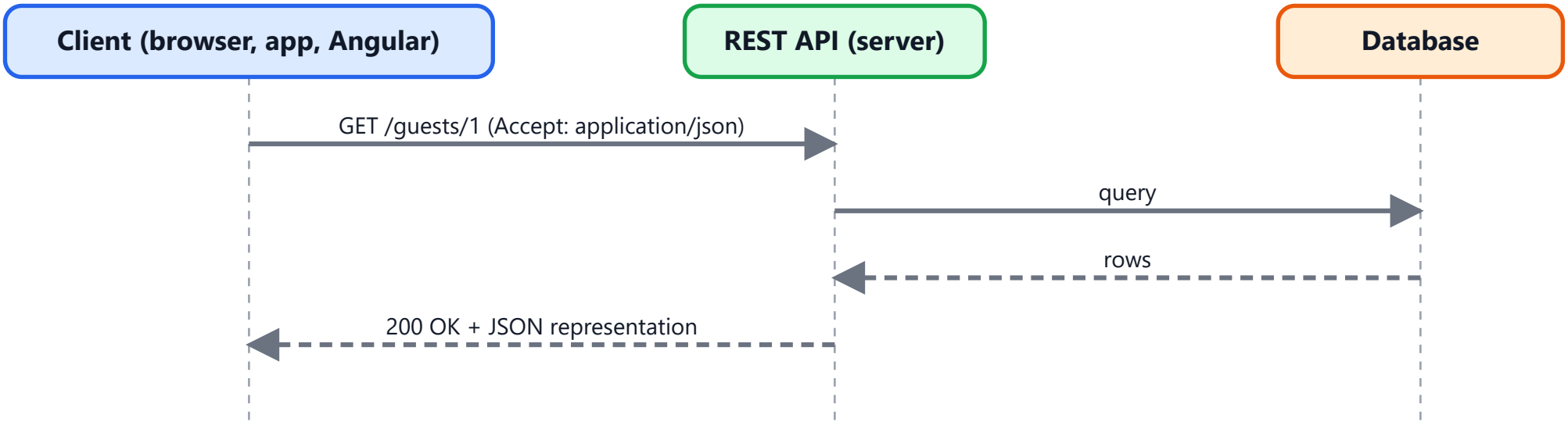
REpresentational **S**tate **T**ransfer.

Representational
resources can be shown
in any form:
XML, JSON, HTML ...

State
the state of a resource
matters more than
the actions on it

Transfer
resource data moves
from one application
to another

- A basic philosophy: **«everything is a resource on the web»**, addressable by a URI, e.g. `http://twitter.com/users?screen_name=james`.
- Uses HTTP with four basic operations (cf. CRUD): GET, PUT, POST, DELETE.
- REST is an **architectural style**, not a protocol like SOAP.



The REST architectural style: six constraints

Client–server
loosely coupled

Stateless
no user session kept by the server

Cacheable
infrastructure may cache responses

Uniform interface
unique address + generic interface

Layered
several implementation layers

Code on demand
(optional) download client code

Resources

A RESTful resource is anything that is **addressable** over the web: a logical, temporal mapping to a concept in the problem domain — a news story, the temperature in Leuven at 4 p.m., a person working for a company, the list of participants of a course. The **URI** is a hyperlink to the resource and should not change over time (it would break the contract between client and server). Different requests can return the same underlying binary representation.

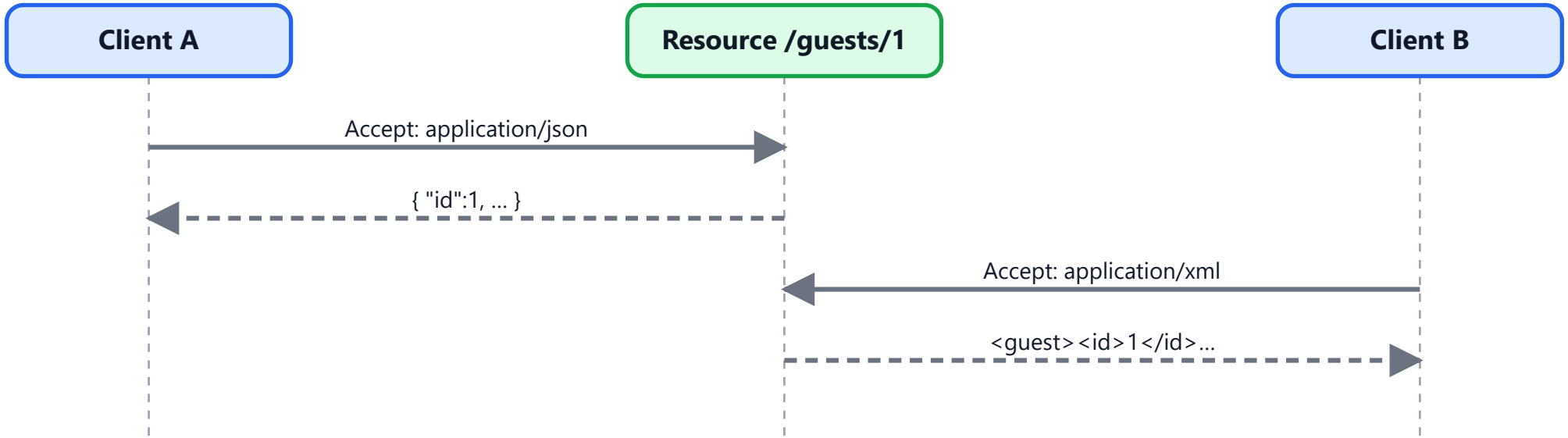
HTTP methods

Data action	CREATE	READ	UPDATE	DELETE
HTTP equivalent	**POST**	**GET**	**PUT** (or PATCH)	**DELETE**
Spring annotation	`@PostMapping`	`@GetMapping`	`@PutMapping`	`@DeleteMapping`

Other methods: **HEAD** and **OPTIONS**.

The representation of resources

A *representation* is what is sent back and forth between client and server: the state of the data at the time of the request, as a binary stream together with **metadata** describing how the client should consume it (image, text, XML, JSON …). All clients use the same URI; the **Accept header** selects the representation. Metadata can also carry extra information: validation, encryption, code to execute at runtime.



6.3 REST resource naming best practices

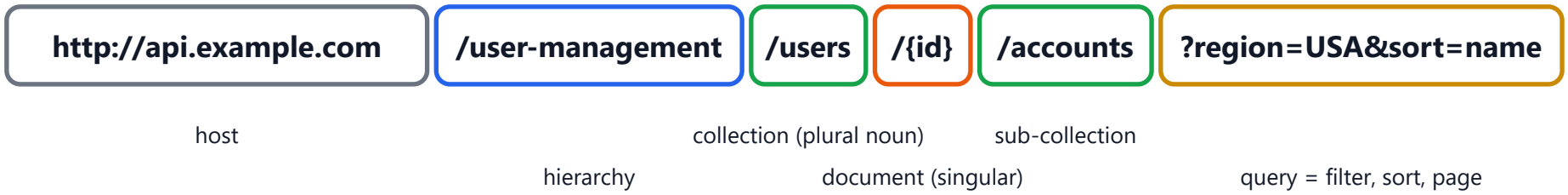
Four rules: use **nouns** for resources · **consistency** is the key · **never** use CRUD function names in URIs · use the **query component** to filter collections.

Four resource archetypes

Archetype	What	Naming	Example
document	a single resource inside a collection	singular noun	`/user-management/users/{id}`
collection	server-managed directory; clients propose new resources, the collection decides	plural noun	`/user-management/users` `/users/{id}/accounts`
store	client-managed repository; the client chooses the URI of what it puts in	plural noun	`/users/{id}/playlists`
controller	a procedural concept: executable function with parameters and return value	**verb**	`/users/{id}/cart/checkout`

Consistency is the key

anatomy of a good URI — nouns, hyphens, lowercase, no extension, no trailing slash



GET /users/7/accounts
= the accounts of user 7

POST /users/7/cart/checkout
= controller archetype → verb allowed

Rule	Not	But
Use `/` for hierarchy, no trailing slash	`/managed-devices/`	`/managed-devices/{id}/scripts`
Hyphens for readability	`managedEntities/{id}/installScriptLocation`	`managed-entities/{id}/install-script-location`
No underscores	`managed_entities`	`managed-entities`
Lowercase (paths are case-sensitive; scheme and host are not)	`/My-Folder/my-doc`	`/my-folder/my-doc`
No file extensions (use the Content-Type / Accept header)	`/managed-devices.xml`	`/managed-devices`

Never use CRUD names in URIs — the HTTP method says what happens

Request	Meaning
`GET /managed-devices`	get all devices
`POST /managed-devices`	create a new device
`GET /managed-devices/{id}`	get the device with that id
`PUT /managed-devices/{id}`	update the device
`DELETE /managed-devices/{id}`	delete the device

Filter with query parameters — do not create new APIs

```
/managed-devices
/managed-devices?region=USA
/managed-devices?region=USA&brand=XYZ
/managed-devices?region=USA&brand=XYZ&sort=installation-date
```

6.4 XML and/vs. JSON

XML

Extensible **M**arkup **L**anguage: a universal way of structuring text documents (XML 1.0, W3C, 1998/2000; cf. SGML, HTML). Independent, interoperable, simple, human readable. Important for data definition (well formed), validation (valid), interchange, hierarchy and namespaces.

JSON

Java**S**cript **O**bject **N**otation: ECMA standard since 2013, open file format for serialization (simpler than XML), also used outside JavaScript: REST, JSON-RPC, Ajax.

XML

```
<CourseParticipants>
  <Participant>
    <firstName>Maria</firstName>
    <lastName>Meuris</lastName>
    <age>52</age>
  </Participant>
  <Participant> … </Participant>
</CourseParticipants>
```

JSON

```
[
  { "firstName": "Maria",
    "lastName": "Meuris",
    "age": 52 },
  { "firstName": "Bert",
    "lastName": "Mak",
    "age": 34 }
]
```

JSON syntax

- Name-value pairs in curly braces `{}` form **objects**; pairs separated by commas, a colon between name and value; **names are strings in double quotes**.
- Values: string, number, object, array, boolean, `null`. Square brackets `[]` hold arrays; curly braces also denote nesting.

CHAPTER 7

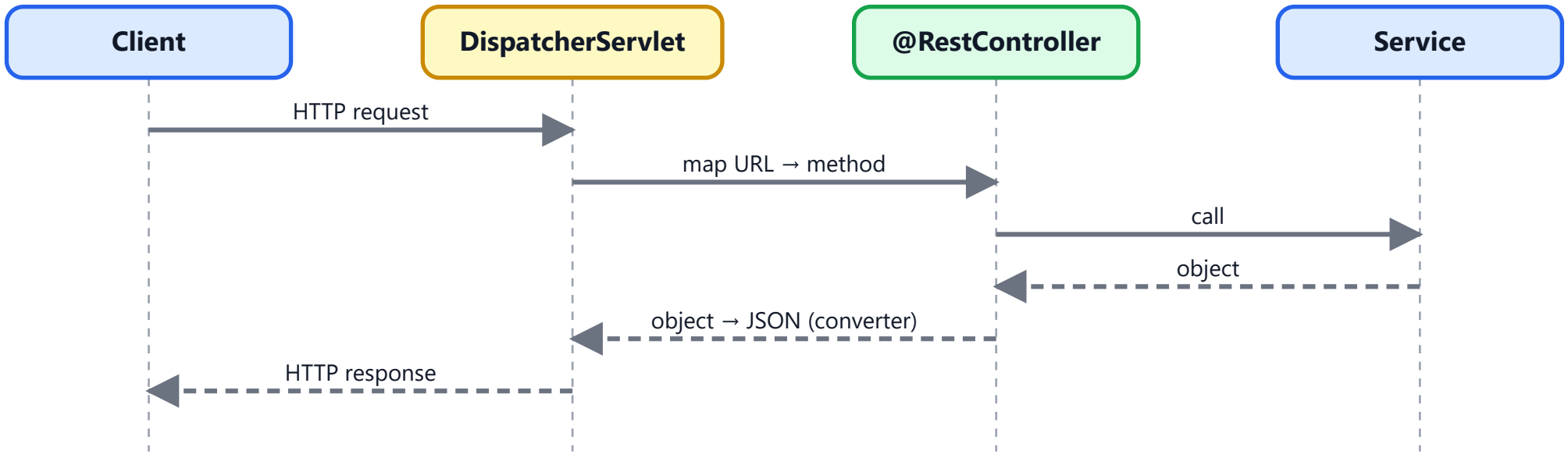
Spring REST: basics and more functionality

Hello World REST API · testing APIs · @RequestMapping · HTTP method mappings · @PathVariable / @RequestParam / @RequestBody · JSON and XML mapping

7.1 Spring REST basics

Spring REST builds upon **Spring MVC**.

- Controllers can handle all HTTP methods (PATCH since Spring 3.2).
- `@PathVariable` lets controllers handle parameterized URLs.
- Resources can be represented in many ways (views and view resolvers: XML, **JSON (default)**, Atom, RSS). View-based rendering can be bypassed with `@ResponseBody` and *HttpMessageConverter* implementations.
- `@RequestBody` + message converters turn inbound HTTP data into Java objects.
- Applications can *consume* REST resources with `RestTemplate` or `WebClient` (chapter 9).



Configuration

Maven dependency `spring-boot-starter-web` (Spring MVC/REST + Tomcat starter + Jackson + spring-web). Packaging **jar** uses the embedded Tomcat; **war** deploys on your own server (extra configuration needed). With Boot auto-configuration you only write the controllers; `application.properties` sets port and context root:

```
server.port=8080
server.servlet.context-path=/helloapi
```

Writing a controller — Hello World

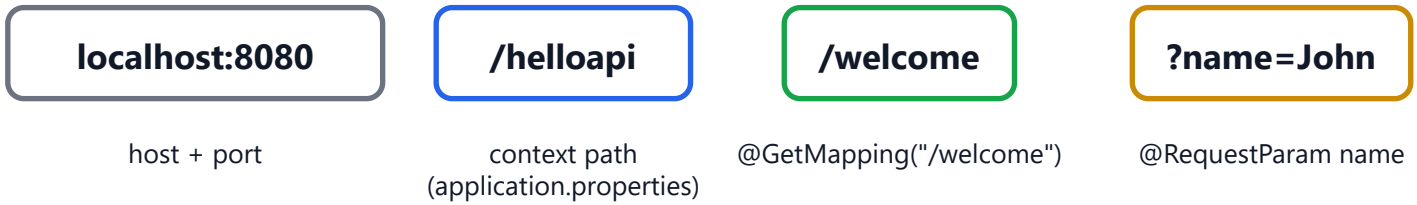
- Annotation-based; the main one is `@RestController`. `@RequestMapping` maps the URL (default method GET; `@GetMapping` is the short form).
- Wire in your service(s); the handler method can have any name.
- Returning a `String` gives `text/plain`; returning any other object gives `application/json`.
- Input: `@RequestParam`, passed as `localhost:8080/helloapi/welcome?name=John`.

```
@RestController
public class HelloRestController {
    @Autowired HelloService helloService;

    @GetMapping("/welcome")
    public String returnMessage(@RequestParam String name) {
        return helloService.sayHelloTo(new Guest(name));
    }

    @GetMapping("/welcomeGuest")
    public Guest returnGuest(@RequestParam int id) {
        return helloService.findGuest(id);
    }
}
```

```
// test:  localhost:8080/helloapi/welcome?name=John
// localhost:8080/helloapi/welcomeGuest?id=2
```



Testing REST APIs

SoapUI

Open-source web-service testing application for SOA and REST (SmartBear). Inspect, invoke, develop, simulate/mock and functionally test services. A commercial Pro version adds productivity features.

Postman

Standalone app (originally a Chrome plugin); more REST-oriented, SOAP is possible with some effort. Needs a free account; Pro and Enterprise add features.

Browser vs. Postman

GET requests can be tested by typing the URI in a browser. POST, PUT and DELETE need a tool such as Postman.

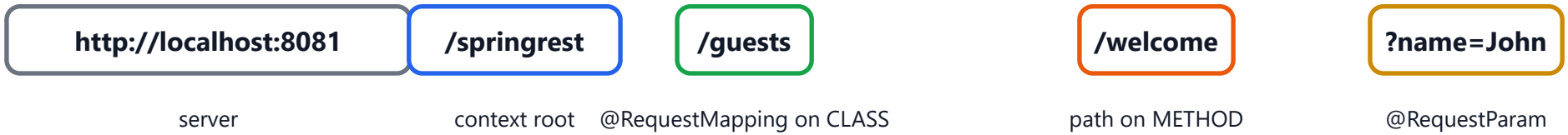
7.2 Defining RESTful resources

@RequestMapping basics

The URI path to which a resource class or method responds, relative to the server URI. It can be placed on the **class** (relative to the base path `http://host:port/<context-root>`) and on the **method** (relative to the class path).

```
@RestController
@RequestMapping("guests")
public class RestReceptionController {
    @RequestMapping(path = "/welcome")
    public String returnMessage(@RequestParam("name") String name) {
        ...
    }
}

// URI: http://localhost:8081/springrest/guests/welcome?name=John
```



Path variables are placeholders in curly braces, linked with `@PathVariable` and replaced at runtime:

```
@RequestMapping("{id}")
public Guest returnGuestById(@PathVariable("id") int id) {  // /springrest/guests/2
    ...
}
```

7.3 Mapping the HTTP request methods

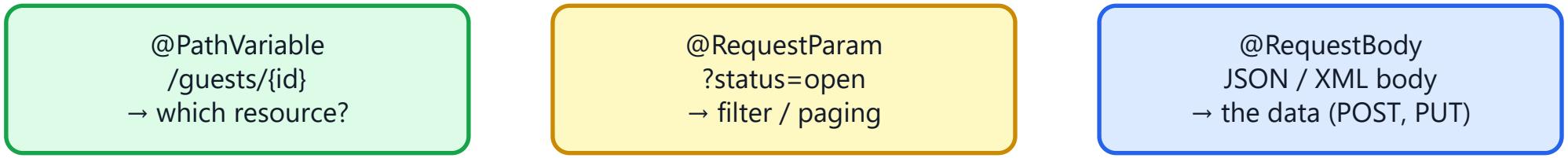
Two ways: `@RequestMapping(path="", method=RequestMethod.GET)` or the short syntax (since Spring 4.3) `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`.



Annotation	Purpose & notes	Example
`@GetMapping`	retrieve resources; often with request parameters; returned object becomes JSON	`@GetMapping("{id}")` `@GetMapping("")` (all) `@GetMapping("/query")` + `@RequestParam`
`@PostMapping`	send a new object to the server, linked with `@RequestBody`; the incoming JSON is transformed to the object type — **a no-arg constructor must exist**; return void. Cannot be tested by typing the URI in a browser	`@PostMapping("") void addGuest(@RequestBody Guest g)`
`@PutMapping`	update the resource the URI points to; message body carries the payload; the **whole object** is passed (alternative: `@PatchMapping` for partial updates)	`@PutMapping("{id}")`
`@DeleteMapping`	delete a resource; pass its id	`@DeleteMapping("{id}")`

7.4 Accessing request data

http://localhost:8081/springrest/guests/1/orders?status=open + body



URL path → identifies · query string → narrows · body → carries the payload

@PathVariable vs. @RequestParam — when to use which?

No official standard, but good practice; they can be combined.

@PathVariable

- link to a **specific resource**
- denote **hierarchy**
- links are easily bookmarked and cached
- parameters are mandatory

@RequestParam

- for **filtering, pagination**
- non-hierarchical data
- required *unless* a `defaultValue` is given

```
@GetMapping(path = "/query")
public Guest returnGuestByName(@RequestParam(value="name",
        defaultValue="An") String name) {
    ...
}
```

@RequestBody

Binds a method parameter to the **body** of the request: Spring deserializes the JSON into the Java type. By default the type must correspond to the JSON sent by the client; other media types can be specified with `consumes`.

7.5 Returning responses

`@ResponseBody` tells the controller that the returned object is serialized (JSON) into the HttpResponse. It is **not needed any more** because `@RestController = @Controller + @ResponseBody`. Other media types: the `produces` attribute.

7.6 More JSON mapping (Jackson)

Three techniques to modify the JSON outcome (Jackson is included in `spring-boot-starter-web`): **annotations**, **wrapper elements** for lists, and programmatic mapping with the **ObjectMapper**.

Jackson annotations (`com.fasterxml.jackson.annotation`)

Category	Annotations
Property naming & inclusion	`@JsonProperty`, `@JsonIgnore`, `@JsonIgnoreProperties`
(De)serialization details	`@JsonFormat`, `@JsonUnwrapped`, `@JsonPropertyOrder`, `@JsonRootName`

```
@JsonRootName("guest")
@JsonPropertyOrder({"id","name","enroldate"})
@JsonIgnoreProperties(ignoreUnknown=true)
public class Guest {
    private int id;
    @JsonProperty("first") private String name;
    @JsonIgnore private int age;
    @JsonFormat(pattern="dd/MM/yyyy") private LocalDate enrolDate = LocalDate.now();
}
```

```
{ "guest": {
    "id": 1,
    "first": "John",
    "enrolDate": "11/06/2019"
} }
```

root name = @JsonRootName, "first" = @JsonProperty,
age hidden by @JsonIgnore, date by @JsonFormat

Adding a wrapper element around a list

A JSON list gets no wrapper element by default. Create an extra class holding the list, annotate it with `@JsonRootName`, add (de)serialization settings and return the wrapper from the controller (this also applies to JAXB).

```
@JsonRootName("guests")
public class Guests {
    private List<Guest> guests;
    ...
}

# application.properties
spring.jackson.serialization.WRAP_ROOT_VALUE=true
spring.jackson.deserialization.UNWRAP_ROOT_VALUE=true
spring.jackson.serialization.indent-output=true

@GetMapping(path="all")
public Guests returnallGuestsViaWrapper() {
    return new Guests(receptionService.findAllGuests());
}
```

Jackson ObjectMapper (`com.fasterxml.jackson.databind`)

(De)serializes a JSON string to/from an object **programmatically**: `readValue()`, `writeValue()`, `writeValueAsString()`, `configure()` for extra options; lists via `new TypeReference<List<Guest>>(){}`.

```
ObjectMapper om = new ObjectMapper();
om.configure(DeserializationFeature.UNWRAP_ROOT_VALUE, true);
Guest g = om.readValue(jsonString, Guest.class);  // JSON → object

om.configure(SerializationFeature.WRAP_ROOT_VALUE, true);
String s = om.writeValueAsString(g);  // object → JSON
```

7.7 Producing and consuming XML

Use the `produces` and `consumes` attributes of the mapping annotations (`consumes` mostly with POST/PUT; both can also be on the class — method-level overrides class-level). Add `jackson-dataformat-xml`. The Jackson mappings are re-used, but can give «strange» results, so extra mappings map Java objects to the correct XML representation.

JAXB and Jackson's JAXB-style annotations

JAXB (Java Architecture for XML Binding) marshals/unmarshals XML to/from Java types using annotations (`@XmlRootElement`, `@XmlAccessorType`, `@XmlElement`, `@XmlTransient` ...) and is supported automatically in Spring REST. Jackson adds its own: `@JacksonXmlRootElement`, `@JacksonXmlProperty`, `@JacksonXmlElementWrapper`; JSON annotations also work for XML.

```
@JacksonXmlRootElement(localName="persons")
public class Persons {
    @JacksonXmlProperty(localName="person")
    @JacksonXmlElementWrapper(useWrapping=false)
    private ArrayList<Person> persons;
}

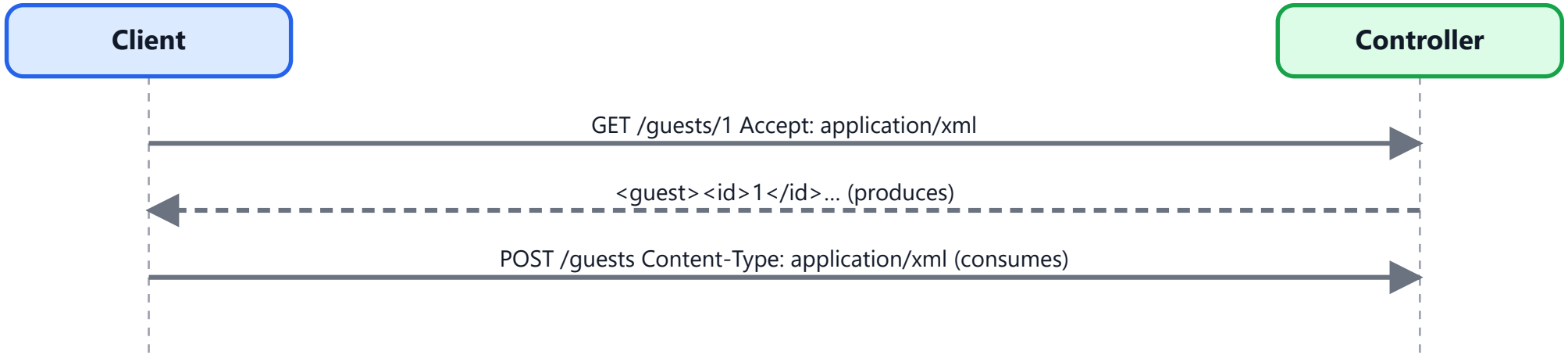
@JacksonXmlRootElement(localName="person")
public class Person {
    private String firstName;

    @JacksonXmlProperty(localName="lastname")
    private String lastName;
}
```

```java
@RestController
@RequestMapping(path="guests", produces="application/json")
public class RestReceptionController {
    @GetMapping(path="{id}", produces=MediaType.APPLICATION_XML_VALUE)
    public Guest returnGuestById(...) {  // XML
        ...
    }

    @PostMapping(path="", consumes=MediaType.APPLICATION_XML_VALUE)
    public void addGuest(@RequestBody Guest g) {
        ...
    }
}
```

Test: `/guests/query?name=John` returns JSON (class default), `/guests/1` returns XML; the **Accept** header states which media type to return; POST an XML body such as `<guest><id>3</id><name>James</name><age>25</age></guest>`.



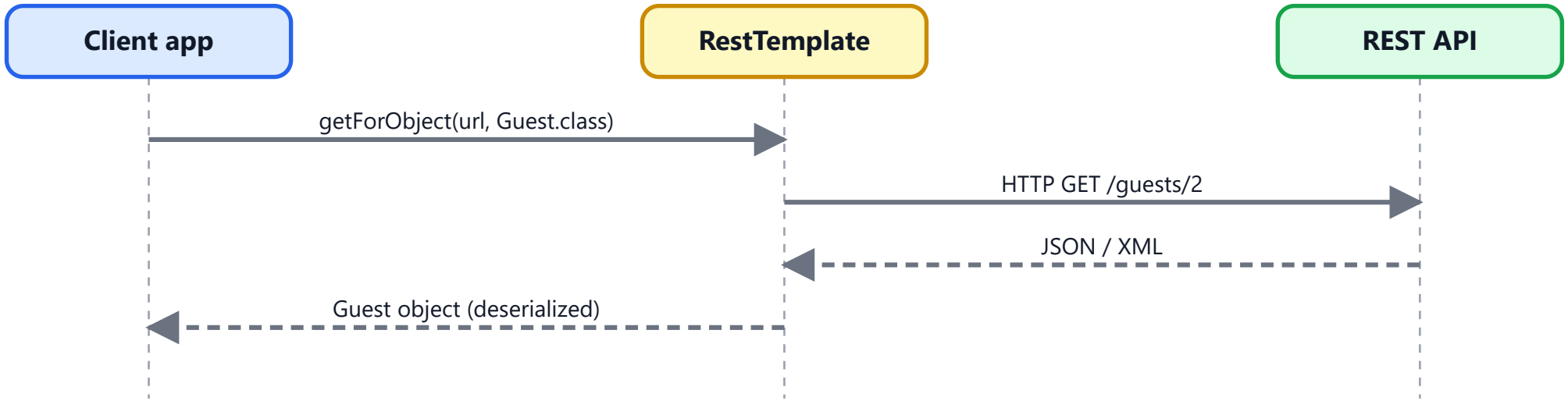
CHAPTER 8

Creating REST Clients

Project settings · RestTemplate · getting, posting, updating and deleting data

8.1 Project settings

A client does not need to run on a server (although it probably will, if you attach Thymeleaf/HTML/JavaScript/Angular to it). Dependencies: `spring-boot-starter-web` and `spring-boot-starter-test` (test scope). The **same model** (including mappings) as in the REST API has to be present in the client.



8.2 RestTemplate

`org.springframework.web.client.RestTemplate` is designed to call a REST API. Its main methods follow the HTTP methods (HEAD, GET, POST, PUT, DELETE). It defines **36 methods** (11 unique operations), e.g. `getForObject()`, `postForObject()`, `delete()`, `exchange()`. Wire it into a controller (web client), a service class or a JUnit test; configure it as a bean:

```
@SpringBootApplication
public class SpringRestTemplateClientApplication {
    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.build();
    }
}

@SpringBootTest
public class SpringRestTemplateClientApplicationTests {
    @Autowired private RestTemplate rt;
    private String baseUrl = "http://localhost:8081/springrestxml/guests";
}
```

Which RestTemplate method for which HTTP call?

HTTP	simple call	variant
GET	getForObject(url, Class)	getForEntity() → ResponseEntity
GET + headers / params	exchange(..., HttpMethod.GET, entity, Class)	UriComponentsBuilder
POST	postForObject(url, obj, Class)	postForEntity() → ResponseEntity
PUT	put(url, obj)	exchange() for headers
DELETE	delete(url)	—

8.3 Getting data

getForObject()

Parameters: the URI (based on the baseUrl) and the class of the object to return. The model classes matching the JSON/XML must be on the classpath; `getForObject` deserializes the response into Java.

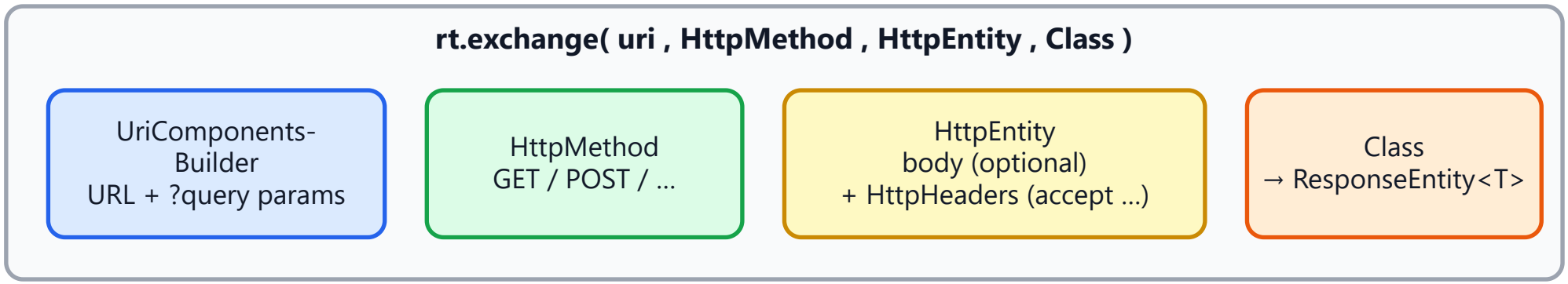
```
@Test
public void testFindGuestById() {  // JSON behind the scenes
    Guest g = rt.getForObject(baseUrl + "/" + 2, Guest.class);
    assertEquals("An", g.getName());
}

@Test
public void testFindAllGuests() {  // XML behind the scenes
    Guests g = rt.getForObject(baseUrl, Guests.class);
    assertEquals(2, g.getGuests().size());
}
```

exchange() — more possibilities

Needed when passing **request parameters** or **headers** (e.g. to choose the media type that comes back). Parameters: a `UriComponentsBuilder` (URI + query params), the `HttpMethod`, an `HttpEntity` (built with an `HttpHeaders` object) and the class of the returned object.

```
@Test
public void testFindGuestByName() {
    UriComponentsBuilder uriBuilder = UriComponentsBuilder.fromHttpUrl(baseUrl
            + "/query").queryParam("name", "John");
    HttpHeaders requestHeaders = new HttpHeaders();
    requestHeaders.add("accept", MediaType.APPLICATION_JSON_VALUE);
    HttpEntity<Guest> requestEntity = new HttpEntity<>(requestHeaders);
    ResponseEntity<Guest> g = rt.exchange(uriBuilder.toUriString(), HttpMethod.GET,
            requestEntity, Guest.class);
    assertEquals("John", g.getBody().getName());
}
```



8.4 Posting data

Create a new instance (e.g. from a web form) and call `postForObject()` with: the URI, the object to send, and the type that comes back (or `Void.class`). The object is transformed to the requested media type (possibly via JAXB). You may send an `HttpEntity` (object + headers) instead of the plain object.

```
Guest g = new Guest(3, "James", 25);
rt.postForObject(baseUrl, g, Void.class);

HttpHeaders requestHeaders = new HttpHeaders();
requestHeaders.add("content-type", MediaType.APPLICATION_JSON_VALUE);
// with headers
rt.postForObject(baseUrl, new HttpEntity<>(g, requestHeaders), Void.class);
```

8.5 Updating data

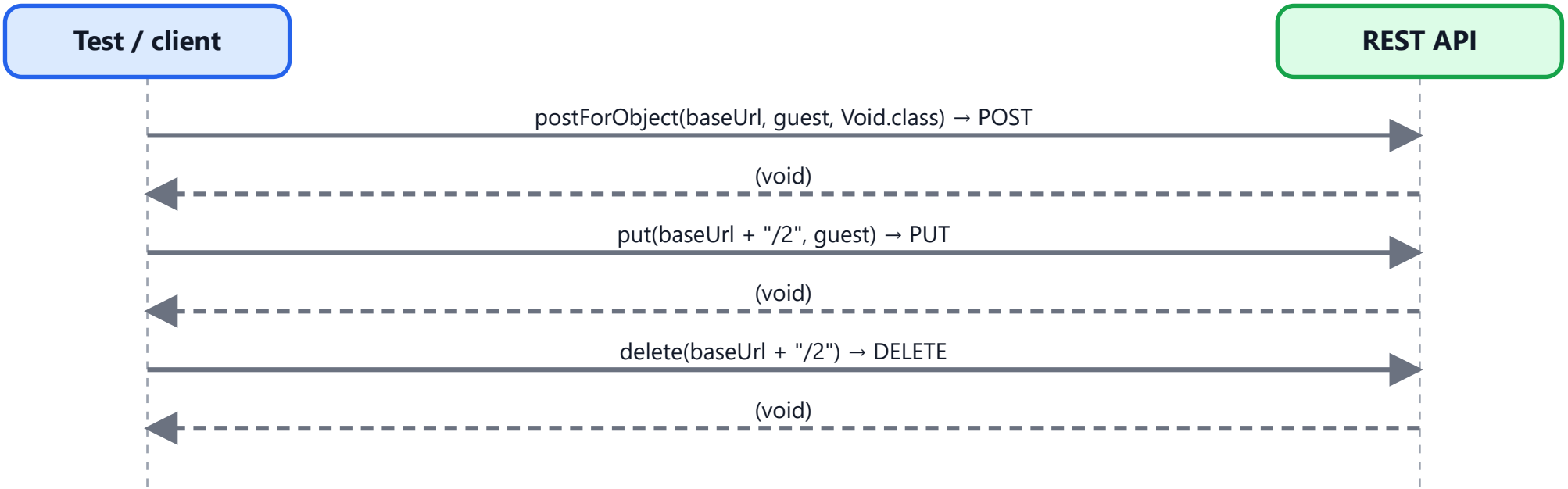
Change the instance, then call `put(uri, object)`; returns void. Use `exchange()` if you need to pass headers or parameters.

```
Guest g = new Guest(2, "xxxxx", 32);
rt.put(baseUrl + "/" + g.getId(), g);
```

8.6 Deleting data

`delete(uri)` — the only parameter is the URI of the object to delete; returns void.

```
rt.delete(baseUrl + "/2");
```


Test / client
REST API
postForObject(baseUrl, guest, Void.class) → POST
(void)
put(baseUrl + "/2", guest) → PUT
(void)
delete(baseUrl + "/2") → DELETE
(void)

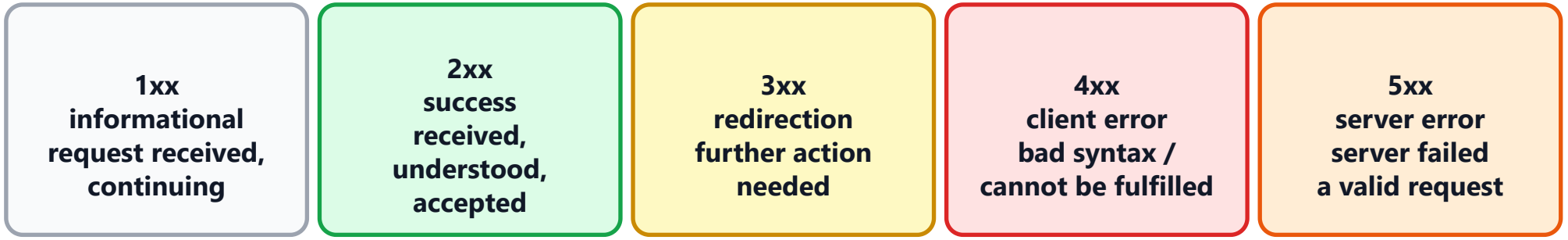
CHAPTER 9

Exception Handling and Validation

HTTP status codes · ResponseEntity · error objects · centralised exception handling · bean validation

9.1 HTTP status codes

Not Java, but necessary to understand for exception handling in services/clients, for notifying the client that a POST/PUT/DELETE was processed, and for debugging. Five categories:



2xx — success

Code	Meaning
200 OK	standard success. GET: contains the requested entity. POST: contains an entity describing the result
201 Created	request fulfilled, a new resource was created
202 Accepted	accepted for processing, but not completed (may still be disallowed later)
203 Non-Authoritative Information	a transforming proxy returned a modified version of the origin's 200
204 No Content	processed successfully, nothing to return
205 Reset Content	like 204, but the requester must reset the document view
206 Partial Content	only part delivered because of a *Range* header (resume or split downloads)
207 Multi-Status	XML body with several separate response codes

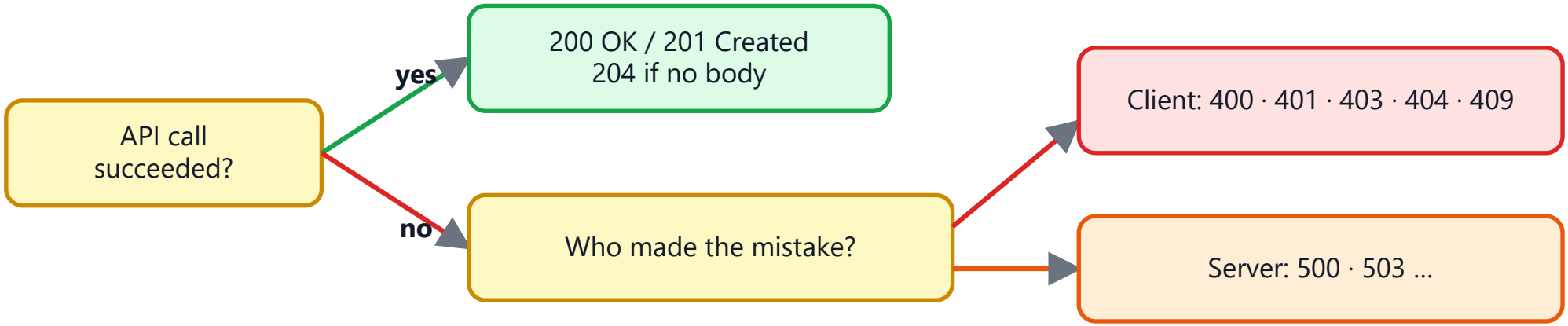
4xx — client errors

Code	Meaning
400 Bad Request	apparent client error (malformed syntax, too large…)
401 Unauthorized	means *unauthenticated*: authentication required and failed/not provided; response must include a `WWW-Authenticate` header
402 Payment Required	reserved (e.g. used when a developer exceeded a daily API limit)
403 Forbidden	valid request, but the server refuses (missing permissions / account)
404 Not Found	resource not found (maybe available in the future)
405 Method Not Allowed	method not supported for that resource (GET on a POST-only form, PUT on read-only)
406 Not Acceptable	can only generate content that the *Accept* headers do not allow
407 / 408	proxy authentication required / request timeout
409 Conflict	conflict with the current state of the resource (e.g. edit conflict)
410 Gone	intentionally removed, will not return
411 / 412 / 413 / 414	length required / precondition failed / payload too large / URI too long
415 Unsupported Media Type	the request's media type is not supported
418 I'm a teapot	IETF April Fools' joke (1998), used as an Easter egg
422 Unprocessable	well-formed, but semantic errors

Entity	
429 Too Many Requests	rate limiting
431 / 451	header fields too large / unavailable for legal reasons (Fahrenheit 451)

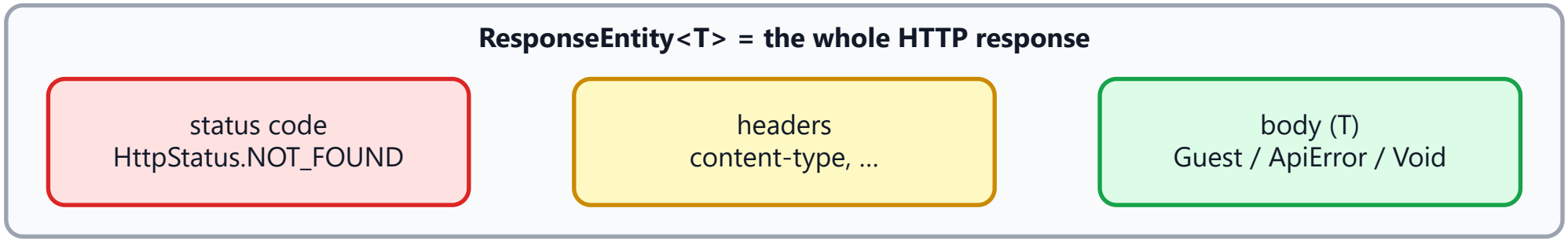
5xx — server errors

Code	Meaning
500 Internal Server Error	generic unexpected condition
501 Not Implemented	method not recognised / not (yet) supported
502 Bad Gateway	gateway/proxy got an invalid upstream response
503 Service Unavailable	overloaded or in maintenance, usually temporary
504 / 505 / 507 / 508 / 511	gateway timeout / HTTP version not supported / insufficient storage / loop detected / network authentication required



9.2 The ResponseEntity class

`ResponseEntity` represents the **whole HTTP response**: status code, headers and body. It is generic (any body type, even `Void`) and can be created with a builder. It is used in @RestController methods *and* in the client (returned from `getForEntity()`, `postForEntity()` and `exchange()`); main methods `getBody()`, `getStatusCode()`.




```
@GetMapping(path = "{id}")
public ResponseEntity<Guest> returnGuestById(@PathVariable("id") int id) {
    Guest g = receptionService.findGuest(id);
    return new ResponseEntity<Guest>(g, HttpStatus.OK);
}

@PostMapping(path = "", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> addGuest(@RequestBody Guest guest) {
    receptionService.addGuest(guest);
    return ResponseEntity.noContent().build();  // 204
}

// client
ResponseEntity<Guest> g = rt.getForEntity(baseUrl + "/2", Guest.class);
System.out.println(g.getStatusCode());
Guest guest = g.getBody();
```

9.3 Exception handling

If nothing is found, an empty page is returned with status **200 (OK)** — but it should be **404** with an error message in the body. Spring offers three options: **error objects** that carry the message, **ResponseEntity** carrying error object + HttpStatus, and a **centralised handler** (`@RestControllerAdvice`).

Creating an error object (RFC 7807)

Do not pass only a status code; pass an object with more information so the client can interpret it. RFC 7807 standardises «problem details»: status codes in the 400/500 range, content type `application/problem+json` (or `+xml`) and the keys **title**, **status**, **detail**, *type* (URL describing the error, optional) and *instance* (URI of the error log, optional).

```
public class ApiError {
    private String title;
    private int status;
    private String description;
    private String type = "about:blank";
    private String instance = "";

    public ApiError(String title, int status, String description) {  // + getters/setters
        ...
    }
}
```

Handling it in the controller with ResponseEntity

Return `HttpStatus.NOT_FOUND` with the `ApiError` as body and content-type `application/problem+json`. In the client, `RestTemplate` throws an **HttpStatusCodeException** for any 4xx/5xx; the body arrives as a **String** that you parse with `ObjectMapper`.

```
@GetMapping("{id}")
public ResponseEntity<? extends Object> returnGuest(@PathVariable int id) {
    try {
```

```
        return new ResponseEntity<Guest>(receptionService.findGuest(id),
                HttpStatus.OK);
    } catch (GuestNotFoundException e) {
        HttpStatus status = HttpStatus.NOT_FOUND;
        ApiError err = new ApiError("guest not found", status.value(),
                e.getMessage());
        HttpHeaders h = new HttpHeaders();
        h.add("content-type", MediaType.APPLICATION_PROBLEM_JSON_VALUE);
        return new ResponseEntity<ApiError>(err, h, status);
    }
}

// client
try {
    re = rt.getForEntity(baseUrl + "/3", Guest.class);
} catch (HttpStatusCodeException e) {
    if (HttpStatus.NOT_FOUND == e.getStatusCode()) {
        ApiError ae = new ObjectMapper().readValue(e.getResponseBodyAsString(),
                ApiError.class);
    } else {
        ...
    }
}
```

Works, but…

every resource method must return a ResponseEntity; every own exception must be translated by hand into error object + status; and the error body must be parsed from a String. Hence: centralise it.

Centralised exception handling

Based on **AOP advice**: a class annotated `@RestControllerAdvice`, extending `ResponseEntityExceptionHandler` (which already handles the common HTTP exceptions such as `HttpRequestMethodNotSupportedException` and `HttpMediaTypeNotSupportedException`), with `@ExceptionHandler` methods containing the code that used to live in the controller. The client stays the same.



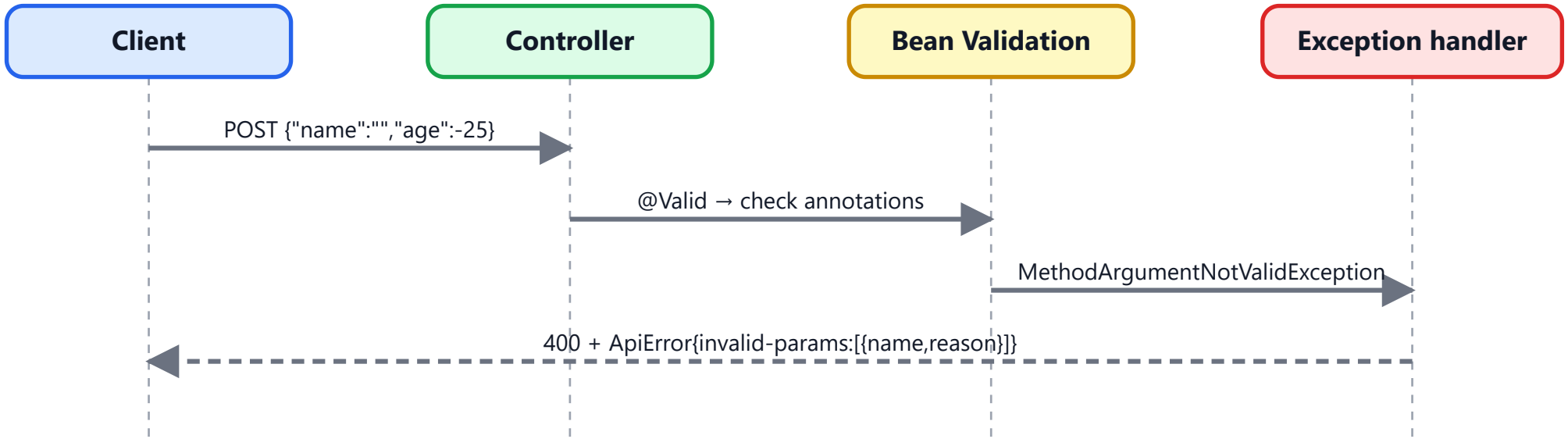

```
@RestControllerAdvice
public class RestResponseEntityExceptionHandler
        extends ResponseEntityExceptionHandler {
    @ExceptionHandler(value = GuestNotFoundException.class)
    protected ResponseEntity<? extends Object> handleGuestNotFound(
            GuestNotFoundException e, WebRequest request) {
        HttpStatus status = HttpStatus.NOT_FOUND;
        ApiError err = new ApiError("guest not found", status.value(),
                e.getMessage());
        HttpHeaders h = new HttpHeaders();
        h.add("content-type", MediaType.APPLICATION_PROBLEM_JSON_VALUE);
        return new ResponseEntity<ApiError>(err, h, status);
```

```
    }
}

// the controller is now clean
@GetMapping("{id}")
public Guest returnGuestById(@PathVariable("id") int id)
        throws GuestNotFoundException {
    return receptionService.findGuest(id);
}
```

9.4 Validation

Syntax validation uses the **JSR-303 Bean Validation** standard: add `spring-boot-starter-validation`. Put annotations such as `@NotNull`, `@NotEmpty`, `@Size`, `@Min`, `@Pattern`, `@Email` on the model class (with messages); mostly used for POST/PUT. `@Valid` on the `@RequestBody` argument triggers it. Failures throw `MethodArgumentNotValidException`; `ResponseEntityExceptionHandler` has `handleMethodArgumentNotValid()` which you override. Following RFC 7807, a `ValidationError` (name + reason) list called `invalid-params` is added to `ApiError`.



```
public class Guest {
    private int id;

    @NotEmpty(message="name must be filled in")
    private String name;

    @Min(value=2, message="age must be bigger than 2")
    private int age;
}

@PostMapping(path="", consumes=JSON)
public ResponseEntity<Void> addGuest(@Valid @RequestBody Guest guest) {
    ...
}
```

```
@Override
protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {
    ApiError err = new ApiError("invalid arguments", status.value(),
            ex.getMessage());
    for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
        err.getInvalidParams().add(new ValidationError(fe.getField(),
                fe.getDefaultMessage()));
    }
    return new ResponseEntity<Object>(err, headers, status);
}
```

Client side: post an invalid guest, catch the `HttpStatusCodeException`, check for **400**, parse the body into `ApiError` and print `getInvalidParams()`.

Exercise ideas

Add exception handling to `findPerson(id)` and `findPerson(email,pwd)`; create `PersonAlreadyExistsException` and `PersonCanNotBeDeletedException` and centralise the handling; validate that a password has more than 5 characters.

CHAPTER 10

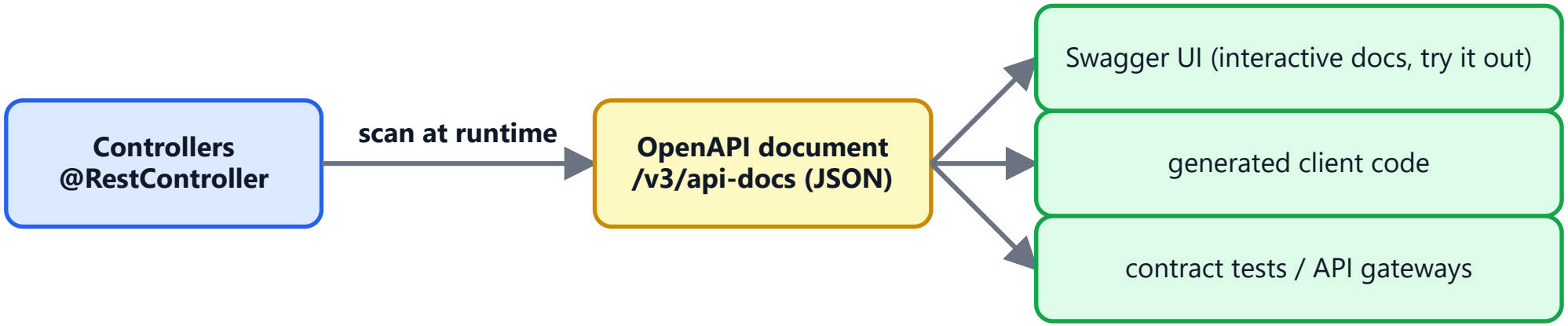
Additional Topics

Swagger for documenting REST APIs · circuit breakers

> **Note**
> In the course text these two topics are only slides with screenshots and demos. This chapter gives the idea behind each of them in a few pictures; check the live demo of your trainer for the exact setup.

10.1 Swagger / OpenAPI: documenting REST APIs

An API is only useful when others can find out how to call it. **OpenAPI** (formerly *Swagger*) is a standard, machine-readable description of a REST API: the URLs, HTTP methods, parameters, request/response bodies and status codes. From that description tools generate **interactive documentation** (Swagger UI), client code and tests.



- In Spring Boot the description is generated from your controllers at runtime (for example with the `springdoc-openapi` starter — add the dependency, start the app, open the Swagger UI page).
- Annotations such as `@Operation`, `@ApiResponse` and `@Schema` add extra explanation to operations and model classes.

- The Swagger UI page lets a developer **try** every call from the browser — an alternative to Postman.

Exercise

Provide Swagger documentation for your API.

10.2 Circuit breakers

In a distributed system a call to another service can fail or hang. If every caller keeps trying, threads pile up and the failure **cascades** through the whole system. A **circuit breaker** works like the fuse in your home: after too many failures it «opens» and fails fast (or returns a fallback) so the failing service can recover.



State	Behaviour
Closed	calls pass through; failures are counted
Open	calls are refused immediately (fail fast) or answered by a **fallback**; after a waiting time the breaker tries again
Half-open	a few trial calls are allowed; success → closed again, failure → open again

In the Spring world this is typically provided by Spring Cloud Circuit Breaker with a library such as Resilience4j.

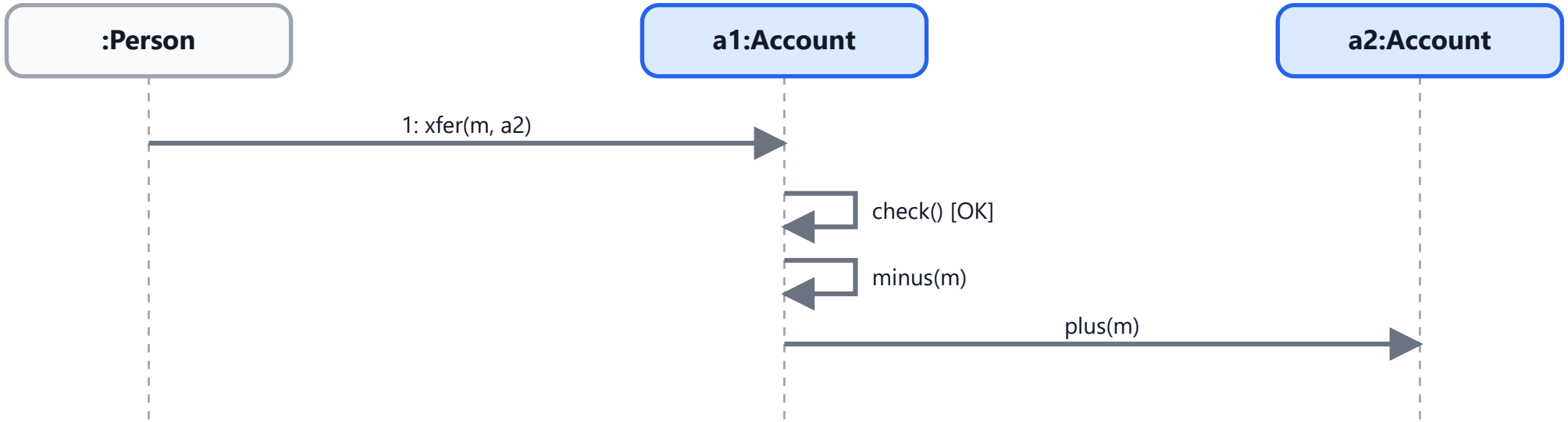
CHAPTER 11

Introduction to ORM and (Spring) JPA

Persisting objects · the object-relational mismatch · features of persistence mechanisms · JPA, Hibernate and Spring Data JPA

11.1 Persisting objects

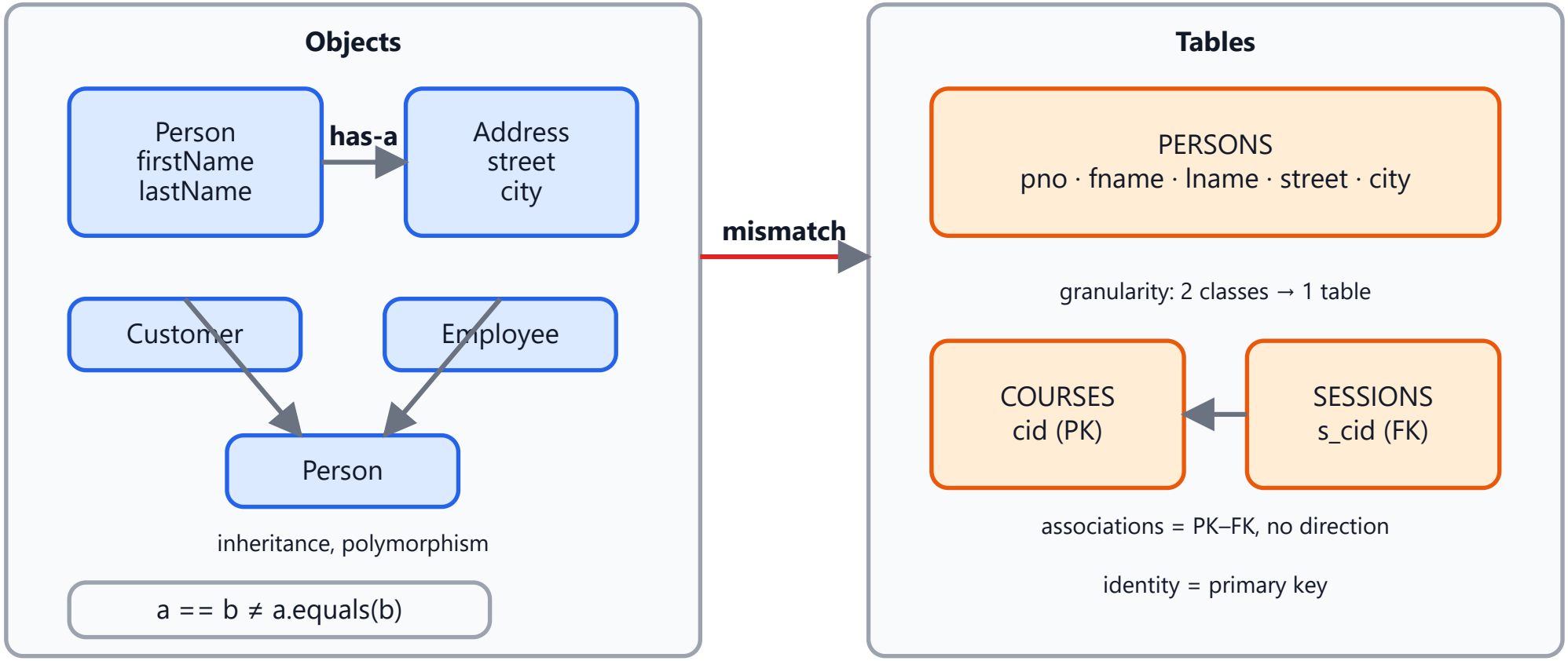
OO applications are composed of objects that consist of **data and behaviour**, are **connected** to each other and **send messages** to each other. Example: transferring money between accounts — a person calls `xfer(m, a2)` on account a1, which checks the balance, withdraws `minus(m)` and calls `plus(m)` on account a2.



Object life cycle: objects are described in classes (the *domain model*), classes are instantiated at runtime, the instances are populated with data — and that data must be preserved, i.e. **persisted**.

11.2 Object-relational mapping (ORM)

Objects and tables are built on different principles. This **O/R mismatch** shows up in several places:

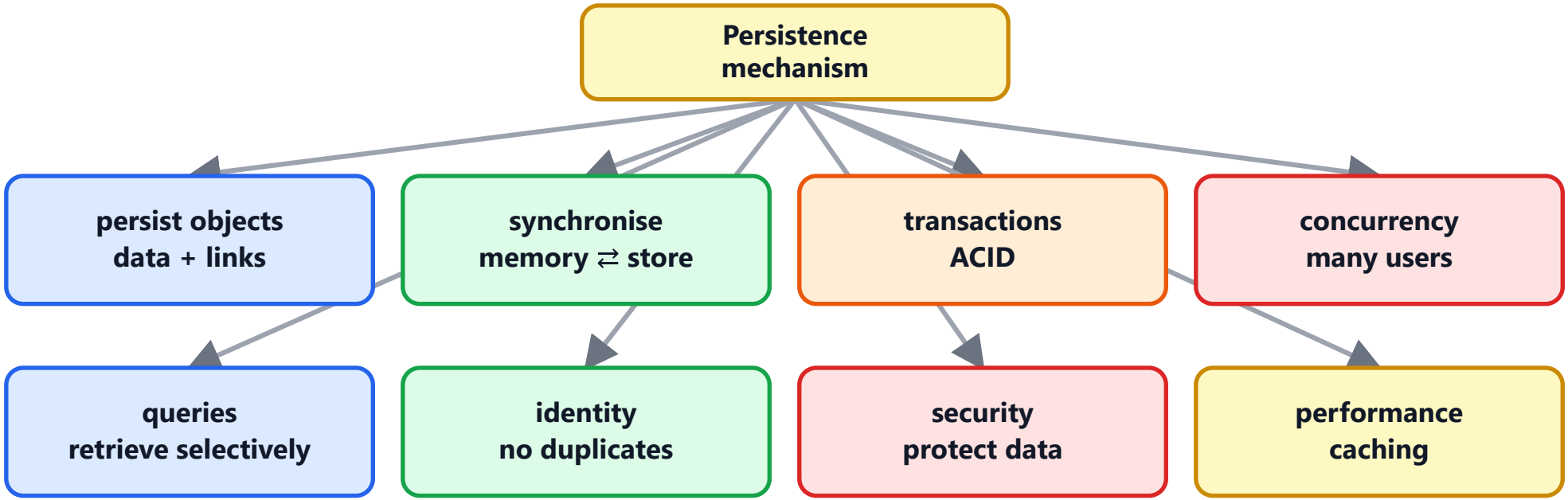


Area	Object world	Relational world
Granularity	fine-grained objects, attributes	tables and columns
Inheritance	class hierarchies, polymorphism	(mostly) not supported → map a tree to table(s); polymorphic queries
Identity	not directly supported (object references)	primary key (PK)
Associations	navigational links between objects (directional)	PK–FK relations; directional problem in 1–many and many–many
Transactions	in the application or by the data store?	concurrency control
Synchronisation	how to keep objects in sync with the database and vice-versa	
Performance	caching, lazy loading	

ORM solutions

How to specify the mapping: hard-coding the DB schema in Java, **XML** metadata, or **annotations** (today's choice). Frameworks: doing it yourself is extremely difficult and expensive, so use an existing solution — **JPA** (Jakarta EE entity beans are based on it), **Spring Data JPA** (built on Hibernate), or alternatives (Hibernate, OJB, EclipseLink, Cayenne, MyBatis ...). They are based on the *Unit of Work* enterprise application architecture pattern.

11.3 Features of persistence mechanisms



- **Persist the information:** data in the objects (attributes) and the links between them (relationships).
- **Synchronisation:** memory ↔ data store, important if several applications use the same data.
- **Transactions:** a set of actions that moves data from one consistent state to another — **ACID**: Atomicity, Consistency, Isolation, Durability.
- **Concurrency control:** different users reach the same data at the same time while it stays consistent.

- **Query mechanism:** retrieve data selectively. **Identity support:** unique identification, avoid several copies of the same data.
- **Security** and **performance optimisation** (caching).

11.4 (Spring) JPA

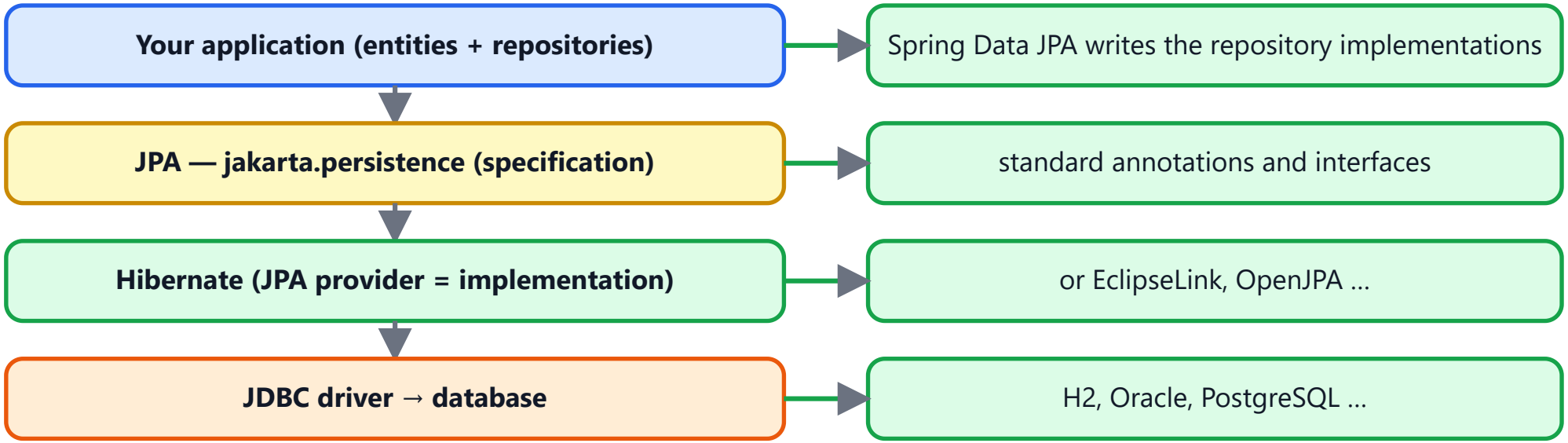
JPA and Hibernate

JPA — Java Persistence API (2006)

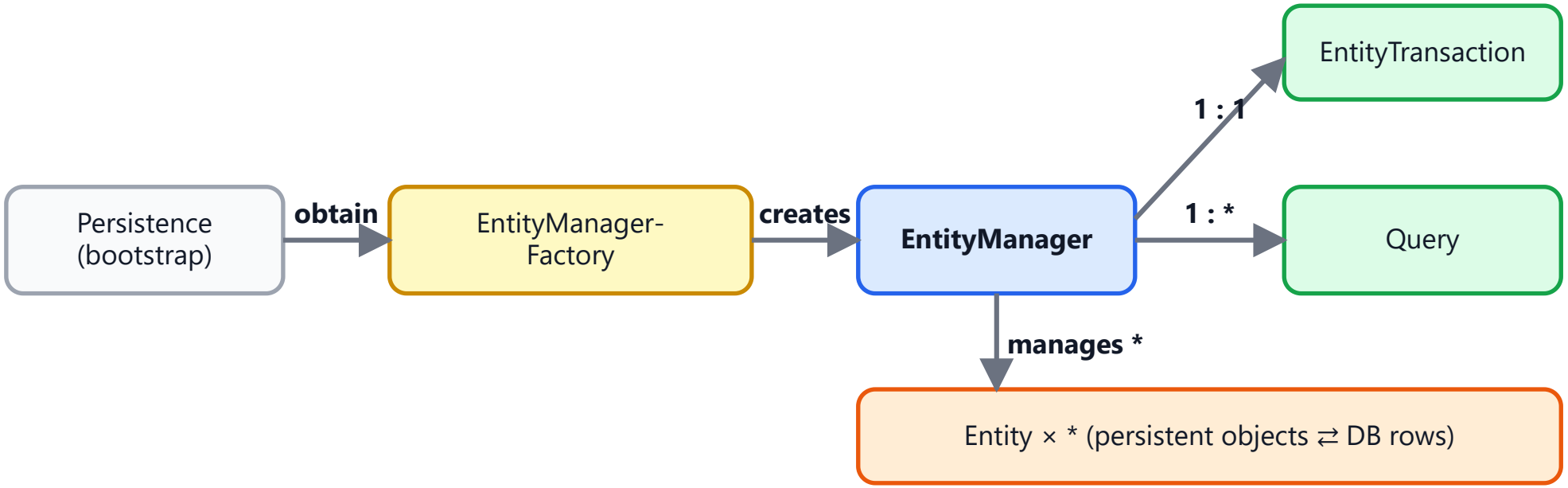
A Java **specification** for accessing, persisting and managing data between Java objects and a relational database. Defined in EJB 3.0 as the replacement for EJB 2 CMP entity beans. Package `jakarta.persistence` (formerly `javax.persistence`). A specification, **not an implementation** — implementations are *JPA providers*: EclipseLink, OpenJPA, **Hibernate**, DataNucleus.

Hibernate

Started in 2001 as an open-source project, later developed under JBoss (Red Hat). Its basic interface used to be the `SessionFactory`; it became a certified JPA implementation (2010) and now mainly uses the `EntityManager`.



JPA architecture (`jakarta.persistence`)

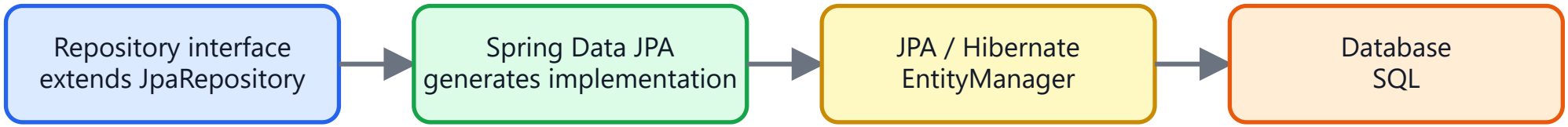


Type	Role
`Persistence`	obtain `EntityManagerFactory` instances (vendor neutral)
`EntityManagerFactory`	factory for `EntityManager`s
`EntityManager`	manages a set of persistent objects; also a factory for `Query`
`Entity`	a persistent object representing datastore records
`EntityTransaction`	(one per EntityManager) groups operations on persistent data into consistent units of work
`Query`	finds persistent objects meeting criteria, using JPQL, the Criteria API and/or native SQL

Spring's JPA integration: Spring Data JPA

- powerful **repository** and custom object-mapping abstractions;
- **query derivation** from repository method names;
- domain base classes with basic properties; transparent **auditing** (created, last changed);

- custom repository code can be integrated; easy configuration in Spring Boot;
- advanced integration with Spring MVC controllers; experimental cross-store persistence.



CHAPTER 12

Configuration

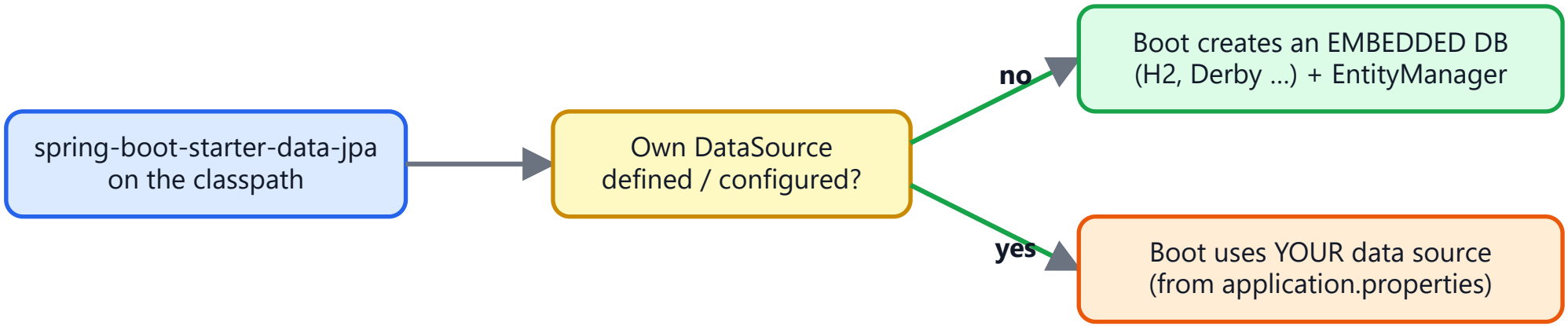
Spring Data JPA configuration · data sources (embedded, basic, pooled, JNDI) · Spring Boot simplifications · extra options · testing the connection

12.1 Spring JPA: basic configuration

Add the starter and Spring Boot auto-configures the rest:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

- Spring Boot tries to use an **embedded database** by default.
- When you define your own `DataSource` this auto-configuration is switched off.
- To disable it manually: `@SpringBootApplication(exclude={DataSourceAutoConfiguration.class})`.

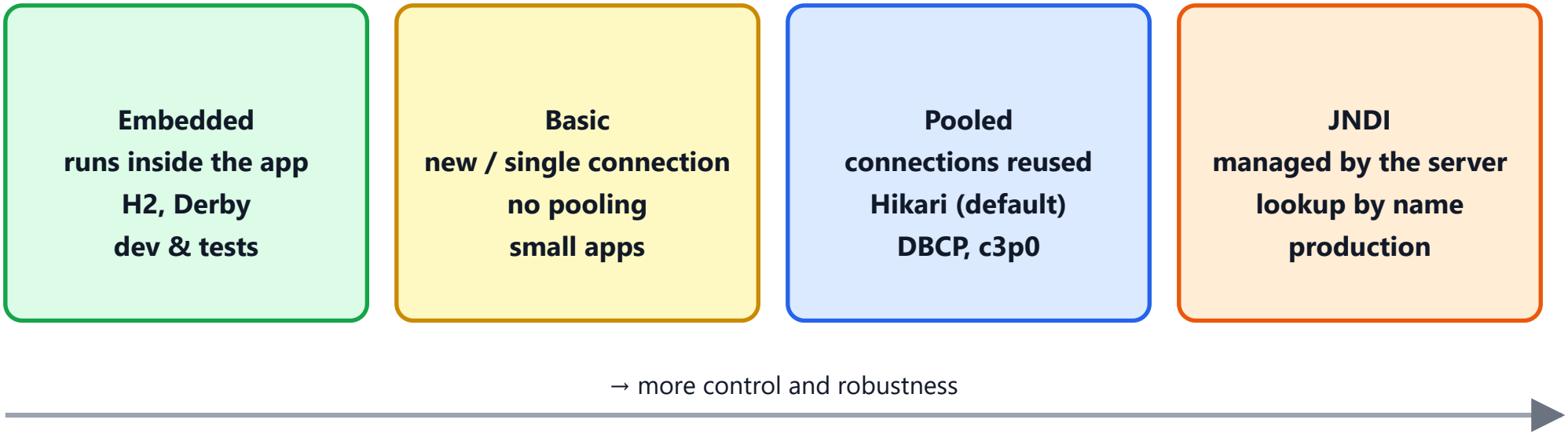


12.2 Configuring a data source

A **data source** defines the driver, the database URL, the user name and the password. A central configuration makes it easy to change later. The driver depends on the database; add the matching Maven dependency:

Database	Dependency (groupId : artifactId)
H2	`com.h2database : h2`
Oracle	`com.oracle.database.jdbc : ojdbc11`
PostgreSQL	`org.postgresql : postgresql`
SQL Server	`com.microsoft.sqlserver : mssql-jdbc`

12.3 Data source types



Embedded data source

The database runs **as part of your application** instead of as a separate database: invisible to the end user, perfect for development and tests; the test data is **reset on every restart**; you pass a script that sets up the DB. Products: Derby, H2.

```
@Bean
@Profile("development")
public DataSource testDataSource() {
    return new EmbeddedDatabaseBuilder().setType(EmbeddedDatabaseType.H2)
            .addScript("classpath:h2.sql").build();
}
```

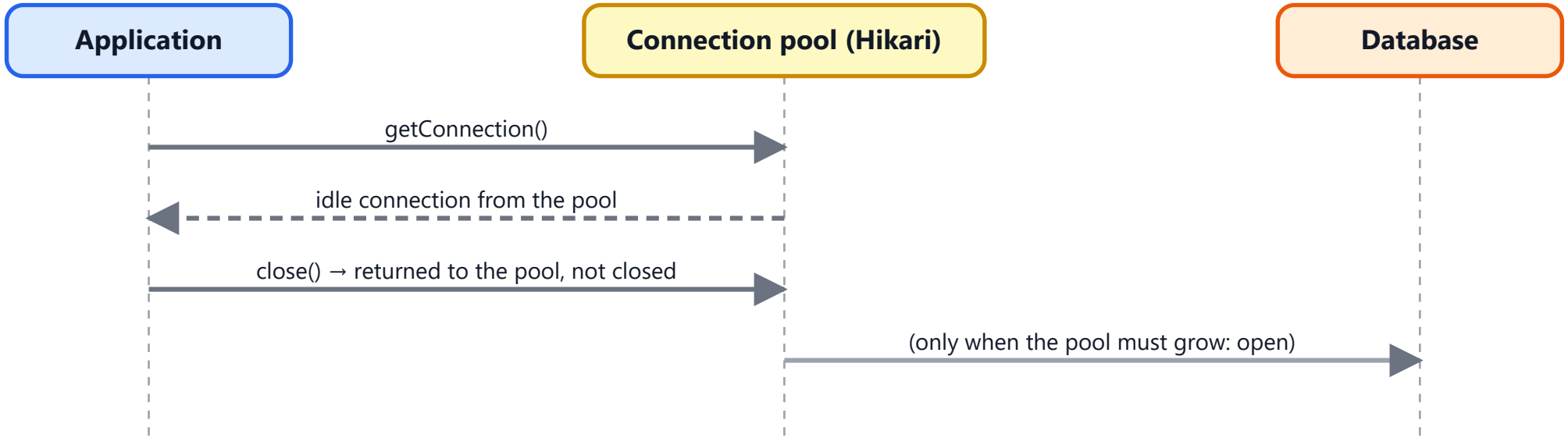
Basic data source

Simplest but limited, provided by Spring (`org.springframework.jdbc.datasource`): `DriverManagerDataSource` (a **new connection** for every request) and `SingleConnectionDataSource` (the **same** connection every time). DBMS vendors have their own (e.g. `OracleDataSource`). **No pooling** — for small applications or the development phase.

```
# application.properties
spring.datasource.url=jdbc:oracle:thin:@//delphi:1521/TSTA
spring.datasource.username=tu00010
spring.datasource.password=tu00010
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
```

Pooled data source

Connections are kept in a **pool** and re-used. Provided by third parties (Apache Commons DBCP — which relies on Commons Pool —, c3p0, **HikariCP**). **Hikari is preferred** (Oracle, PostgreSQL, ...) and already included in `spring-boot-starter-data-jpa`.




```
# basic connection settings as before, plus
spring.datasource.hikari.minimumIdle=5
spring.datasource.hikari.maximumPoolSize=20
spring.datasource.hikari.idleTimeout=30000
spring.datasource.hikari.maxLifetime=2000000
spring.datasource.hikari.connectionTimeout=30000
```

JNDI data source

Best choice in production: the data source is managed by the server (WebSphere, Tomcat, ...) and retrieved via JNDI.

```
spring.datasource.jndi-name=java:comp/env/jdbc/MyJndiName
```

Java config is deprecated
The slides also show the `@Bean DataSource` variants (OracleDataSource, HikariDataSource, JndiTemplate). With Spring Boot you no longer need them: the properties above are enough.

12.4 Simplifications from Spring Boot

- Data sources are auto-configured from **properties files** — no need to configure them or load resources yourself.
- Use `application-<profile>.properties` files when profiles are used.
- Pre-configured Boot properties; the **driver is deduced from the database URL**.
- When nothing is configured, an **embedded database** is used based on the pom.xml.
- The `EntityManager` is used behind the scenes.

12.5 Extra configuration options

Schema initialisation: `spring.jpa.hibernate.ddl-auto`

none	validate	update	create	create-drop
no DDL	check only	apply the diff	drop + create	create, drop at exit

← safer for existing / production schemas destructive, for development →

JpaSolution uses ddl-auto=none: the schema is owned by the DBA script createH2DB.sql, Hibernate only reads and writes rows

Value	What happens
`create`	existing tables are dropped first, then recreated
`create-drop`	tables created, dropped when the application ends
`validate`	checks that the tables and columns exist, otherwise an exception
`update`	the model derived from the mappings is compared with the schema, which is updated with the difference
`none`	no DDL at all

Boot defaults to `create-drop` if no schema manager was detected (embedded DB), otherwise `none`.

Formatting and logging SQL

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
# also shows the bound parameter values
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
```

12.6 Testing the connection

```java
@SpringBootTest
public class DataSourceTest {
    @Autowired DataSource dataSource;

    @Test
    public void testConnectionViaDataSource() {
        try (Connection c = dataSource.getConnection()) {
            System.out.println("connection succeeded via " + c.getMetaData()
                    .getDatabaseProductName());
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In the solution project (`JpaSolution`)

```properties
spring.datasource.url=jdbc:h2:file:C:/temp/javacourses/h2dbBNP/h2d;AUTO_SERVER=TRUE
spring.datasource.username=h2user
spring.datasource.password=h2pass
spring.datasource.driver-class-name=org.h2.Driver
# the tables come from createH2DB.sql, not from Hibernate
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
```

Run `createH2DB.sql` once (IntelliJ database view or the H2 console) to create and fill the five tables.

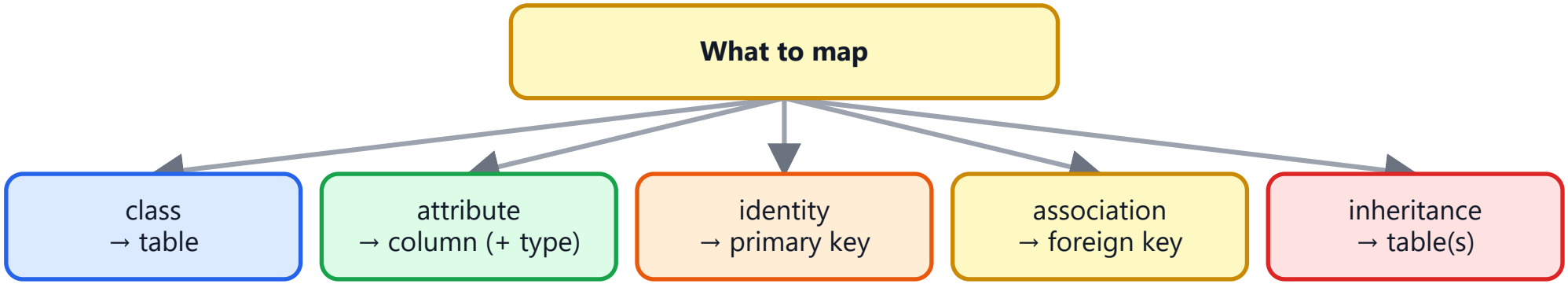
CHAPTER 13

Basic OR Mapping and JPA Repositories

Class-to-table mapping · object identity · composite keys · property mapping · JPA repositories and query methods · Spring Data JUnit testing

13.1 Mapping concepts

What has to be mapped?



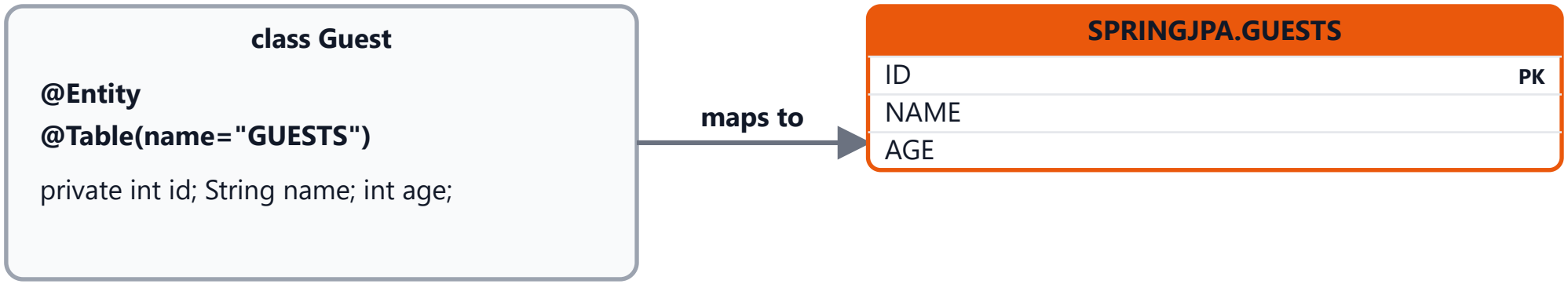
Mapping is done with **(JPA) annotations** (since Hibernate 3). Because JPA works by *configuration by exception*, simple mappings need very few annotations: everything has a sensible default and you only annotate what differs.

13.2 Class to table mapping

Class-level annotations: `@Entity` makes the class persistent; `@Table` associates it with a table name (default = class name). The schema can be added with the `schema` attribute, or set once in `application.properties`.

```
@Entity
@Table(name="GUESTS", schema="SPRINGJPA")
public class Guest {}

# default schema for all entities
spring.jpa.properties.hibernate.default_schema=springjpa
```



13.3 Object identity

Three kinds of «same»

- **identical** objects: `a == b`
- **equal** objects: `a.equals(b)`
- **database identity**: the same primary key

Which key?

- **natural** key (e.g. e-mail)
- **synthetic / surrogate** key (generated)

A good PK is *not null*, *unique* and *never changes* (no setter).

Mapping the key: @Id

`@Id` on a **field** → *field access* for **all** attributes; on a **get method** → *property access* for all. Without a generator the application must supply the PK value (e.g. a constructor with the key argument). A unique value can be generated with `@GeneratedValue`:



JpaSolution uses IDENTITY — the H2 tables have GENERATED BY DEFAULT AS IDENTITY columns

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="gno")
private int id;

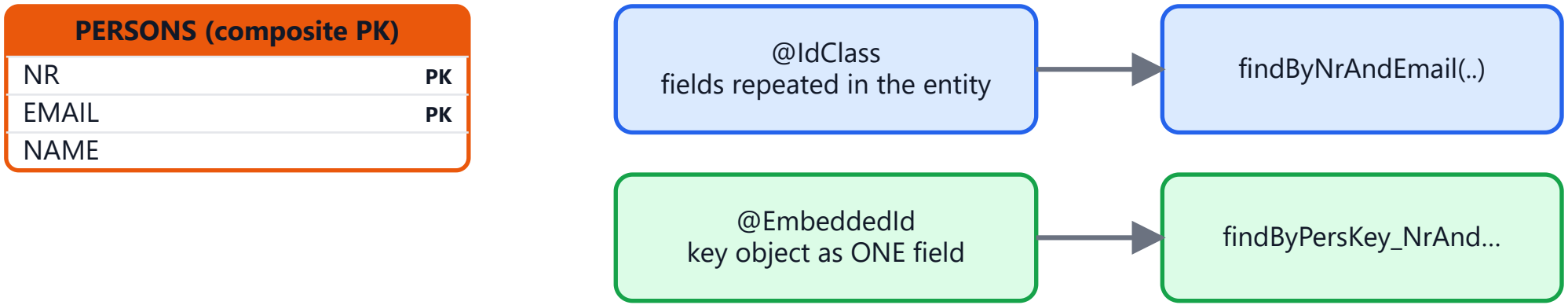
@SequenceGenerator(name="mySeqGen", sequenceName="guestsequence", allocationSize=1)
@Id
@GeneratedValue(strategy=GenerationType.SEQUENCE, generator="mySeqGen")
@Column(name="gno")
private Long id;

@TableGenerator(name="myTabGen", table="keytable", pkColumnName="keygenName",
        pkColumnValue="myGen", valueColumnName="newValue", initialValue=100,
        allocationSize=10)
```

```java
@Id
@GeneratedValue(strategy=GenerationType.TABLE, generator="myTabGen")
@Column(name="gno")
private Long id;
```

Composite primary key

Two techniques: **@IdClass** and **@EmbeddedId**. In both an extra class represents the id; it must implement `Serializable` (session caching) and override `equals` / `hashCode`. Using the key in a `JpaRepository` differs slightly between the two.



both need a key class that is Serializable and overrides equals() / hashCode()

mind the repository type: JpaRepository<Person, PersonKey>

```java
// @IdClass
public class PersonKey implements Serializable {
    private int nr;
    private String email;
    equals() + hashCode()
}
```

```java
@Entity
@IdClass(PersonKey.class)
public class Person {
    @Id private int nr;
    @Id private String email;
}

interface PersonJpaRepository extends JpaRepository<Person, PersonKey> {
    Person findByNrAndEmail(int nr, String email);
}
```

```java
// @EmbeddedId
@Embeddable
public class PersonKey implements Serializable {
    private int nr;
    private String email;
    equals() + hashCode()
}

@Entity
public class Person {
    @EmbeddedId private PersonKey persKey;
}

interface PersonJpaRepository extends JpaRepository<Person, PersonKey> {
    Person findByPersKey_NrAndPersKey_Email(int nr, String email);
}
```

In JpaSolution

`Enrolment` uses `@EmbeddedId EnrolmentId(sessionId, enrolmentInSession)`, so its repository is `JpaRepository<Enrolment, EnrolmentId>` and the derived query is called `existsByIdSessionId(...)` (property `id` → `sessionId`).

13.4 Property mapping

A persistent **field** (direct access) or **property** (via get/set). `@Column(name="...")` gives the column name (default = property name); fields may be private, protected or package-visible. Remember *configuration by exception*: **every non-static, non-transient field is mapped by default**.

```
@Column(name="ctitle") private String title;  // field access

@Column(name="cdur")
public short getDuration() {  // property access
    return duration;
}
```

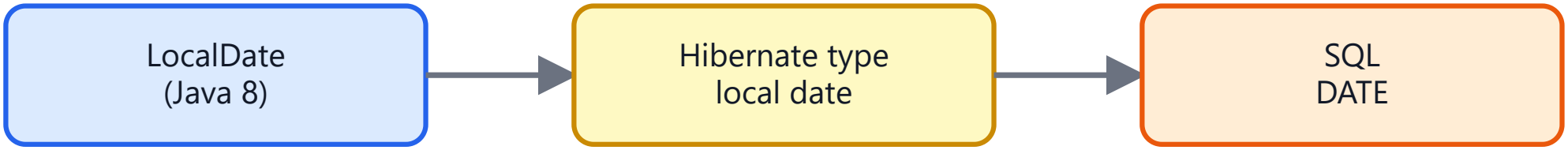
@Column attribute	Meaning (default)
`length`	column length (**255**!)
`updatable` / `insertable`	part of UPDATE / INSERT statements (true)
`nullable`	column may be NULL (true)
`unique`	unique constraint (false)
`precision` / `scale`	decimal precision / scale (0)
`table`	target table (the primary table)
`columnDefinition`	e.g. `CHAR(45)`

`@Transient`: the field is **not** persisted. Bean Validation annotations (`@NotNull` ...) can be used as well.

Type conversion — built-in Hibernate types

Mapping type	Java type	SQL type
integer / long / short / byte	int, long, short, byte (or wrappers)	INTEGER / BIGINT / SMALLINT / TINYINT
double / float / big_decimal / big_integer	double, float, BigDecimal, BigInteger	DOUBLE / FLOAT / NUMERIC
string / character / text	String, Character	VARCHAR / CHAR(1) / CLOB
boolean / yes_no / true_false	boolean, Boolean	BIT / CHAR(1) 'Y','N' / CHAR(1) 'T','F'
date / time / timestamp	java.util.Date, java.sql.*	DATE / TIME / TIMESTAMP
local date / local time	**java.time.LocalDate / LocalTime**	DATE / TIME
calendar / calendar_date	java.util.Calendar	TIMESTAMP / DATE
locale / currency / timezone / class	Locale, Currency, TimeZone, Class	VARCHAR
binary / blob / clob / serializable	byte[], Blob, Clob, Serializable	VARBINARY (or BLOB) / BLOB / CLOB

- The type is derived by **reflection**. Extras: `@Temporal(DATE | TIME | TIMESTAMP)` for date/time, `@Lob` for large objects, `@Enumerated(EnumType.STRING | ORDINAL)` for enums, `@Type` for user-defined types.
- Prefer `EnumType.STRING`: ordinals break when somebody reorders the enum constants.



Derived properties — @Formula

A property that is **not** a column: its value is calculated at runtime by an SQL fragment (columns, SQL functions, subselects). It is read-only (select only).

```
// SELECT ... cprice*1.21 AS finalPrice ...
@Formula("cprice*1.21") private float finalPrice;
```

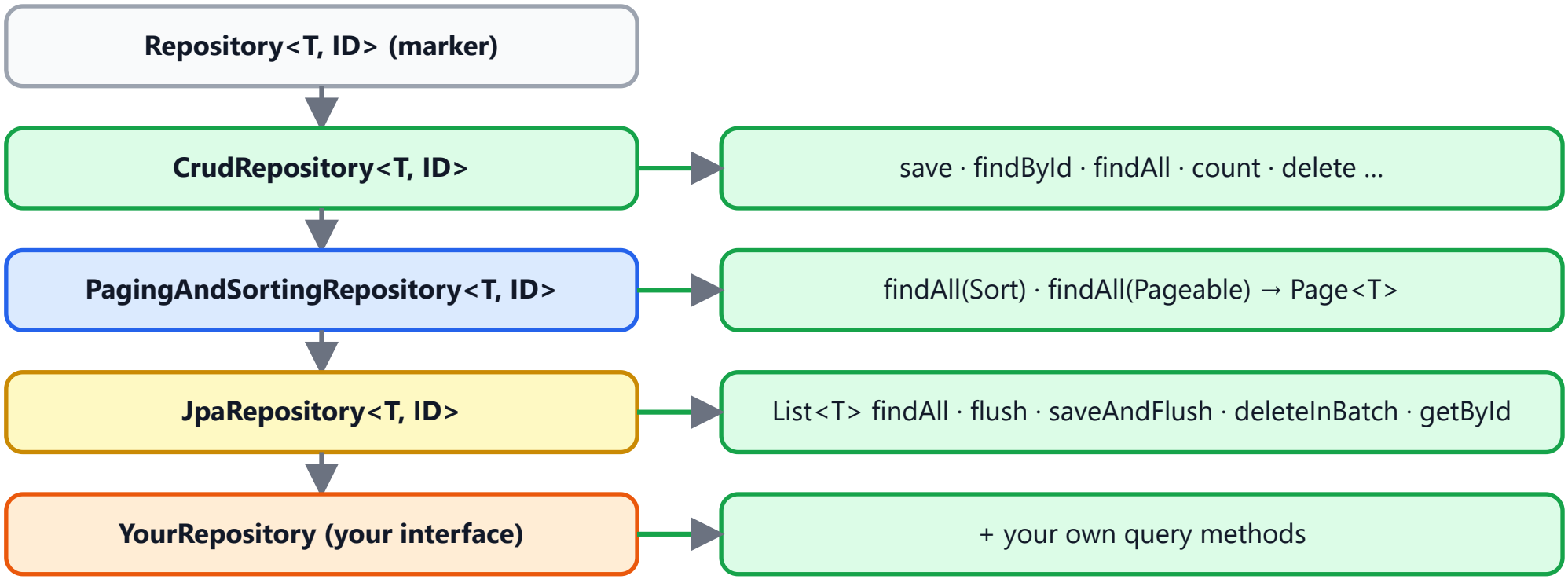
13.5 JPA repositories

In any reasonably-sized application the same methods (add, remove, …) reappear for every class — only the domain type differs. The solution: write only an **interface** that extends `JpaRepository`; Spring generates the implementation at application startup.

```
// persists Guest objects with a Long id
public interface GuestJpaRepository extends JpaRepository<Guest, Long> {}
```

`JpaRepository` contains 18 methods for common operations (`save` = **merge**!, `delete`, find by id, …); extra methods are translated to SQL. Configuration: `@EnableJpaRepositories(basePackages="be.abis.ch8jparepo.dao")`, or nothing at all — it is auto-configured by `@SpringBootApplication`.

The repository interfaces



Interface	Methods
CrudRepository<T,ID>	`save(S)` save/update · `findById(ID)` → Optional · `existsById` · `findAll()` · `count()` · `deleteById` · `delete(T)` · `deleteAll()`
PagingAndSortingRepository	`findAll(Sort)` · `Page<T> findAll(Pageable)`
JpaRepository	`List<T> findAll()` · `findAll(Sort)` · `flush()` · `saveAndFlush(S)` · `deleteInBatch(..)` / `deleteAllInBatch()` (one query) · `getById(ID)` (a reference)

Query methods

Add a method to the interface and Spring derives the implementation from its **name**: `List<Company> getCompanyByCountryOrderByName(String country);`

findByNameStartsWithAndAgeIsLessThanOrderByNameDesc



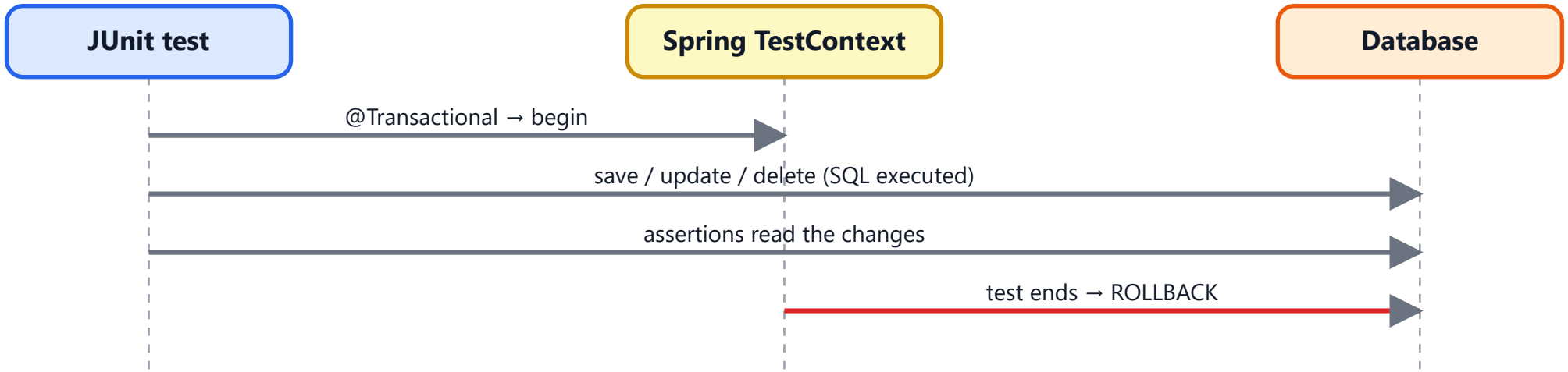
→ select g from Guest g where g.name like 'x%' and g.age < ? order by g.name desc

- Four verbs: **get, read, find, count**. The subject is mostly optional.
- Syntax: *query verb + subject (optional) + predicate + order by*.
- The predicate references one or more properties (combined with `And` / `Or`); with no operator **equals** is used; others: `IsAfter, IsGreaterThan, IsLessThan, IsBetween, IsNull, IsLike, Containing, IsNotIn, IsStartingWith, IsTrue …`; `IgnoreCase` can be added.

```java
public interface GuestJpaRepository extends JpaRepository<Guest, Long> {
    Guest findById(long id);
    List<Guest> findByName(String name);
    List<Guest> findByNameStartsWith(String name);
    List<Guest> findByAge(int age);
    List<Guest> findByAgeIsLessThan(int age);
    long countByName(String name);
    List<Guest> findByNameStartsWithAndAgeIsLessThan(String name, int age);
}
```

13.6 Spring Data JUnit testing

- Put `@Transactional` (`org.springframework.transaction.annotation`) on the test class or method: a transaction managed by the `TransactionalTestExecutionListener` (configured automatically) is started and **rolled back** after the test.
- Change it with `@Commit` or `@Rollback` on class or method; `TestTransaction` offers static methods to start/end transactions, flag commit/rollback and check the status.
- **Watch out:** tests through `RestTemplate` or Postman run in the *server's* transaction and are **not** rolled back automatically.



CHAPTER 14

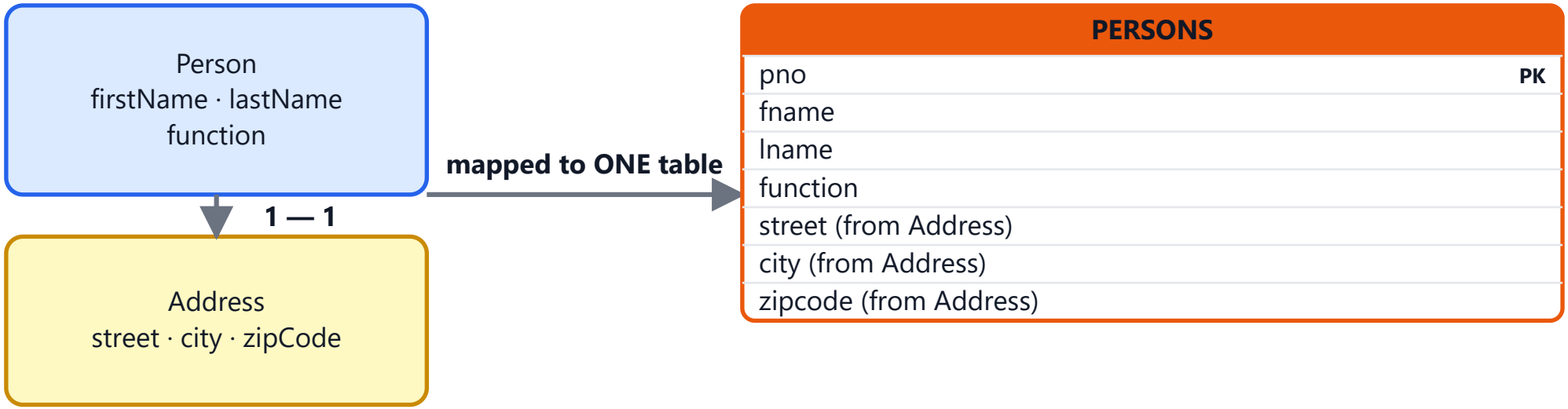
Mapping Associations

Value types · one class for two tables · one-to-one, many-to-one, one-to-many, many-to-many · cascading and fetching strategies

14.1 Value types

Difference between:

- **entity type**: has its **own database identity**;
- **value type**: depends on the database identity of the **owning** entity type — 2 (or more) classes map to **1** table.



Embedded objects

`@Embedded` on the contained property of the «parent» class; `@Embeddable` on the contained («child») class. Notes: **shared references** to an Address object are not possible (it is *composition*); `@Embedded` is optional when `@Embeddable` is defined.

```
@Entity
@Table(name="Persons")
public class Person {
    @Embedded private Address address;
    ...
}

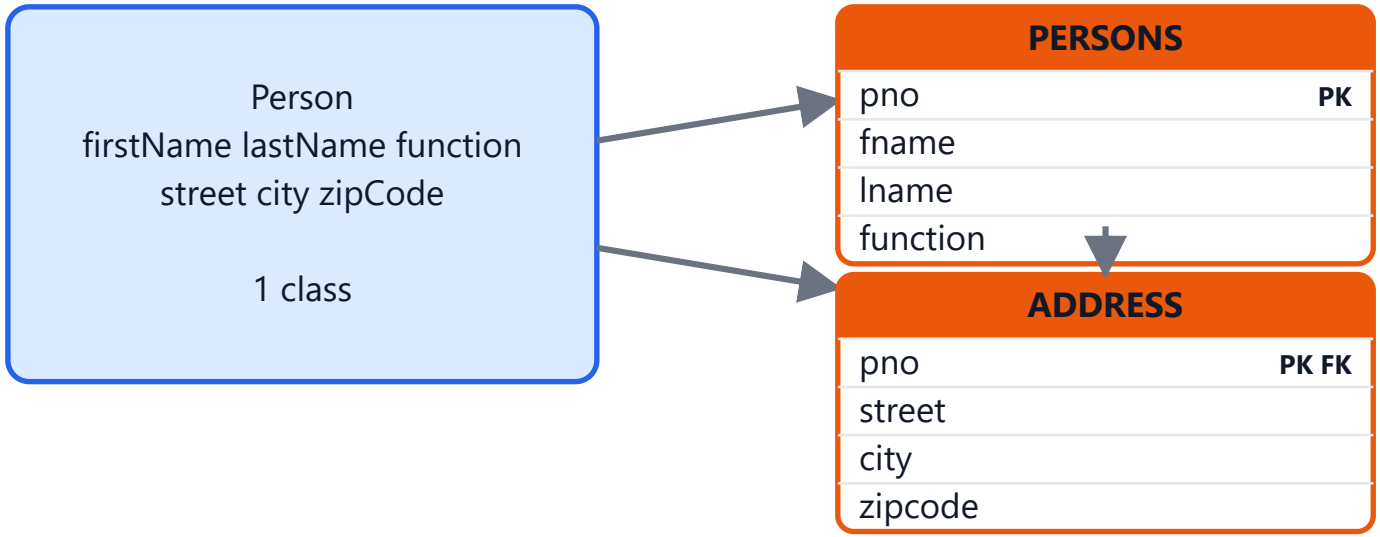
@Embeddable
public class Address {
    private String street;
    private String city;
}
```

In JpaSolution

`Company` embeds `Address`; the five address fields are the columns `COSTREET, COSTRNO, COTOWNNO, COTOWN, COCOUNTR` of `COMPANIES` (see the Appendix for the picture).

14.2 One class for two tables

The opposite problem: **one** class whose data lives in **two** tables. Use `@SecondaryTable`; fields from the second table say `@Column(table="Address", ...)`.



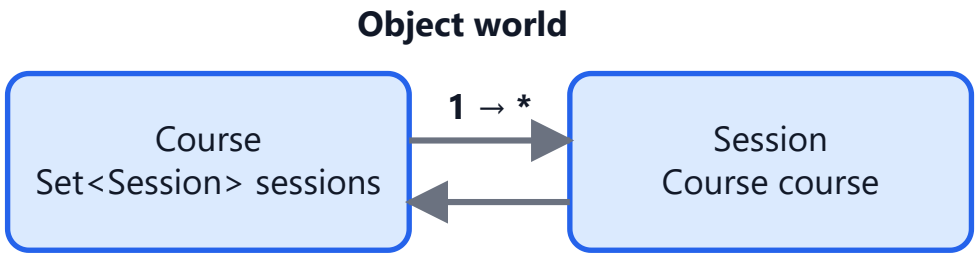
```java
@Entity
@Table(name="Persons")
@SecondaryTable(name="Address", pkJoinColumns = @PrimaryKeyJoinColumn(name = "pno"))
public class Person {
    ...
    @Column(name="lname") private String lastName;

    @Column(table="Address", name="street")
    private String street;

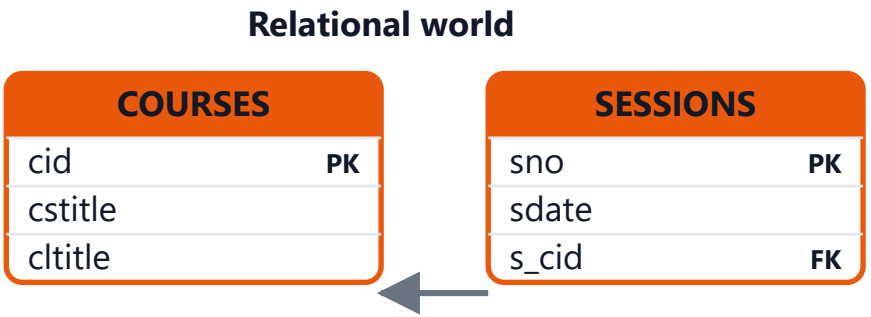
    @Column(table="Address", name="city")
    private String city;
}
```

14.3 Association mapping

Object world vs. relational world (example: a `Course` has many `Session`s):



navigation in both directions (if you code both)

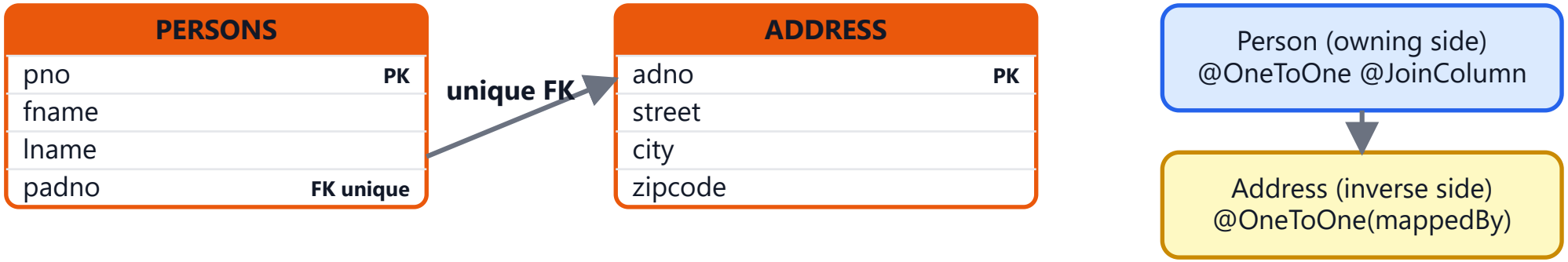


one FK, no direction

- **Multiplicity**: one-to-one, one-to-many, many-to-one, many-to-many.
- **Directionality** (navigational access): uni- or bidirectional; must be implemented in Java code. There are **no managed associations** in JPA → the **owner** class is responsible for the updates.
- In the RDBMS it is just a PK–FK relationship (no direction).

One-to-one

Use it when value-type mapping (embedding) is not possible and each class must be an entity with its own table. Possibilities: **foreign key association**, primary key association, association table (rarely used).



the side WITH the foreign key owns the association

- The FK in `Persons` (`padno`) refers to the PK of `Address`; it must be **unique** — add a unique constraint in the DB.
- Annotations: `@OneToOne` on the association property, `@JoinColumn(name=..)` on the FK property, `@Column(name=..)` on the PK id.
- **Bidirectional**: the class that contains the FK (Person) is the **owning side**; on the **inverse side** add a property annotated `@OneToOne(mappedBy="homeAddress")` (refers to the property name in the owner).

```java
@Entity
@Table(name="Persons")
public class Person {
    @OneToOne
    @JoinColumn(name="padno")
    private Address homeAddress;
}
```

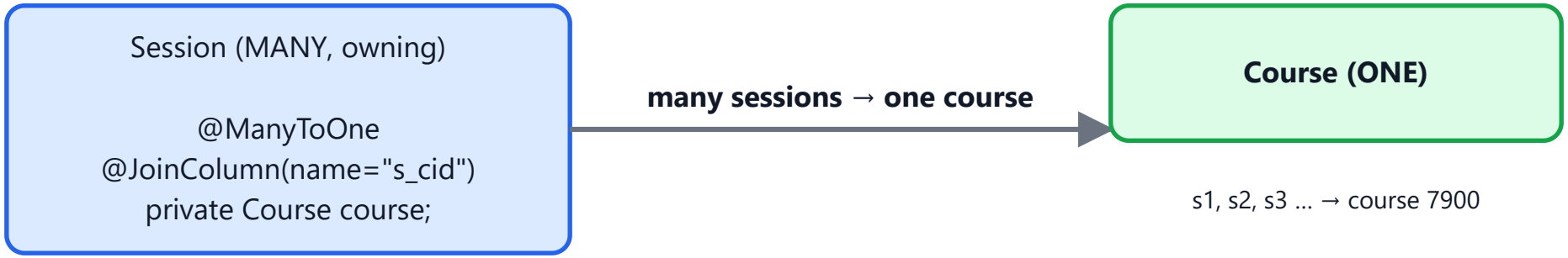
```java
@Entity
@Table(name="Address")
public class Address {
    @Id
    @GeneratedValue
    @Column(name="adno")
    private int addressNr;
    @OneToOne(mappedBy="homeAddress") private Person pers;  // inverse side
}
```

Many-to-one

Put `@ManyToOne` on the Course property in the Session class; the FK column is named with `@JoinColumn`. This is the **owning** side.

```
@Entity
public class Session {
    ...

    @ManyToOne
    @JoinColumn(name="s_cid")
    private Course course;
}
```



One-to-many

`@OneToMany` on a property of type `Set` or `List`. The FK column is defined with **`mappedBy`** (the property in the many side); `targetEntity` is optional when the generic type is given. The reverse direction uses `@ManyToOne` + `@JoinColumn` — the **many side is the owning side**.

```
@Entity
public class Course {
    ...
```

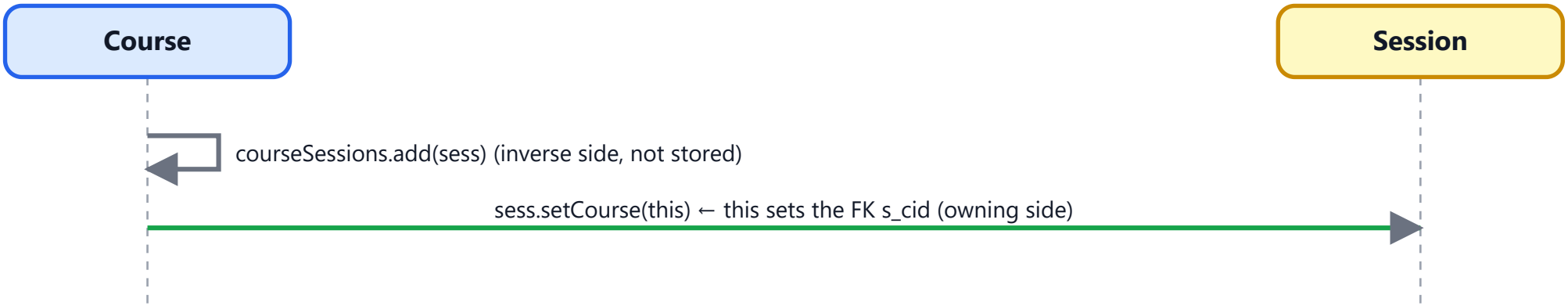
```
    @OneToMany(targetEntity=Session.class, mappedBy="course")
    // initialise the collection!
    private Set<Session> courseSessions = new HashSet<Session>();

    public void addSession(Session sess) {
        getCourseSessions().add(sess);
        sess.setCourse(this);
    }

    public void removeSession(Session sess) {
        getCourseSessions().remove(sess);
        sess.setCourse(null);
    }
}
```

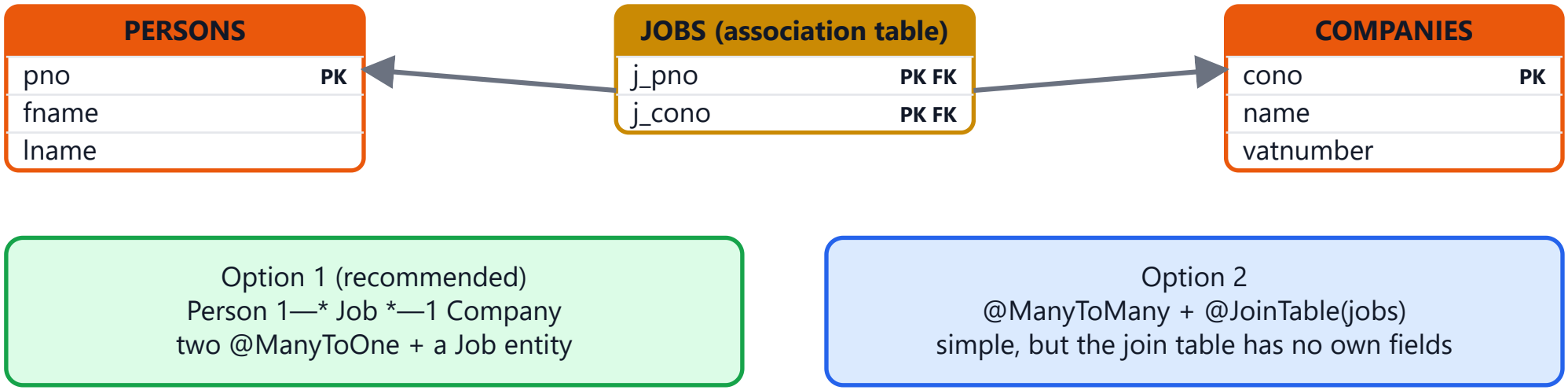
Remarks

- Do not forget to **initialise** the collection.
- Override `equals()` and `hashCode()` for entities used in a Set (business-key equality).
- Keep both sides in sync yourself, e.g. with the convenience methods `addSession` / `removeSession` above.
- If the collection is a `java.util.Map`, use `@MapKey(name="number")` for the key (e.g. a person with several e-mail addresses).



Many-to-many

Example: a person can work for several companies and a company has several people. In the RDBMS an additional table (the **intermediate association class**) is needed: `Jobs(j_pno FK, j_cono FK)`. It can **always** be mapped with two many-to-one associations (**recommended**), or with `@ManyToMany` (uni- or bidirectional).



- By default a mapping table is generated **per direction**; use `mappedBy` to point to the same mapping table. The PK is the combination of both foreign keys.

- Define the join table with `@JoinTable(name="...", joinColumns=@JoinColumn(name="..."), inverseJoinColumns=@JoinColumn(name="..."))`; for a bidirectional mapping repeat `@JoinTable` and swap the join column names (or use `mappedBy` on the other side).

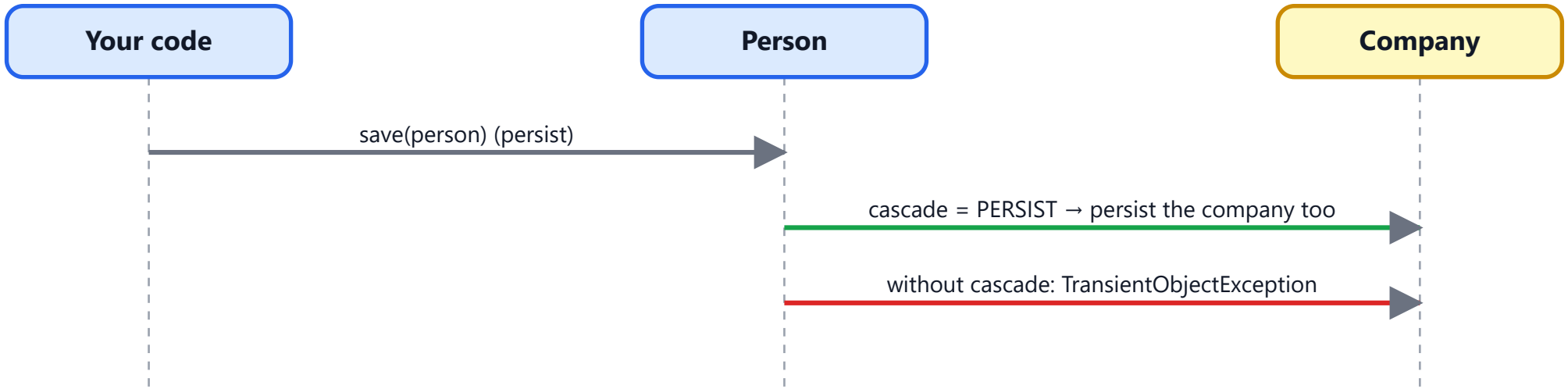
```
@ManyToMany
@JoinTable(name = "jobs", joinColumns = @JoinColumn(name = "j_pno"),
        inverseJoinColumns = @JoinColumn(name = "j_cono"))
private List<Company> listOfCompanies = new ArrayList<Company>();  // in Person

@ManyToMany(targetEntity=Person.class, mappedBy="listOfCompanies")
private List<Person> listOfEmployees = new ArrayList<Person>();  // in Company
```

14.4 Additional options

Cascade strategy

How does an operation travel along the association path? The default is **no cascading**. Define `cascade = CascadeType.…` (`jakarta.persistence`). If the cascade type does not correspond with the database action you get a `TransientObjectException`.



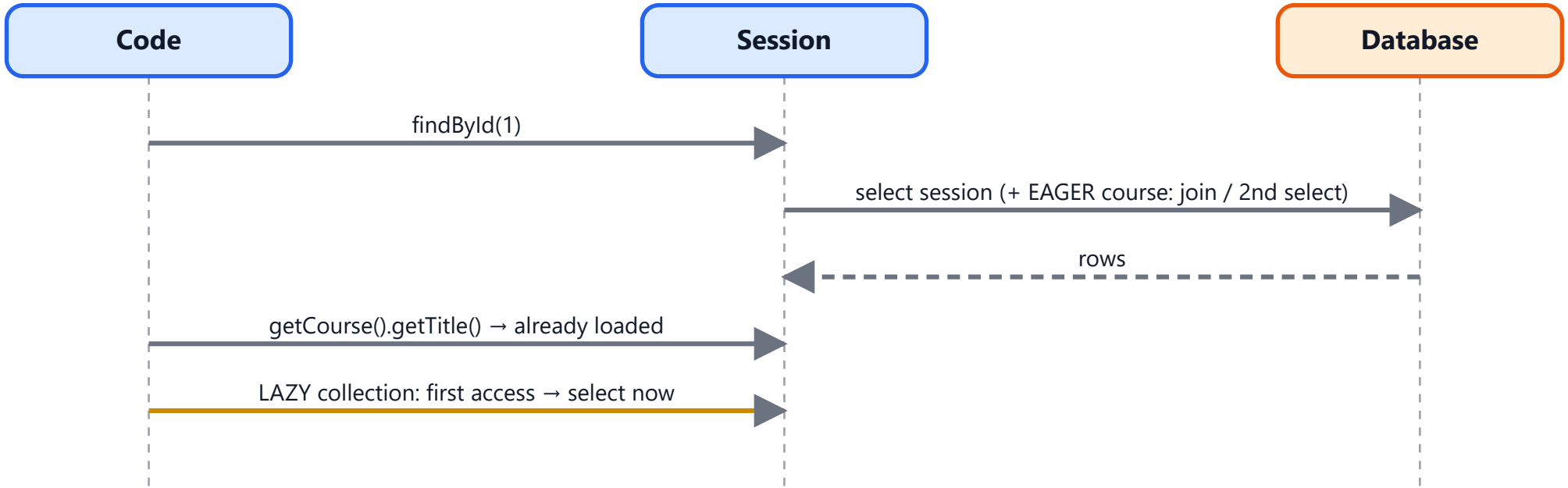
CascadeType	Effect on the associated record
PERSIST	save / persist the corresponding record in the other table
REMOVE	delete the corresponding record
MERGE	merge (after detach) the corresponding record
REFRESH	refresh the corresponding record from the other table
DETACH	detach the corresponding record
ALL	all of the above

In JpaSolution

`Person.company` is `@ManyToOne(cascade = PERSIST)`: a person working for a *new* company inserts the company too. For an *existing* company the service first looks it up (by name and town) and links the managed entity.

Fetching strategy

How are the associated records loaded? `fetch = FetchType.…`



Type	Behaviour	Default for
EAGER	load the corresponding record immediately	one-to-one, many-to-one
LAZY (lazy initialisation)	do **not** load it until it is accessed	one-to-many, many-to-many

Lazy + closed session
Accessing a lazy collection after the transaction/session ended throws a `LazyInitializationException`. Fix it by loading inside the transaction, with a `join fetch` query (chapter 15), or by returning a DTO.

CHAPTER 15

Querying in Spring JPA

@Query · JPQL · named queries · native queries · changing the «select» object · DTOs and mappers

15.1 @Query

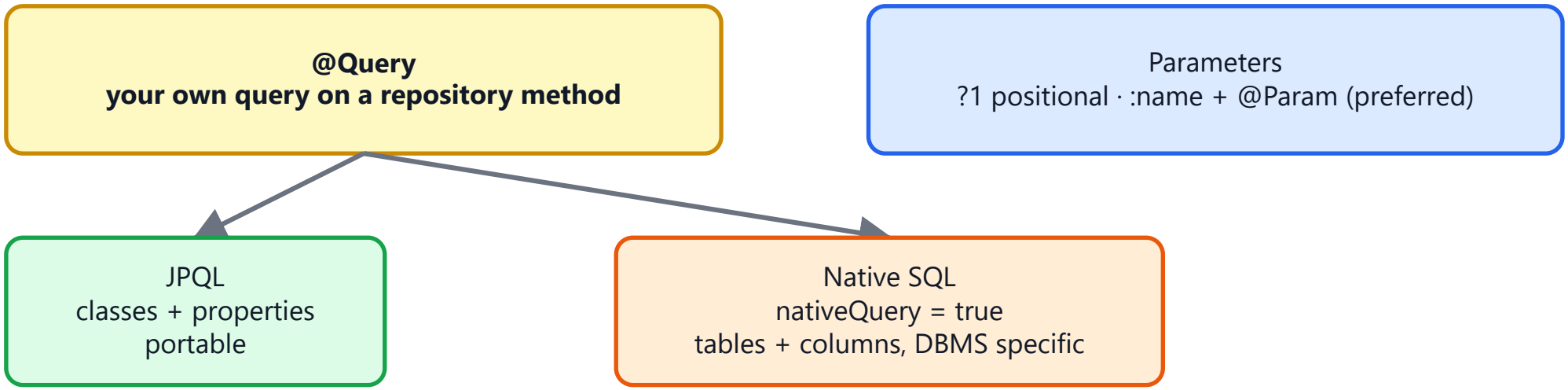
`@Query` on a repository method specifies your **own query** instead of a generated one. It can be written in **JPQL** or in **SQL** (`nativeQuery=true`), with positional (`?1`) or named (`:title` + `@Param`) parameters.

```
@Query("select c from Course c where c.title = 'Java'")
List<Course> findJavaCourses();

@Query(value="select * from courses where title = 'Java'", nativeQuery=true)
List<Course> findJavaCoursesNative();

@Query("select c from Course c where c.title = ?1")
List<Course> findCoursesWithTitle(String title);

@Query("select c from Course c where c.title = :title")
List<Course> findCoursesWithTitle(@Param("title") String title);
```



15.2 JPQL

The **JPA Query Language** works with **classes and properties** instead of tables and columns — querying in an object-oriented way. Characteristics: polymorphic queries, automatic joins for associations, full support for relational operations, paging.

JPQL (objects)

```
select p
from Person p
where p.company.address.town = :t
```

SQL (tables)

```
select p.* from persons p
join companies c on p.pa_cono = c.cono
where c.cotown = ?
```

Hibernate translates the join over the association automatically

case sensitive: Person, p.company, address (class and property names, not column names)

Basic JPQL

Clause	Name	Purpose
`SELECT`	projection	specify the selected data
`FROM`	selection	the data source (class + alias)
`WHERE`	restriction	match records against criteria

Selection: FROM

- Class name + alias: `select cc from ConcreteCourse [as] cc`.
- JPQL is **case sensitive** for class names and properties; if `@Entity(name="abc")` is used, write `abc` in the from clause. The alias is used in the where clause.
- **Polymorphic**: `select c from Course c` returns all ConcreteCourse and CourseGroup objects; `select o from java.lang.Object o` all persistent objects; `select s from java.io.Serializable s` all serializable ones.

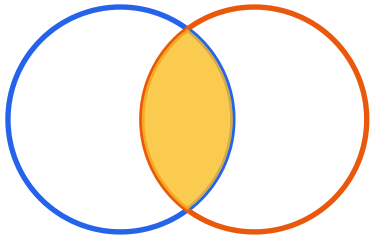
Restriction: WHERE

Operator	Examples
comparison `= <> < > <= >=`, `NOT`	`where c.title = 'Java'` · `c.title <> 'Java'` · `c.duration > 3` · `not(c.title = 'Java')`
(NOT) IN	`where c.title in ('JAVAPROG','SPRINGJPA')`
(NOT) BETWEEN (inclusive)	`where c.price between 200 and 300`
(NOT) LIKE	`like 'Java%'` · `like '%Java%'` · `not like '%Java%'` · `lower(c.title) like '%java%'`
IS (NOT) NULL	`where c.title is null` / `is not null`
AND, OR, NOT	`where c.title not like '%java%' and (c.duration < 2 or c.duration > 4)`
collections	`c.employees is not empty` · `p member of c.employees` · `size(c.employees) > 100`

More functions: `en.wikibooks.org/wiki/Java_Persistence/JPQL`.

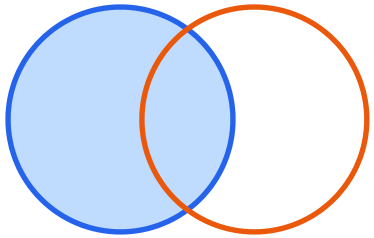
Joins in JPQL

An **inner join is generated automatically** when you traverse entities: `select p from Person p where p.address.country = 'B'`. You can also write joins explicitly:



inner join

only matching rows



left join

all left + matching right

join fetch
loads the association in the
SAME select (overrides LAZY)

implicit join: p.address.country = 'B' → an inner join is generated for you

Style	Example
comma notation	`select d from Employee e, Department d where e.department = d`
(inner) join	`select distinct p from Person p join p.enrolments e where e.pricePerDayPayed > 400`
left join	`select p from Person p left join Company c where p.age>40`
join fetch	`select p from Person p join fetch p.companies c where p.age>40` — overrides `FetchType.LAZY`; mainly for @OneToMany / @ManyToMany

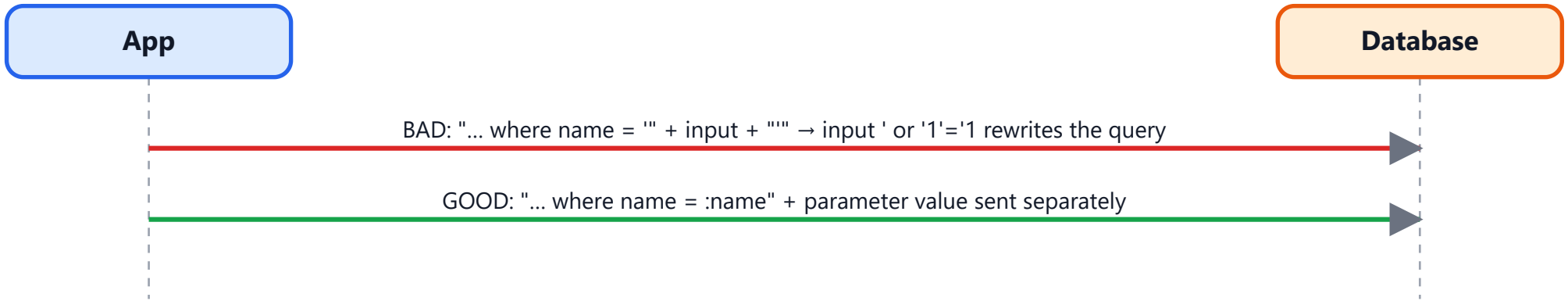
Cartesian product!
Without a where condition or a traversed association, a join produces every combination: `select p from Person p join Address a`.

Parameter binding

```java
// named parameters (preferred): ":" + @Param
@Query("select c from Course c where c.duration = :dur and c.title like %:title%")
List<Course> getCourseByDurationAndTitle(@Param("dur") int duration,
        @Param("title") String title);

// positional parameters (deprecated)
@Query("select c from Course c where c.duration = ?1 and c.title like %?2%")
List<Course> getCourseByDurationAndTitle(int duration, String title);
```

SQL injection
Never build a query by concatenating user input into the string. Bound parameters are sent separately and cannot change the meaning of the query.

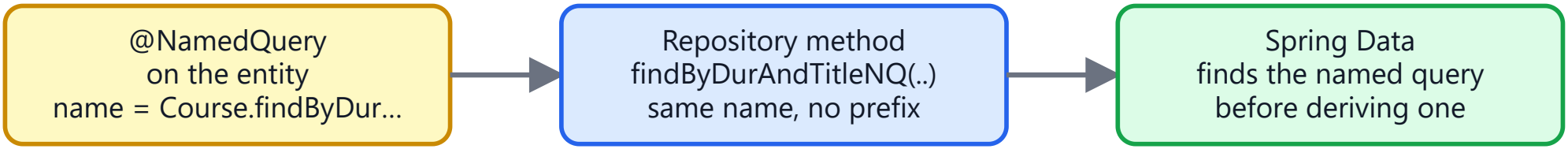


15.3 Named queries

Predefine commonly used queries on the persistent class with `@NamedQuery` (JPQL by default; `@NamedNativeQuery` for SQL). The name is obligatorily **prefixed with the class name**; add a method in the repository interface with **exactly the same name** as the named query (without prefix); parameters are possible.

```
@NamedQuery(name="Course.findByDurAndTitleNQ",
        query="select c from Course c where c.duration = :dur and c.title like %:title%")
public class Course {
    ...
}

public interface CourseJpaRepository extends JpaRepository<Course,Integer> {
    List<Course> findByDurAndTitleNQ(@Param("dur") int duration,
            @Param("title") String title);
}
```



In JpaSolution

`Enrolment` carries `@NamedQuery(name="Enrolment.findEnrolmentsForPerson", ...)` and `EnrolmentJpaRepository` has `findEnrolmentsForPerson(@Param("personId") int)`.

15.4 Native queries

`@Query(value="...", nativeQuery=true)` or `@NamedNativeQuery`; specify the result class in the repository method. Plain SQL with **table and column names**; JPA does **not** generate anything. Mostly used to limit results when using joins; gives you more control but is **DBMS specific**.

```
@Query(value="select * from courses where cname like %:name% order by cname",
        nativeQuery=true)
List<Course> findCoursesOrderedByName(@Param("name") String name);

-- with other functions you may need concat() for LIKE with a parameter:
select * from courses where cname like concat('%', :name, '%') order by cname
```

	JPQL	Native SQL
works on	classes, properties	tables, columns
portable	yes (any DB)	no (DBMS specific)
generated by JPA	translated by Hibernate	sent as-is
typical use	everyday queries, joins over associations	special SQL, limiting columns, DB functions

15.5 Changing the «select» object

The returned type changes when the select does not fetch the full entity: an **aggregate function** returns a number; selecting **specific columns** or **group by** returns `List<Object[]>`. Use **DTOs and mappers** to convert to/from the original entity — mainly for native queries with joins and to limit what a REST API shows. Better for performance, at the «cost» of mapping.

Aggregate functions

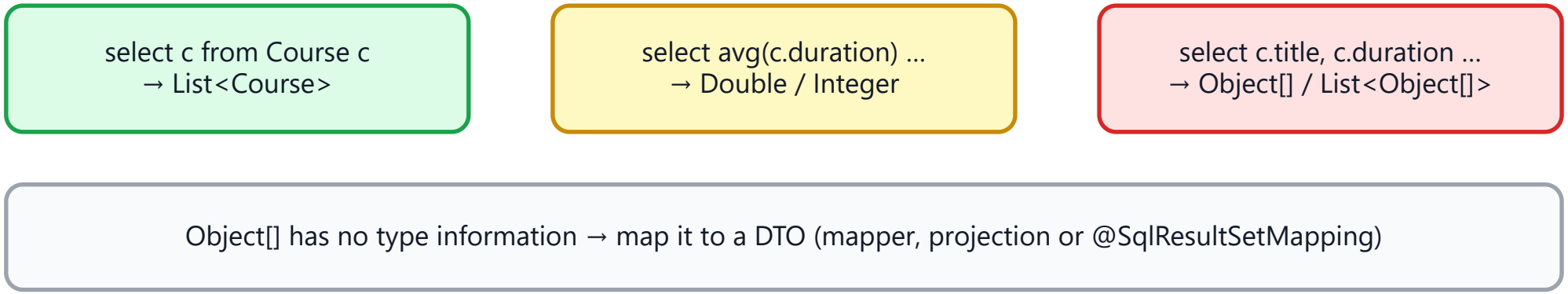
Function	Example
avg	`@Query("select avg(c.duration) from Course c") Double getAverageDurationOfCourses();`
min / max	`@Query("select min(c.duration) from Course c") Integer getMinimumDurationOfCourses();`
sum	`@Query("select sum(c.duration) from Course c") Integer getTotalDurationOfCourses();`
count	`Integer countByDuration(int duration);` (derived)
count distinct	`@Query("select count(distinct c.duration) from Course c") Integer getNumberOfDifferentDurationOfCourses();`

Limiting / changing the selected columns

```
@Query("select c.title, c.duration from Course c where c.id=:id")
Object[] getCourseTitleAndDurationById(@Param("id") int id);

@Query("select c.title, count(c) from Course c group by c.domain having count(c) > 2")
List<Object[]> getNumberOfCoursesPerDomain();

@Query(value="select pfname, plname, coname from persons join companies on pa_cono=cono",
        nativeQuery=true)
List<Object[]> getPersonsWithCompanyName();  // must be mapped to a DTO
```



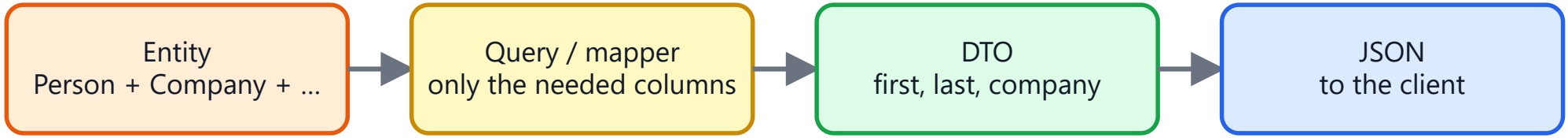
15.6 Using DTOs and mappers

Data Transfer Objects

A DTO is a class that maps the **result of a query**. Provide a constructor with all arguments. It limits the size of the objects passed to the client (web or JSON) and so **increases performance**; it is also better design: it separates the **entity layer** from the **view layer**. It needs a transformation (mapping).

```
public class PersonDTO {
    private String firstName;
    private String lastName;
    private String companyName;

    public PersonDTO(String firstName, String lastName, String companyName) {
        ...
    }
}
```



Four techniques

Technique	How it works	Native queries?
Custom mapper class	a static method `PersonDTO toDTO(Object[])`; the service maps every `Object[]`; every element must be converted to its correct type	yes
@NamedNativeQuery + @SqlResultSetMapping	the named query on the class + a `@ConstructorResult` mapping the columns; the repository returns DTOs directly	yes
Class-based projection	constructor call in the JPQL: `select new be.abis...PersonDTO(p.firstName, p.lastName, p.company.name) from Person p` (fully qualified name!)	no
Interface-based projection	an interface with getters in the same order as the selected columns — easy to write but **bad performance**	no

```java
// 1. custom mapper
@Query(value="select p.pfname, p.plname, co.coname from ...", nativeQuery=true)
List<Object[]> getAllPersonsWithCompanyName();

public class PersonDTOMapper {
    public static PersonDTO toDTO(Object[] o) {
        return new PersonDTO(o[0].toString(), o[1].toString(), o[2].toString());
    }
}
```

```java
// 2. named native query + result set mapping
@NamedNativeQuery(name="Person.getAllPersonsWithCompanyNameNQ",
        query="select p.pfname first, p.plname last, co.coname comp from ...",
        resultSetMapping="PersonDtoMapping")
@SqlResultSetMapping(name="PersonDtoMapping",
        classes=@ConstructorResult(targetClass=PersonDTO.class,
        columns={@ColumnResult(name="first"), @ColumnResult(name="last"),
        @ColumnResult(name="comp")}))
@Entity
public class Person {
    ...
}
```

```java
// 3. class-based projection (JPQL constructor expression, fully qualified name!)
@Query("""
        select new be.abis.demo.dto.PersonDTO(p.firstName, p.lastName, p.company.name)
        from Person p""")
List<PersonDTO> getAllPersonsWithCompanyName();

// 4. interface-based projection
public interface PersonDTO {
    String getFirstName();
    String getLastName();
    String getCompanyName();
}
```

In JpaSolution
DTO + form + `PersonMapper` for persons, `SessionDTO` / `EnrolmentDTO` as Java **records**, mapped inside `AbisTrainingService`; see the Appendix.

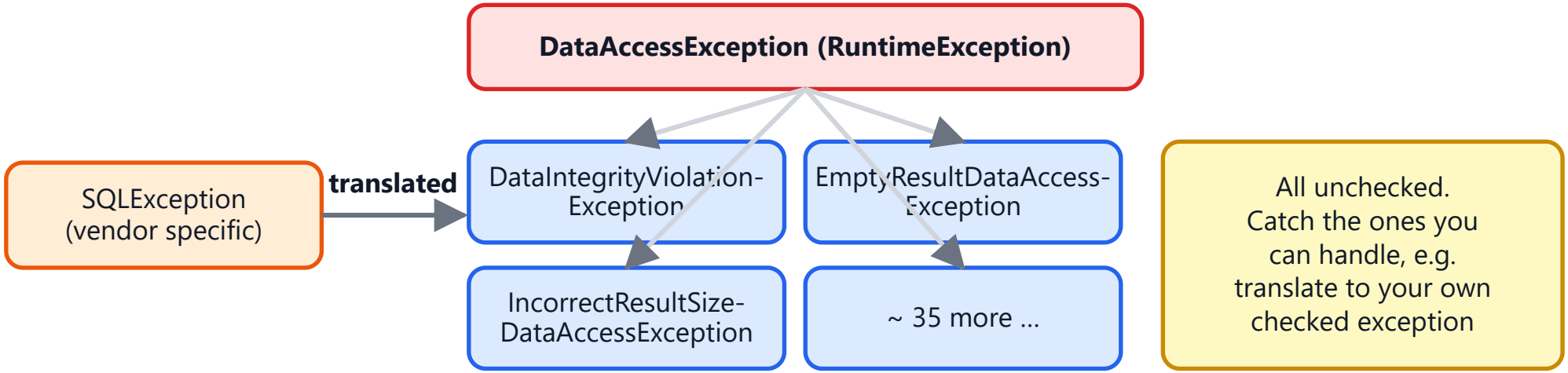
CHAPTER 16

Transaction Management and Exception Handling

Data-access exceptions · modifying queries · understanding transactions · transaction managers, status and definition · declarative and programmatic transactions

16.1 Exception handling

Spring predefines around **35 platform-agnostic** data-access exceptions. The superclass is `org.springframework.dao.DataAccessException`; the names are descriptive: `IncorrectResultSizeDataAccessException`, `DataIntegrityViolationException`, `EmptyResultDataAccessException`, … All are **RuntimeExceptions** (unchecked) — but correct exception handling is still important!



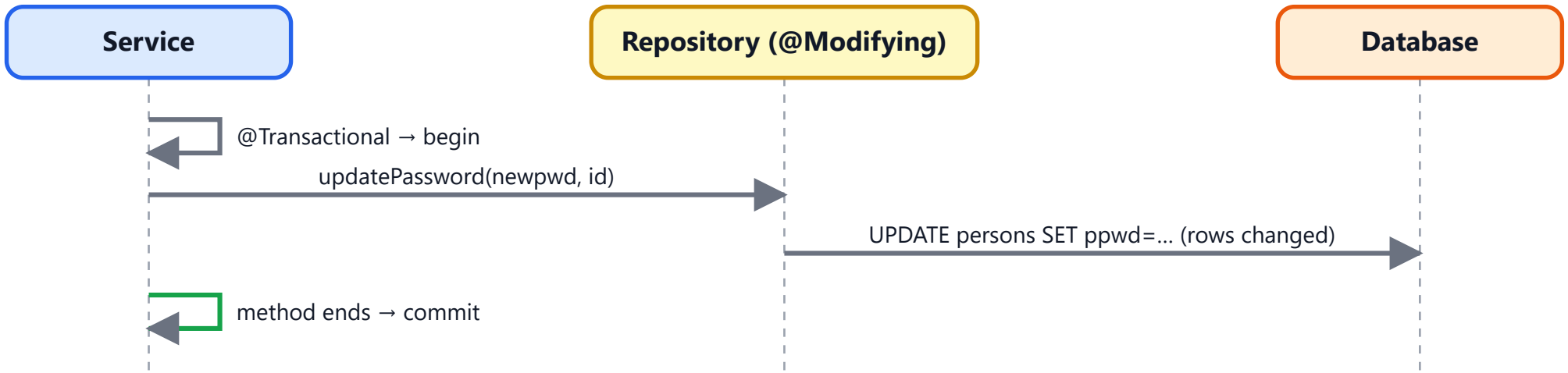
That translation (a vendor-specific `SQLException` becomes a Spring `DataAccessException`) is what `@Repository`'s «persistence exception translation» does (chapter 4).

16.2 Modifying queries

JPQL or native queries can also be used for **DML** (update/delete). Add `@Modifying` on the repository method, and it is **obligatory to call it inside a transaction** (`@Transactional`).

```
public interface PersonJpaRepository extends JpaRepository<Person,Integer> {
    @Modifying
    @Query(value="update persons set ppwd=:newpwd where pno=:id", nativeQuery=true)
    int updatePassword(@Param("newpwd") String newpwd, @Param("id") int id);
}

public class MyPersonService implements PersonService {
    @Transactional
    public void updatePassword(String newpwd, int id) {
        pjr.updatePassword(newpwd, id);
    }
}
```



In JpaSolution

`SessionJpaRepository.cancelSession` (`update sessions set scancel='C' where sno=:id`) and `CourseJpaRepository.changePrice` are modifying native queries. `clearAutomatically = true` makes sure the persistence context does not keep a stale copy of the changed row.

16.3 Understanding transactions

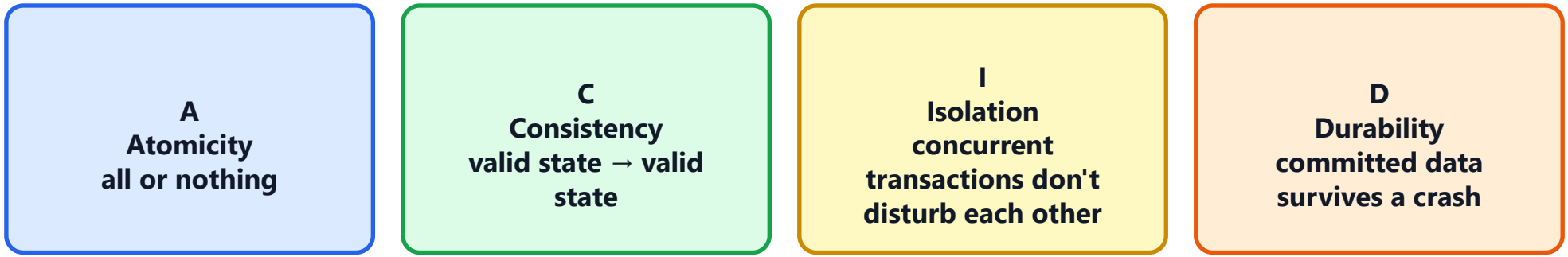
Transactions ensure **all-or-nothing** behaviour: data is never left in an inconsistent state (**ACID**). Hibernate/JPA imposes transaction management. Two kinds:

Global transactions

Work with **multiple transactional resources** (typically relational databases and message queues). Managed by the application server through **JTA**.

Local transactions

Resource-specific, e.g. linked to one JDBC connection. Simpler, no application server involved, but limited.



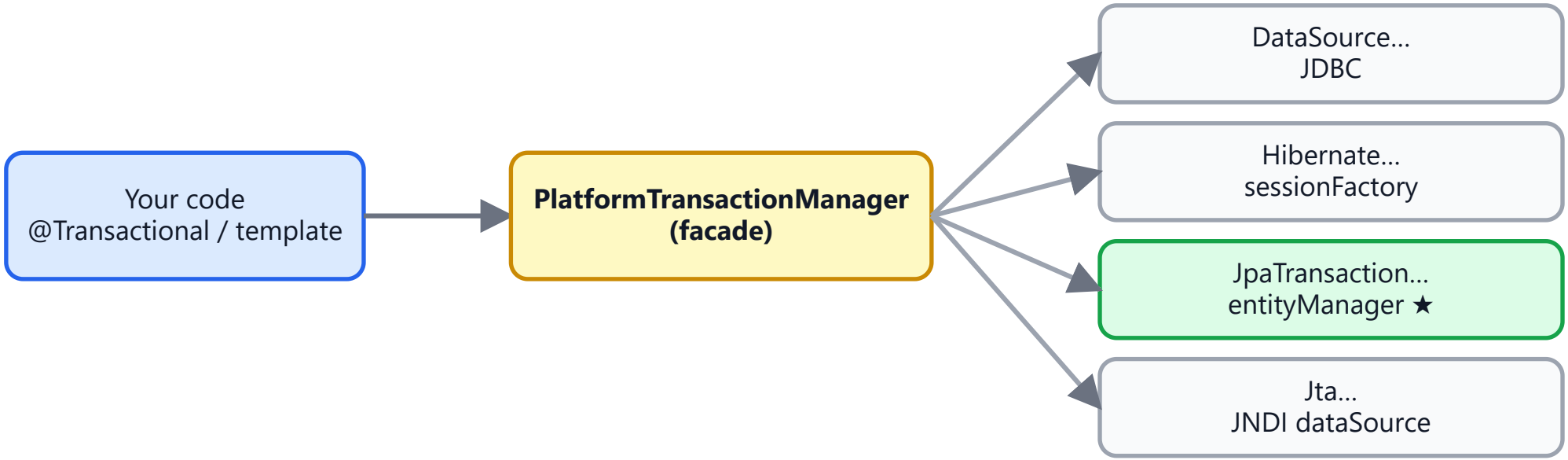
Spring's transaction philosophy

- A **consistent programming model** in any environment (global or local); you only need a JTA application server if you must handle transactions across multiple resources.
- Both **programmatic** and **declarative** management; the real implementation is abstracted away from your code.
- The strategy is based on `PlatformTransactionManager`; `TransactionException` is a RuntimeException.

- By default **everything is auto-committed**, unless `@Transactional` is used.

Transaction managers

Implementations of `PlatformTransactionManager` act as a **facade** to the platform-specific transaction implementation.
`getTransaction()` returns a `TransactionStatus` based on a `TransactionDefinition`.



Implementation	Works with
`DataSourceTransactionManager`	JDBC → a dataSource
`HibernateTransactionManager`	a sessionFactory
`JpaTransactionManager`	an entityManager — **auto-configured by Spring Boot** when no JTA context is found
`JtaTransactionManager`	a dataSource via JNDI (global transactions)

Use them via `@Transactional` (declarative) or a wired-in `TransactionTemplate` (programmatic).

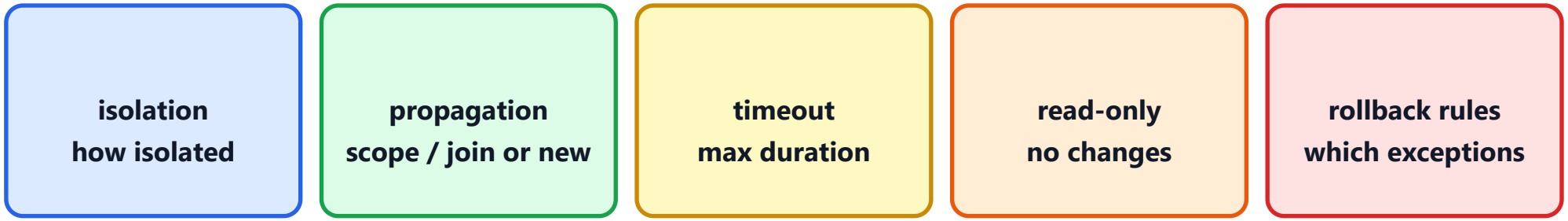
16.4 Transaction status

A `TransactionStatus` may represent a **new** transaction or an **existing** one if a matching transaction exists in the current call stack. It is associated with a **thread**; code can read status information, **programmatically request a rollback** and use savepoints.

Method	Meaning
`flush()`	flush the underlying session to the datastore
`hasSavepoint()`	does the transaction internally have a savepoint?
`isCompleted()`	commit or rollback has happened
`isNewTransaction()`	new, or part of an existing/nested one?
`isRollbackOnly()` / `setRollbackOnly()`	query / mark the transaction as rollback-only

16.5 Transaction definition

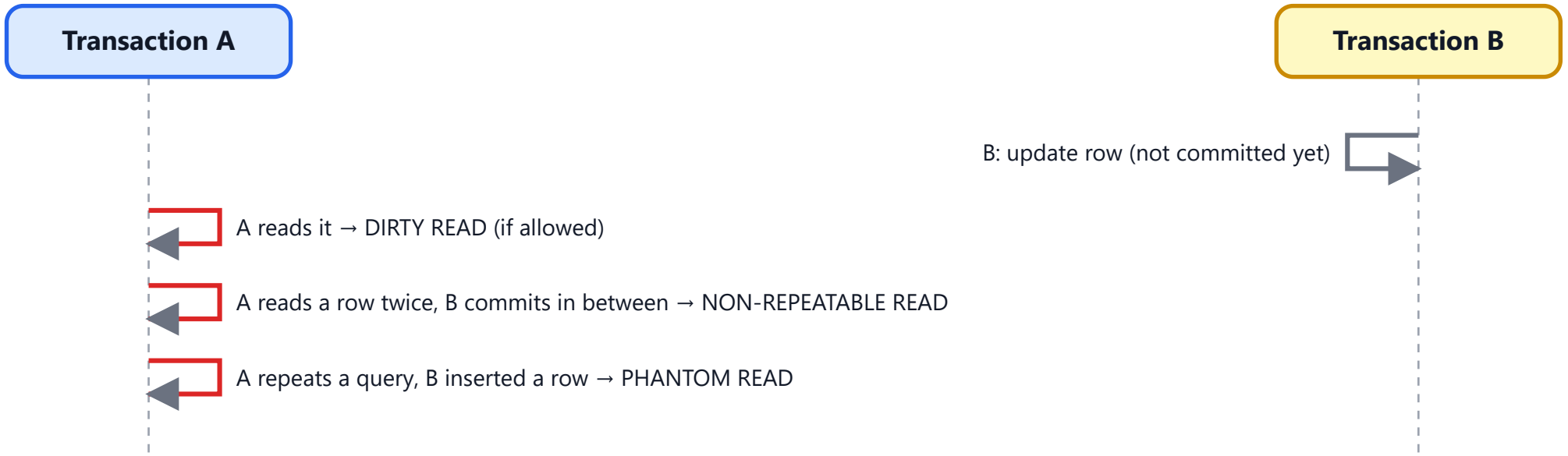
Five properties describe how a transaction behaves:



Isolation level

The degree to which a transaction is isolated from other transactions.

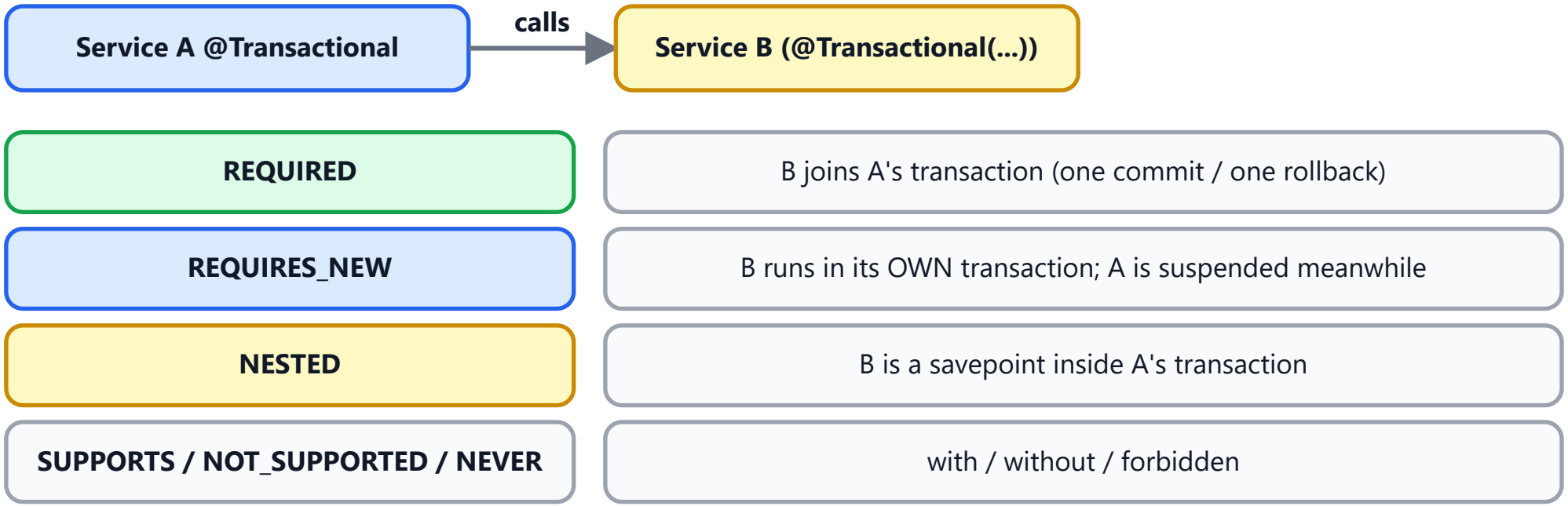
Level	Dirty read	Non-repeatable read	Phantom read
`ISOLATION_READ_UNCOMMITTED`	possible	possible	possible
`ISOLATION_READ_COMMITTED`	prevented	possible	possible
`ISOLATION_REPEATABLE_READ`	prevented	prevented	possible
`ISOLATION_SERIALIZABLE`	prevented	prevented	prevented
`ISOLATION_DEFAULT` default	the default isolation level of the underlying datastore		



Propagation behaviour

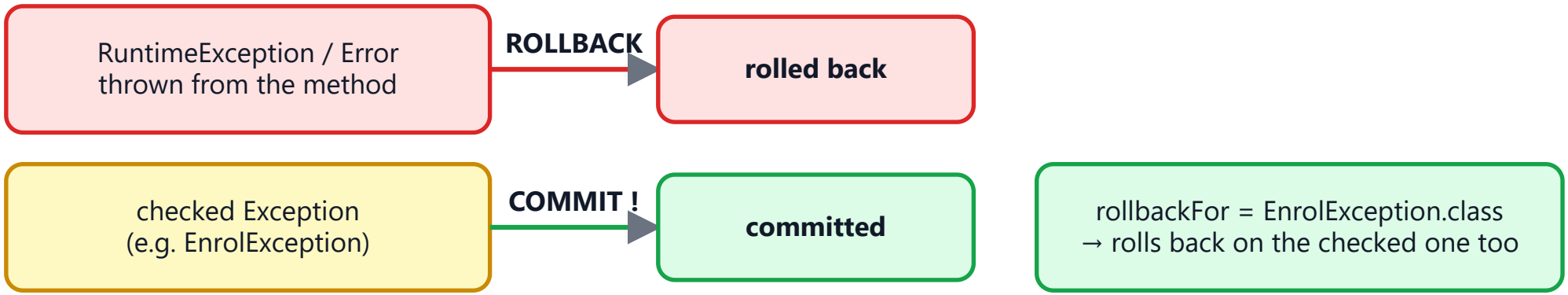
Defines the **transaction scope** — all the propagation options familiar from EJB CMT:

Propagation	Behaviour
`REQUIRED` default	support a current transaction; create a new one if none exists
`REQUIRES_NEW`	always create a new transaction, suspending the current one if it exists
`SUPPORTS`	support a current transaction; run non-transactionally if none exists
`NOT_SUPPORTED`	never run in a transaction; suspend the current one
`NEVER`	no transaction allowed; throw an exception if one exists
`NESTED`	nested transaction (savepoint) if a current one exists, otherwise like REQUIRED
`MANDATORY`	(EJB CMT) must be called within an existing transaction



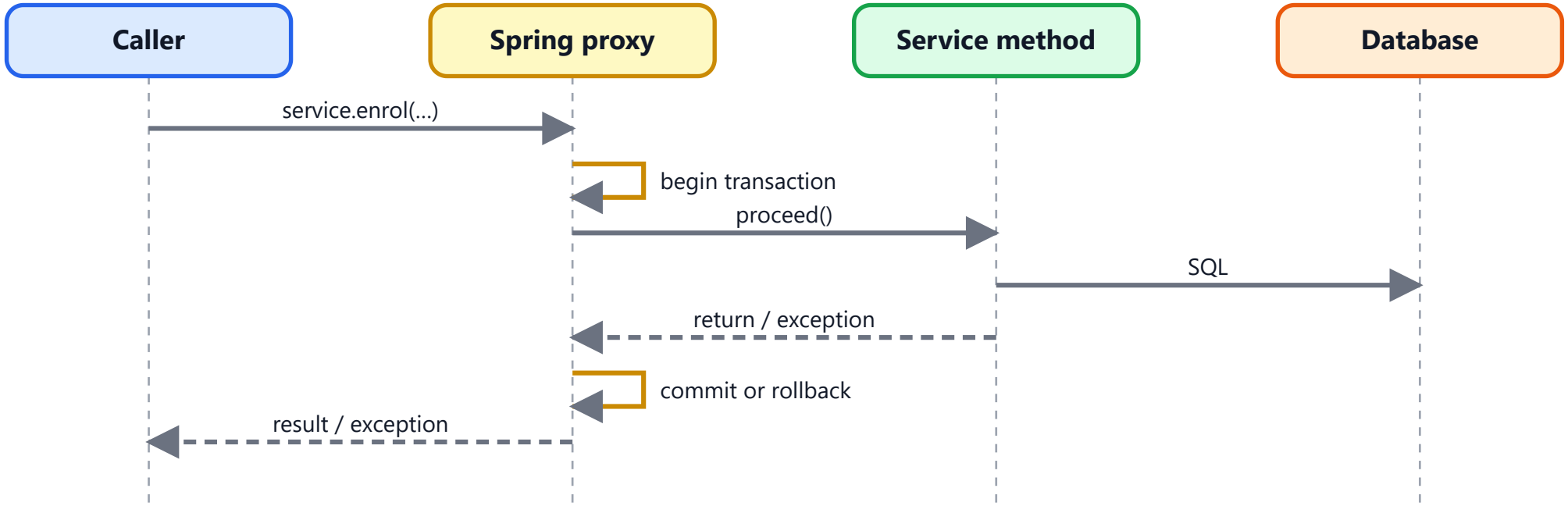
Timeout, read-only, rollback rules

- **timeout**: how long the transaction may run before it is rolled back automatically (int; default `-1` = the underlying system's default).
- **read-only**: is the transaction read-only? (default `false`).
- **rollback rules**: for which exceptions the transaction is rolled back — default **RuntimeException and Error**; checked exceptions need `rollbackFor`.



16.6 Declarative transactions

Implemented through Spring's **AOP framework** (a proxy around your service, chapter 5), annotation-based with `@Transactional` (`org.springframework.transaction.annotation`). The attributes define the transaction definition; only override the non-default ones; method annotations override class annotations. Use it in the **service layer** for DML methods.



- For **delete**: call `myRepo.flush()` so the exception is thrown *within* the transaction boundary (otherwise the constraint violation only appears at commit, outside your try/catch).
- **In JUnit tests**: `@Transactional` on a class/method gives an automatic **rollback**; use `@Commit` to change that. Make sure the test's `@Transactional` does not change the behaviour of the service method; put it only on «**happy flow**» tests — exceptions should be handled by the service layer.

```
@Service
public class AbisReceptionService implements ReceptionService {
    @Autowired private CompanyJpaRepository cjr;
```

```
    @Transactional(isolation = Isolation.SERIALIZABLE,
            rollbackFor = CompanyException.class)
    public void doStuffWithCompanies() throws CompanyException {
        try {
            Company c = new Company(1000, "My Company", "some street", "30", "3000",
                    "Leuven", "B");
            cjr.save(c);
            c.setStreet("Diestsevest");
            cjr.save(c);
            cjr.deleteById(3);
            cjr.flush();  // the SQL runs NOW, inside the transaction
        } catch (EmptyResultDataAccessException | DataIntegrityViolationException e) {
            throw new CompanyException("oops");  // checked → needs rollbackFor
        }
    }
}

@SpringBootTest
public class ReceptionTest {
    @Test
    @Transactional
    public void happyFlow() {
        assertDoesNotThrow(() -> receptionService.doStuffWithCompanies());
    }

    @Test
    public void exception() {
        assertThrows(CompanyException.class,
                () -> receptionService.doStuffWithCompanies());
```

```
    }
}
```

16.7 Programmatic transactions

You control **every step** yourself — more fine-grained than declarative transactions. Use a `TransactionTemplate` that wires in the TransactionManager, implement `TransactionCallback` and call `doInTransaction()`. It is **intrusive** in your code: wire the template into your services.

```
@Service
public class SimpleService {
    private final TransactionTemplate transactionTemplate;

    @Autowired
    public SimpleService(PlatformTransactionManager tm) {
        this.transactionTemplate = new TransactionTemplate(tm);
    }

    public Object someServiceMethod() {
        return transactionTemplate.execute(new TransactionCallback() {
            public Object doInTransaction(TransactionStatus status) {
                try {
                    updateOperation1();
                    updateOperation2();
                } catch (SomeBusinessException ex) {  // ask for a rollback
                    status.setRollbackOnly();
                }
                return null;
            }
        });
```

```
    }
}
```

	Declarative `@Transactional`	Programmatic `TransactionTemplate`
code	one annotation, business code stays clean	transaction code mixed with business code
control	per method (attributes)	every step, very fine-grained
implemented by	AOP proxy	template callback
use	**default choice**	special cases

In JpaSolution: the enrol flow

`AbisTrainingService.enrolForSession` is `@Transactional(rollbackFor = EnrolException.class)`: a new person inserted at the start is **rolled back** when the session turns out not to exist (see the Appendix for the flowchart).

CHAPTER 17

Securing REST APIs

Authentication vs. authorization · API keys · Spring Security · HTTP basic & digest · authorizing access · OAuth 2.0 / OpenID Connect · HTTPS · CORS

17.1 Securing APIs

Security is an important part of any enterprise application — **not all APIs are (or should be) public!** Two essential elements:

Authentication
verifying the IDENTITY of the user
trying to access the API
"who are you?"

Authorization
verifying what an authenticated user
is PERMITTED to do in the API
"what may you do?"

Ways to secure a REST API: **API keys**, **HTTP basic / digest authentication**, **OAuth 2.0**, and **authorizing** the accesses to the RESTful web service.

Application security vs. server security

Application security

- the application has its own way to check authentication & authorization (custom database)
- e.g. login screen; other pages check that login was done; authorization via roles
- «custom» encoding mechanisms via Java frameworks

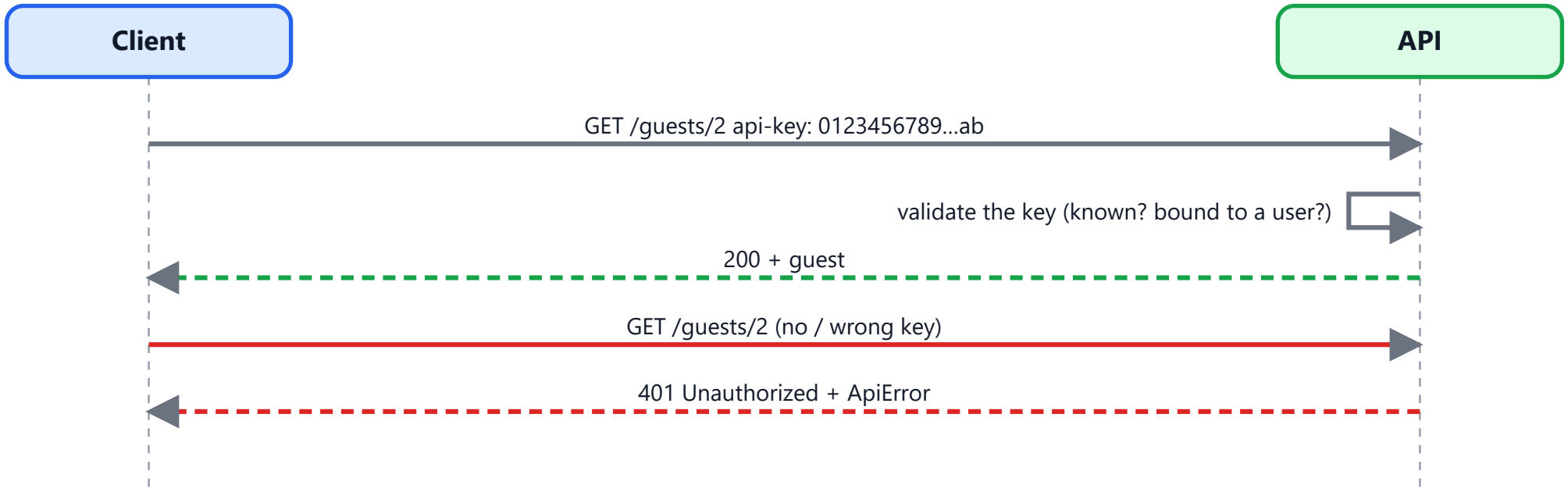
Server security

- the **server** defines users and roles (possibly via LDAP); usable for several applications at once
- the application is **not called** when authentication fails
- encryption via HTTPS, digest authentication …

17.2 API keys

An API key is a **token that a client provides when making API calls**, supplied by the API provider (e.g. when creating an account).

- a **token**: a relatively long random string (e.g. 32 characters); an **identifier** for storage and unique identification;
- transmitted with the request (**requires SSL in production**) in the HTTP header or a cookie — convention: call it `api-key`;
- known to the client, **validated by the server**; unique to a device or software (e.g. a UUID); bound to a user if necessary.



API key in the API — and in the client

```
@GetMapping("{id}")
public Guest returnGuestById(@PathVariable("id") int id,
        @RequestHeader MultiValueMap<String,String> headers)
        throws GuestNotFoundException, ApiKeyNotCorrectException {
    if (headers.containsKey("api-key")
            && checkToken(headers.get("api-key").get(0))) {
        return receptionService.findGuest(id);
    }
    throw new ApiKeyNotCorrectException("please provide a valid API KEY");
}

// the exception handler maps it to
// HttpStatus.UNAUTHORIZED (401)
```

```
// client
HttpHeaders requestHeaders = new HttpHeaders();
requestHeaders.add("api-key", "0123456789012345678901234567890ab");
HttpEntity<Guest> requestEntity = new HttpEntity<>(requestHeaders);
try {
    re = rt.exchange(baseUrl + "/2", HttpMethod.GET, requestEntity, Guest.class);
} catch (HttpStatusCodeException e) {
    if (HttpStatus.NOT_FOUND == e.getStatusCode()
            || HttpStatus.UNAUTHORIZED == e.getStatusCode()) {
        ApiError ae = new ObjectMapper().readValue(e.getResponseBodyAsString(),
                ApiError.class);
    }
}
```

Exercise idea (JpaSolution context)

An `ApiKeyStore` holds an API key per user id; the key is sent to the client when the user logs in correctly; `changePassword` requires that key, and the service checks it — so a person can only change their **own** password.

17.3 Introduction to Spring Security

- Provides **declarative security** for Spring-based applications; roots in *Acegi Security*.
- Tackles security from **two angles**: securing **web requests** via servlet filters, and securing **method invocations** using AOP.
- Dependency: `spring-boot-starter-security` (spring-security-config, spring-security-web, spring-aop, spring-web).

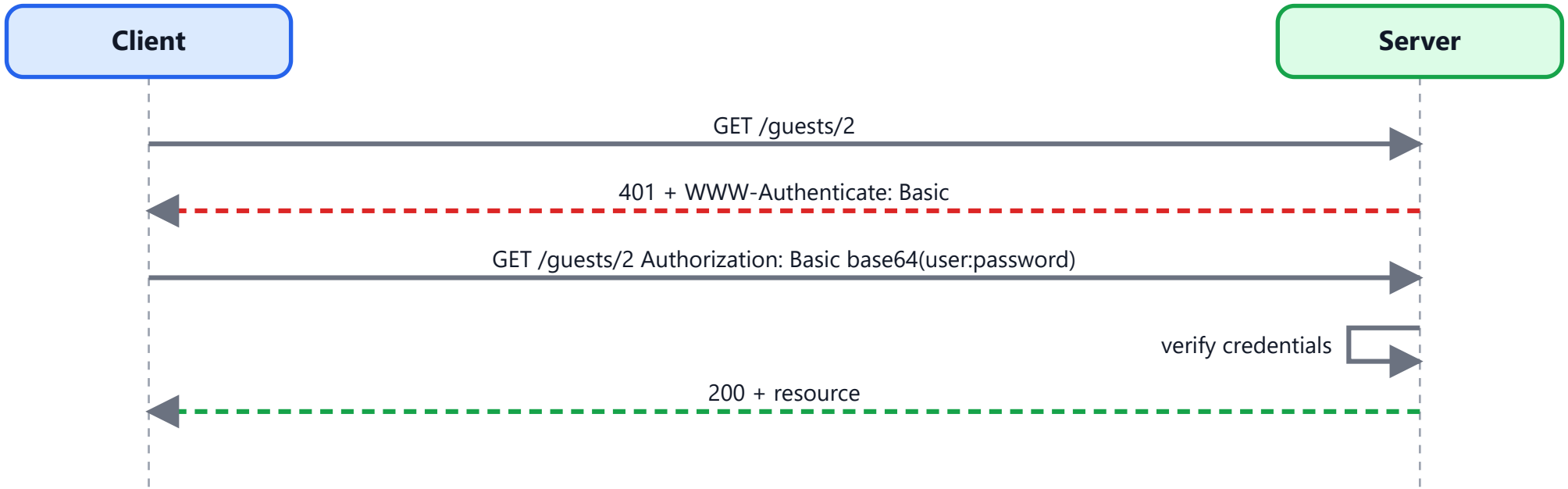
1) web requests are secured by servlet filters 2) methods are secured with AOP



17.4 HTTP Basic and Digest authentication

Basic

The client sends a **Base64-encoded** username and password pair in the HTTP `Authorization` header, with **every** request. Chain of events:



Deficiencies of Basic

- cached credentials could be misused; no way to differentiate authorised from unauthorised requests;
- user and password are only **encoded** (Base64 is easy to decipher), not secured — the intent of Base64 is uniform character transfer;
- not recommended over plain HTTP on the Internet → **solve it with HTTPS**.

Digest

The password is **not sent in clear text**: the client sends a one-way cryptographic **hash** (MD5) of the username, password and other security-related fields; the server regenerates the hash from the same fields and compares. Equal hashes → authenticated. *Not every server (e.g. WAS) supports digest authentication.*

Configuring minimal security

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {}
```

The default configuration demands that **each HTTP request is authenticated**; since no user store is defined, nobody can log in yet; the user is rerouted to a default login screen. To do: configure a user store, specify which requests need authentication and which authorities, optionally a custom login screen. **Since Spring Boot 3** the config class can no longer extend `WebSecurityConfigurerAdapter`!

Creating a user store

Spring authenticates against virtually any store: **in-memory**, relational database, LDAP, or a custom implementation. Since Spring 5 a `PasswordEncoder` is required. In memory: a `PasswordEncoder` bean + a `UserDetailsService` bean returning an `InMemoryUserDetailsManager`.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
```

```
        UserDetails user = User.withUsername("user1")
                .password(encoder.encode("pwd1")).roles("USER").build();
        UserDetails admin = User.withUsername("user2")
                .password(encoder.encode("pwd2")).roles("USER","ADMIN").build();
        return new InMemoryUserDetailsManager(admin, user);
    }
}
```

Intercepting requests

Decide which URIs need login: add a bean returning a `SecurityFilterChain` (taking an `HttpSecurity`), use `authorizeHttpRequests()` with ANT-style matching; a missing login gives **401**. (CSRF is disabled for the moment.)

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests(a -> a
            .requestMatchers("/guests").permitAll()         // everybody (also POST!)
            .requestMatchers("/guests/*").authenticated())  // /guests/1 ... need a login
        .httpBasic(Customizer.withDefaults())
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}
```

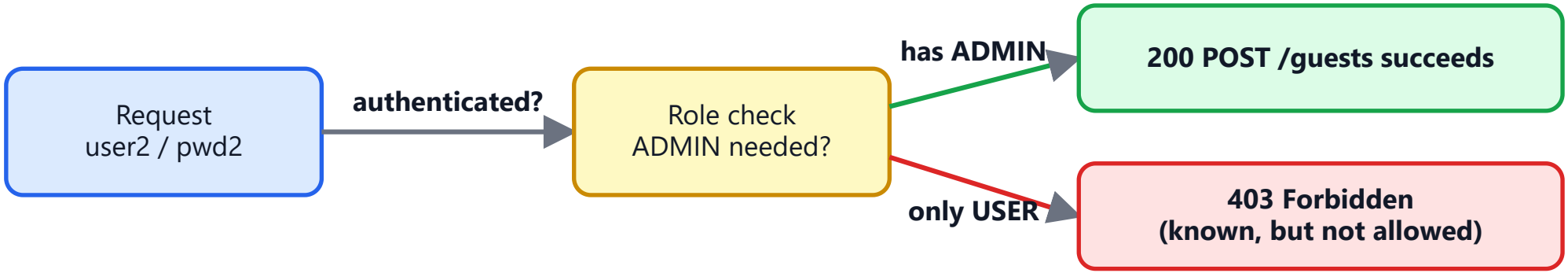
HTTP Basic in the client

The client must now provide credentials for certain requests. A basic `RestTemplate` bean no longer suffices: create an interceptor class (needs `httpclient5`), set up the RestTemplate through a custom component using a Spring `FactoryBean` that refers to the interceptor, and add `BasicAuthenticationInterceptor(user, pwd)` on the RestTemplate for the requests that need it. Make sure exception handling treats the **401** and **403** statuses.

```
rt.getInterceptors().add(new BasicAuthenticationInterceptor("user1", "pwd1"));
// now carries  Authorization: Basic dXNlcjE6cHdkMQ==
re = rt.exchange(baseUrl + "/2", HttpMethod.GET, null, Guest.class);
```

17.5 Authorizing access

Some actions need **extra authorization on top of authentication** (e.g. POST instead of GET). Authorization is **role-based** (roles are passed via the SecurityConfig); Spring uses **method level security** to enforce it. The client stays the same as with basic authentication — don't forget to catch **401** and **403**.



wrong credentials → 401 instead

Securing methods with Spring

A **second line of defence** next to web security: secure bean methods. Three kinds of annotations: Spring Security's own `@Secured`, JSR-250's `@RolesAllowed`, and fine-grained expression-driven annotations. Configure with `@EnableMethodSecurity(jsr250Enabled=true)` on the security config class.

```
@PostMapping("")
@RolesAllowed("ADMIN")
public void addGuest(@Valid @RequestBody Guest guest) {
    receptionService.addGuest(guest);
}

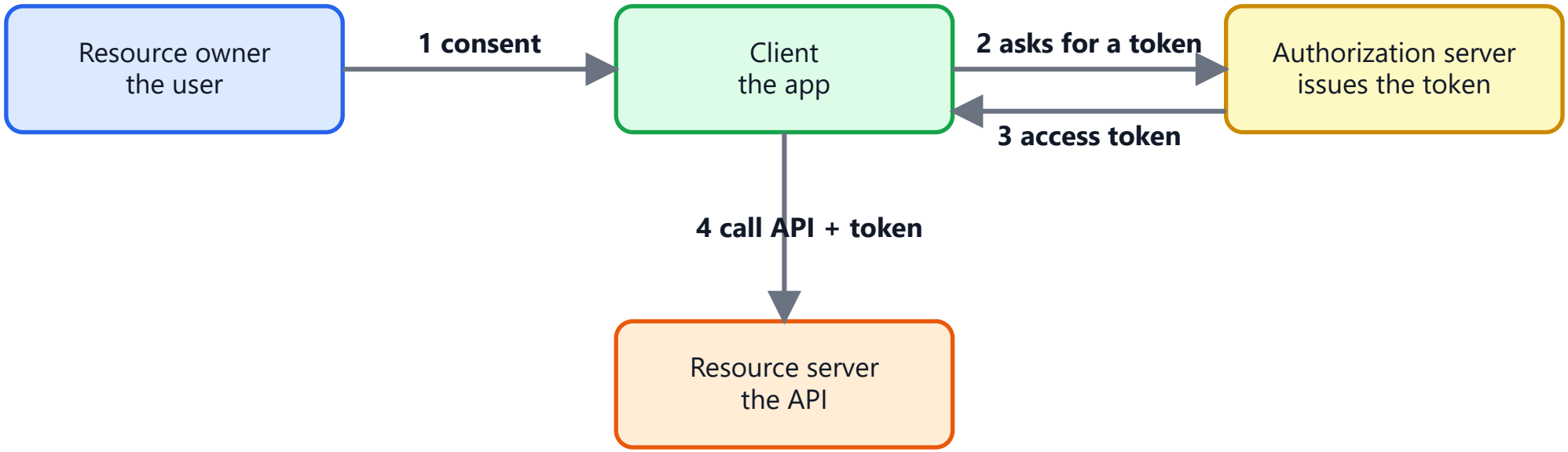
// not allowed → HttpStatus.FORBIDDEN (403)
```

Status	Meaning
401	not authenticated (unknown user / wrong or missing credentials)
403	authenticated, but **not allowed** (missing role)

17.6 Introduction to OAuth 2.0 and OpenID Connect

OAuth 2.0

An **authorization framework** / open standard used by many enterprises and service providers to protect resources. Authentication is done via an **Authorization Server** (can be a 3rd party such as Google). Goals: **delegate access**, **avoid sharing passwords**, use **short-lived access tokens**, allow secure app integration.

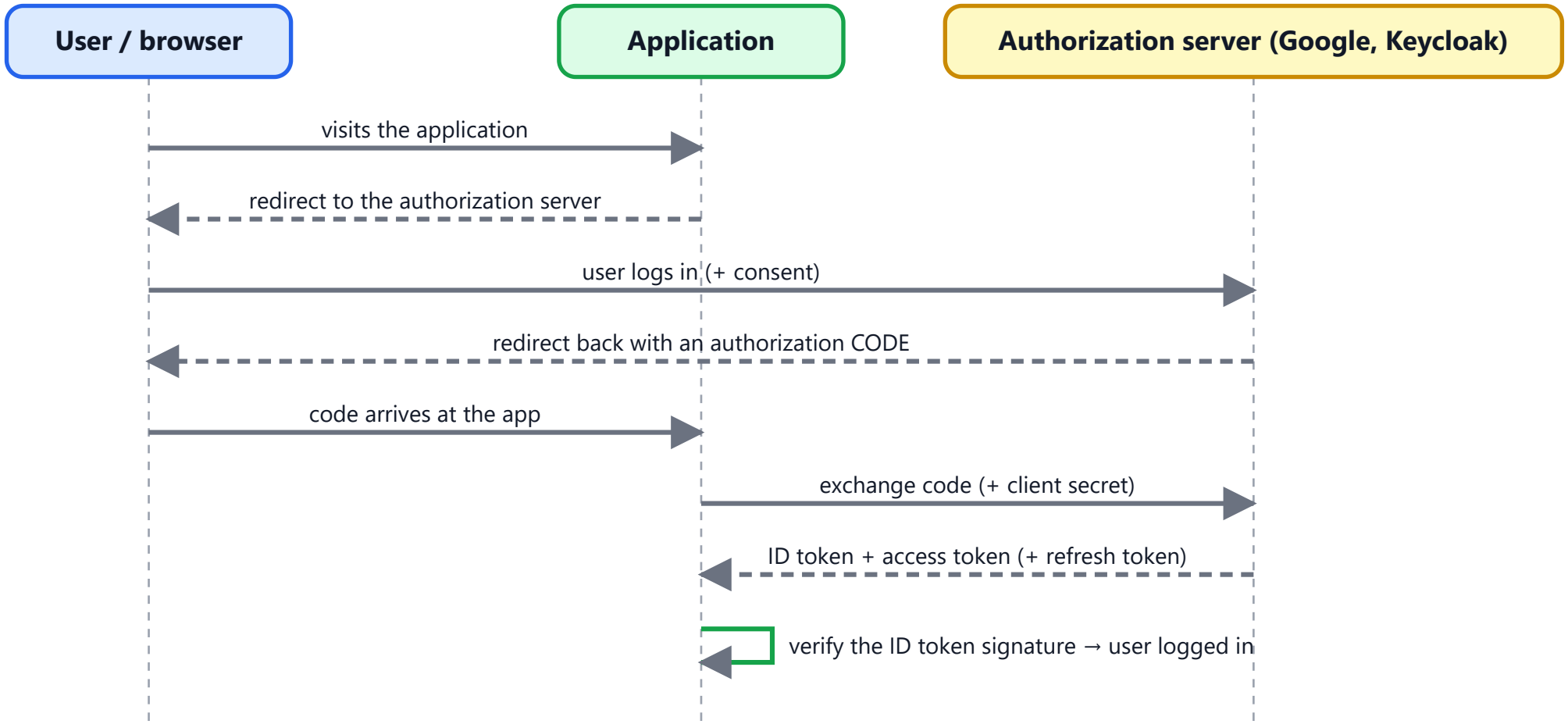


OAuth is NOT authentication
It does not provide identity: an access token only means «you are *allowed*», not «you *are* this person». Another protocol is needed for authentication → OpenID Connect.

OpenID Connect (OIDC)

Built on top of (including) OAuth 2.0, adding **authentication**: an **ID Token** (a JSON Web Token, JWT, containing the user's identity) and a **userInfo endpoint** (profile information). Standard scopes: `openid`, `profile`, `email`.

Authorization code flow (typical)



Integration in Spring

```
<artifactId>spring-boot-starter-oauth2-client</artifactId>

spring.security.oauth2.client.registration.google.client-id=YOUR_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_CLIENT_SECRET
spring.security.oauth2.client.registration.google.scope=openid,profile,email
spring.security.oauth2.client.provider.google.issuer-uri=https://accounts.google.com

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth.requestMatchers("/public").permitAll()
            .anyRequest().authenticated())
            .oauth2Login()    // enables OpenID Connect login
            .oauth2Client();  // enables OAuth2 client capabilities
    return http.build();
}

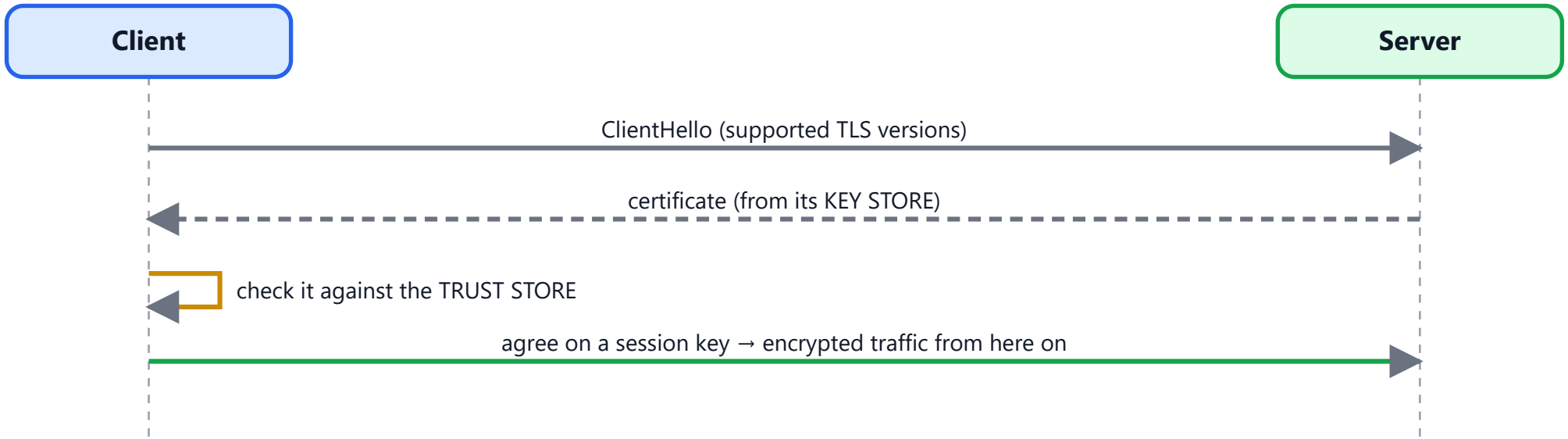
@GetMapping("/me")
public String me(@AuthenticationPrincipal OidcUser user) {
    return "Hello " + user.getFullName();
}
```

The Client ID and Secret come from a project in the Google Cloud Console (enable the OAuth consent screen and credentials).

17.7 HTTPS — trust stores and key stores

> **Note**
> In the course text this section is only a title slide with a demo. The ideas below are the standard ones.

HTTPS = HTTP over TLS: the connection is **encrypted** (nobody can read the traffic) and the server proves its identity with a **certificate**. That also repairs the weakness of Basic authentication and API keys.



Store	Contains	Used by
Key store	your **own** private key + certificate (who *you* are)	the server (e.g. `server.ssl.key-store` in Spring Boot); a client when it must authenticate itself
Trust store	certificates of parties (or CAs) you **trust**	the client, to decide whether to accept the server's certificate

17.8 CORS

CORS = **C**ross-**O**rigin **R**esource **S**haring: a browser security mechanism that restricts HTTP requests from one **origin** to resources of another origin. **Origin = protocol + domain + port.** It prevents unauthorised cross-site calls; **servers must explicitly allow** cross-origin requests.



Without that response header the browser blocks the answer (the server still ran the request!)
Solutions: @CrossOrigin on the API, or an Angular proxy so the browser only talks to its own origin

Configuration

```
// 1. in the API, on the controller
@CrossOrigin(origins = "http://localhost:4200")
@RestController
public class PersonApiController {
    ...
}
```

```
// 2. via a proxy in the client (Angular)
// proxy.conf.json
{ "/ngpersonapi": { "target": "http://localhost:8080",
                    "secure": false } }
// angular.json → serve → options
"proxyConfig": "proxy.conf.json"
```

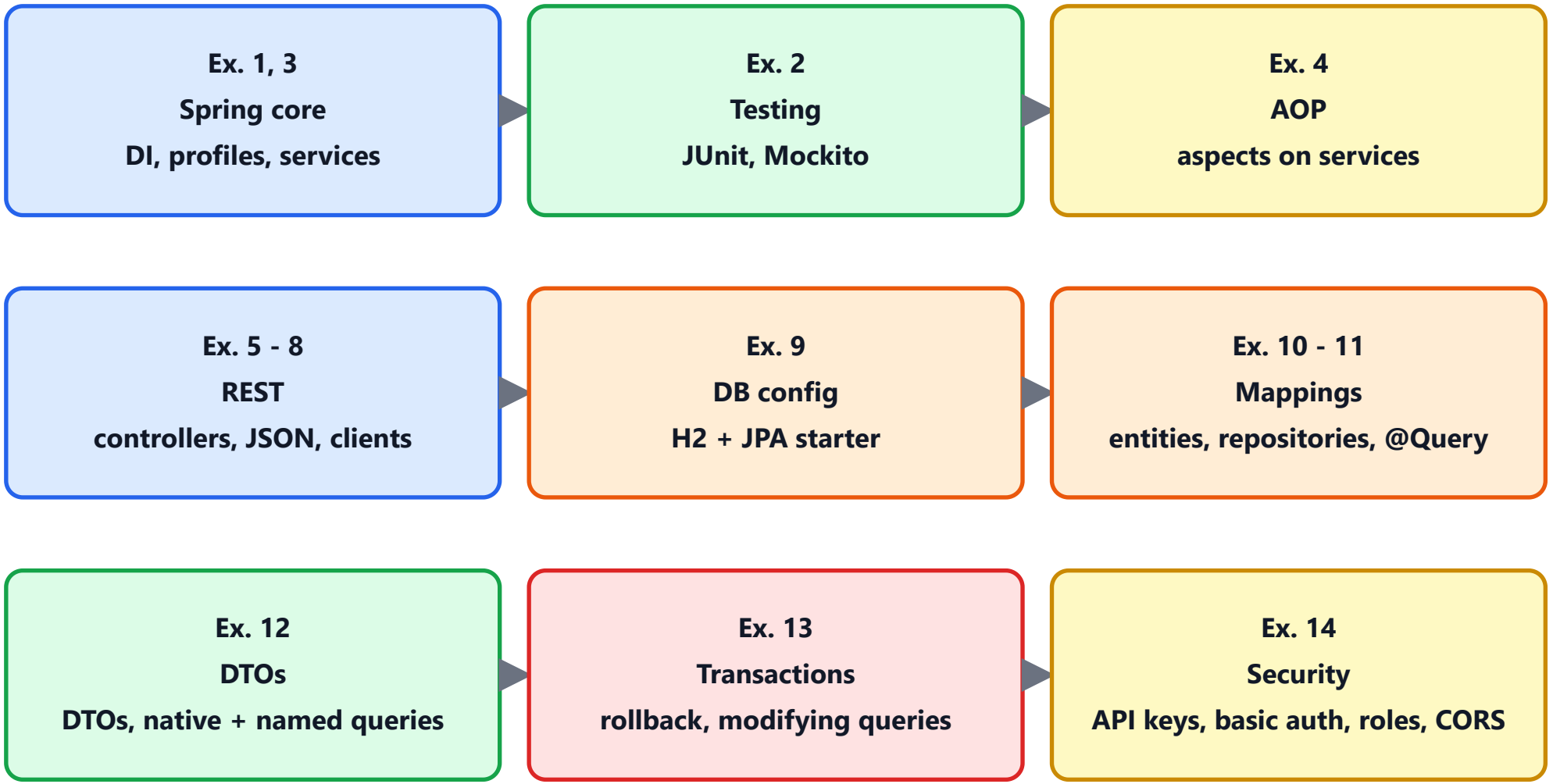
APPENDIX A

The Exercises and the JpaSolution Project

The exercise roadmap · the finished JPA solution in pictures

A.1 The exercises: how one application grows

The exercises of the course all build **one application**: a training-administration system (*persons, companies, courses, sessions, enrolments*) that starts as a plain Java skeleton and gains one technique per chapter.



Exercise group	What you build
1 Getting started	Spring Boot project; make the repositories Spring-managed with `@Component`; run `contextLoads()`
2 Testing	`CourseRepositoryTest`, `PersonRepositoryTest` (assertEquals / assertNull / assertNotNull, `@BeforeEach`), `@TestMethodOrder`
3 Spring core	Services (`CourseService`, `PersonService`, `TrainingService`) with the right stereotypes; Mockito test; constructor / setter / `@Value` injection; `@PostConstruct`; Java config instead of annotations; a second `PersonService` + **profiles** (prod / dev); runtime value injection with `my.properties`
4 AOP	`LoginAspect` (after login, after wrong login); timer for all `find*` methods in the repository package
5 Spring REST	`PersonController` with URIs following the naming rules; GET → POST for login; `LocalDate birthDate` with `d/M/yyyy`; company query that returns **XML** (Jackson JAXB); `addPerson` consuming XML *and* JSON
6 REST clients	A `RestTemplate` client implementing the service methods, tested with JUnit; extra: exchange-rate client for `open.er-api.com`
7 Exception handling & validation	`ResponseEntity` for `findPerson`; `PersonAlreadyExistsException` / `PersonCanNotBeDeletedException`, centralised; password length > 5 with bean validation
8 Additional topics	Swagger documentation
9 Database configuration	JPA + H2 dependencies, datasource properties, `DataSourceTest`, create the DB with `createH2DB.sql`
10 Entity mappings & basic JPA repositories	Map `Course`; `CourseJpaRepository` (fetch before update; DML tests with and without `@Transactional`); look at the SQL; add a course *without* passing the id
11 Association mapping & @Query	`Company` + `Address` mapping, `CompanyJpaRepository` (find by name; JPQL by name and town); `Person` mapping (new / existing / no company); `Session` mapping + JPQL for DB2BAS
12 More querying & performance	`PersonDTO` / `PersonForm` / `PersonMapper`; `SessionDTO` record; native query for non-cancelled sessions; `Enrolment` with composite id, `@NamedQuery`, `EnrolmentDTO`
13 Exception handling & transactions	`PersonAlreadyExistsException` again; delete a session (`SessionCanNotBeDeletedException`); modifying native queries to change a course price and to cancel a session; transactional `enrolForSession` with rollback

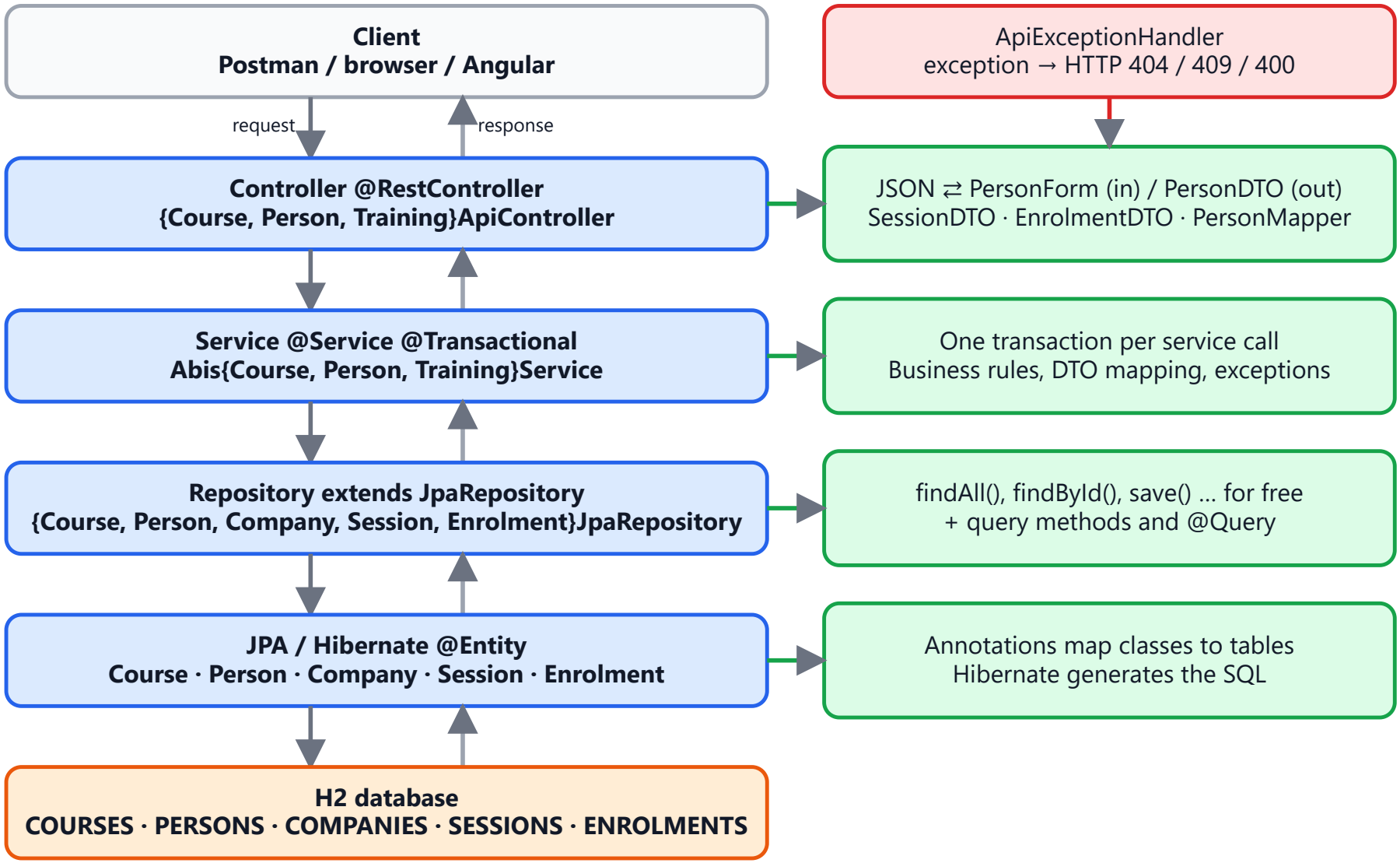
14 Spring Security	API keys so a person can only change their **own** password; in-memory users `abis01` (AbisUser) / `abis02` (AbisUser + AbisAdmin); basic authentication; only AbisAdmin may DELETE courses; CORS for Angular on port 4200

> **The solution project**
> The pages below describe `JpaSolution`, which implements exercise groups **9 to 13** (the JPA part). It keeps the old code as `// BEFORE:` comments next to the new code.

A.2 The layers of the application

A request travels top to bottom; the answer travels back up. Each layer has exactly one job. The exercises replace the **bottom two layers**

(a CSV file and an in-memory list) by a real database.

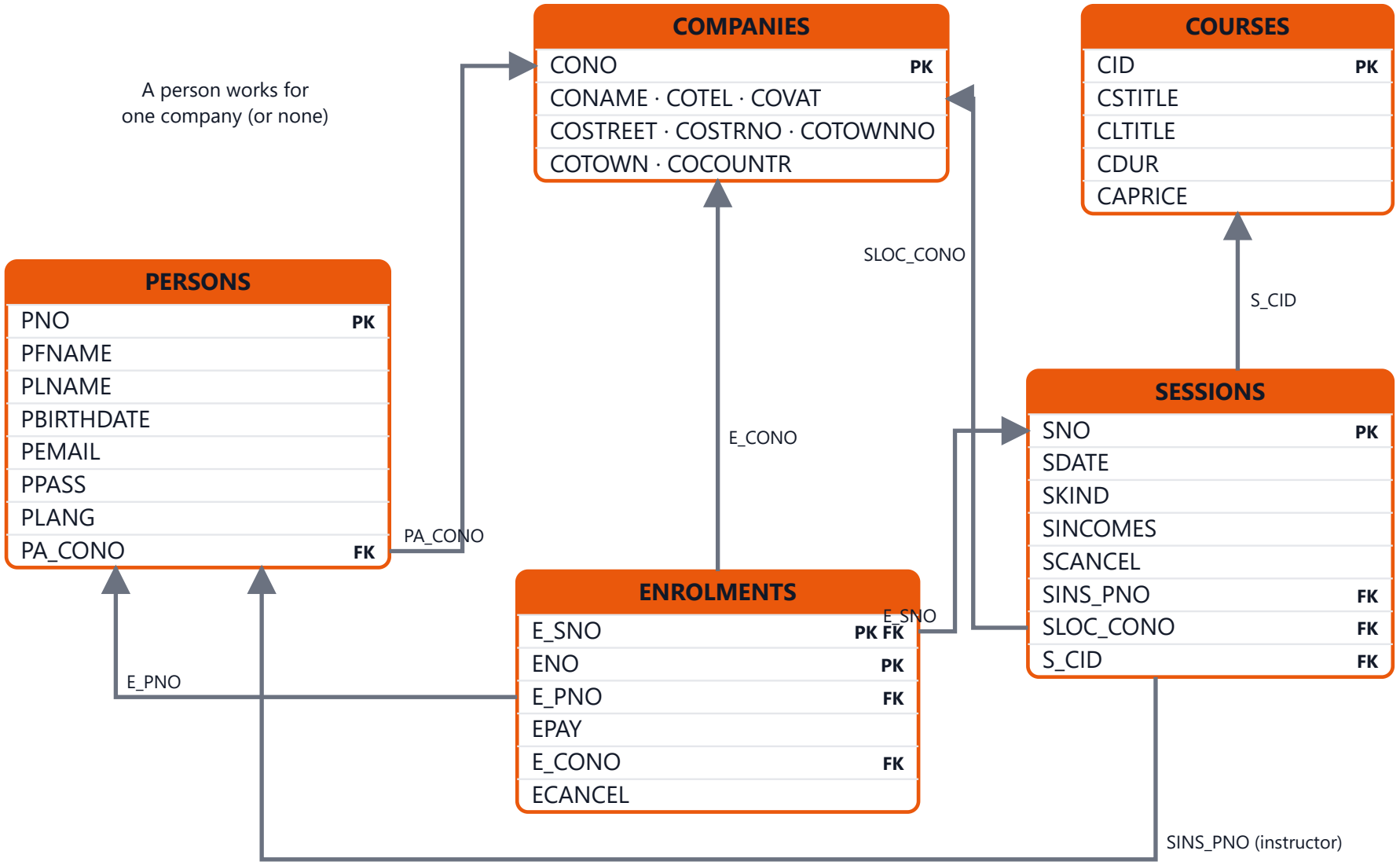


Java / Spring Database Error handling

Rule of thumb: the controller never talks to a repository directly, and entities never leave the service layer as JSON (from chapter 12 on, DTOs and forms are used instead).

A.3 The database (what the entities must mimic)

Five tables. An arrow points from the table that holds the **foreign key** to the table it refers to (many → one).

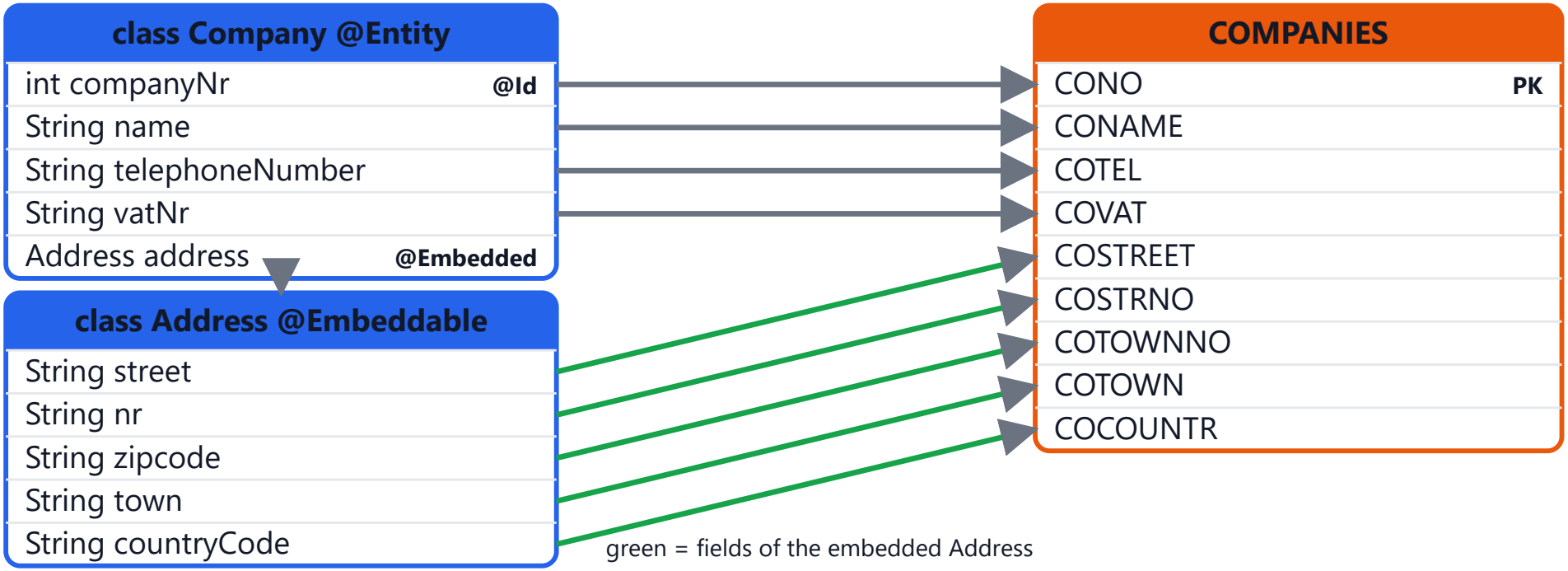


A.4 Annotation cheat sheet

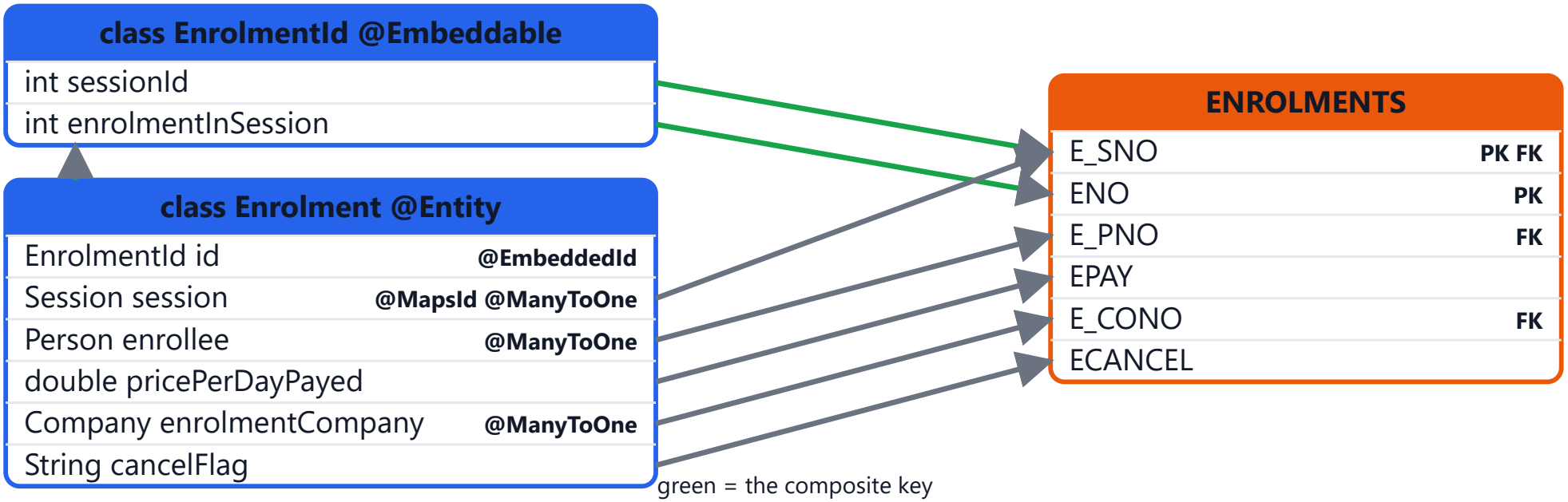
Annotation	Meaning	Used in the solution
`@Entity` `@Table(name=..)`	This class is stored in that table	all model classes
`@Id` `@GeneratedValue(IDENTITY)`	Primary key, filled by the database (auto-increment)	`Course.courseId`, `Person.personId`
`@Column(name=..)`	Property ↔ column when the names differ	`shortTitle` ↔ `CSTITLE`
`@ManyToOne` `@JoinColumn`	Many rows point to one row via a foreign key column	`Person.company` → `PA_CONO`
`cascade = PERSIST`	Saving the person also inserts a new company	`Person.company`
`@Embeddable` / `@Embedded`	Value object stored in the *same* table	`Address` in `COMPANIES`
`@EmbeddedId` `@MapsId`	Composite primary key	`Enrolment` (`E_SNO` + `ENO`)

A.5 From class to table — two mappings that need a closer look

a) One table, two classes — `Address` is not an entity: its fields become extra columns of `COMPANIES`.



b) Composite primary key — an enrolment is identified by *session number + enrolment number in that session*. The key columns live in a small `@Embeddable` class, and `@MapsId` says that the `session` association supplies the `E_SNO` part.

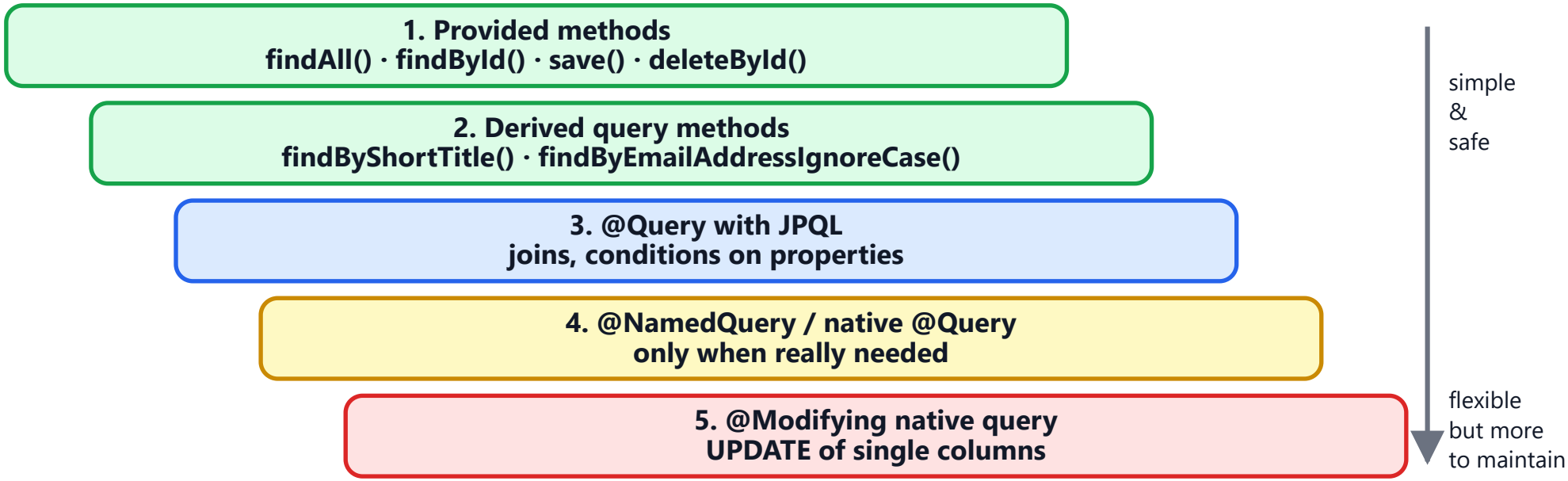


Cancelled? The database stores a letter (`'C'`) or `NULL` in `SCANCEL` / `ECANCEL`. The solution maps it as a `String cancelFlag` and derives `isCancelled()` from it, so queries can simply say `where scancel is null`.

A.6 Five ways to query

Start with the simplest one that works. Every repository extends `JpaRepository<Entity, IdType>`, which already gives `findAll()`,

`findById()`, `save()`, `deleteById()` … without writing any code.

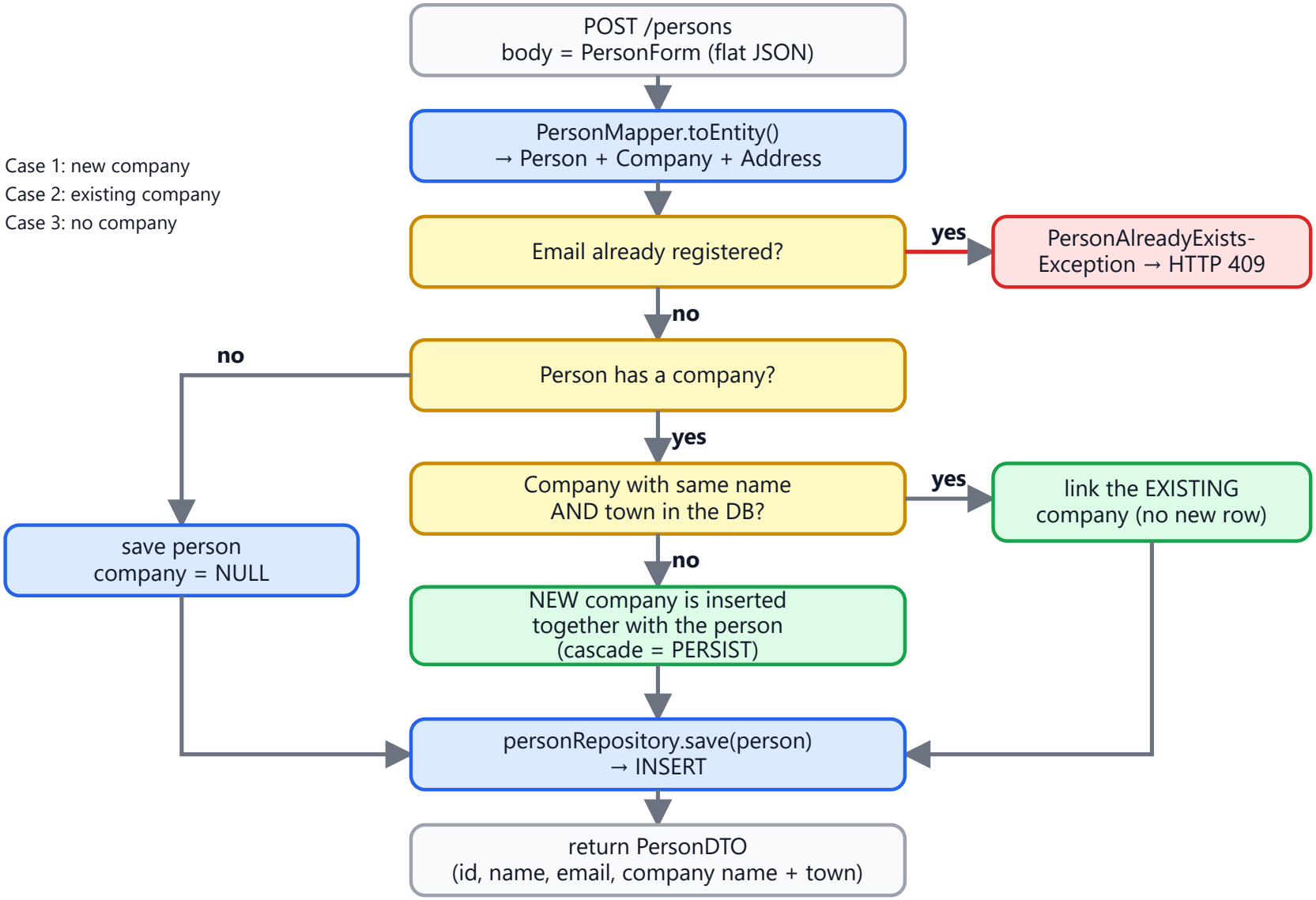


Kind	Example from the solution	What it does
Derived query	`findByShortTitle(String t)` `findByCompanyNameIgnoreCase(String n)`	Spring builds the SQL from the method name; `CompanyName` follows `person.company.name`
JPQL `@Query`	`select c from Company c where upper(c.name)=upper(:name) and upper(c.address.town)=upper(:town)`	Query on **classes and properties**, not on tables and columns
Native `@Query`	`select s.* from sessions s join courses c on s.s_cid=c.cid where ... and s.scancel is null`	Plain SQL, returns entities; needed for DB-specific SQL
`@NamedQuery`	`Enrolment.findEnrolmentsForPerson` on the entity, repository method `findEnrolmentsForPerson(personId)`	Query defined once on the entity; the repository method name must match
`@Modifying` native	`update sessions set scancel='C' where sno=:id` `update courses set caprice=:price where cid=:id`	Changes only *one column*, returns the number of rows; **needs a transaction**

JPQL vs SQL: `from Company c where c.address.town = :t` (JPQL, property path) ↔ `from companies where cotown = ?` (SQL, column). Hibernate translates the first into the second.

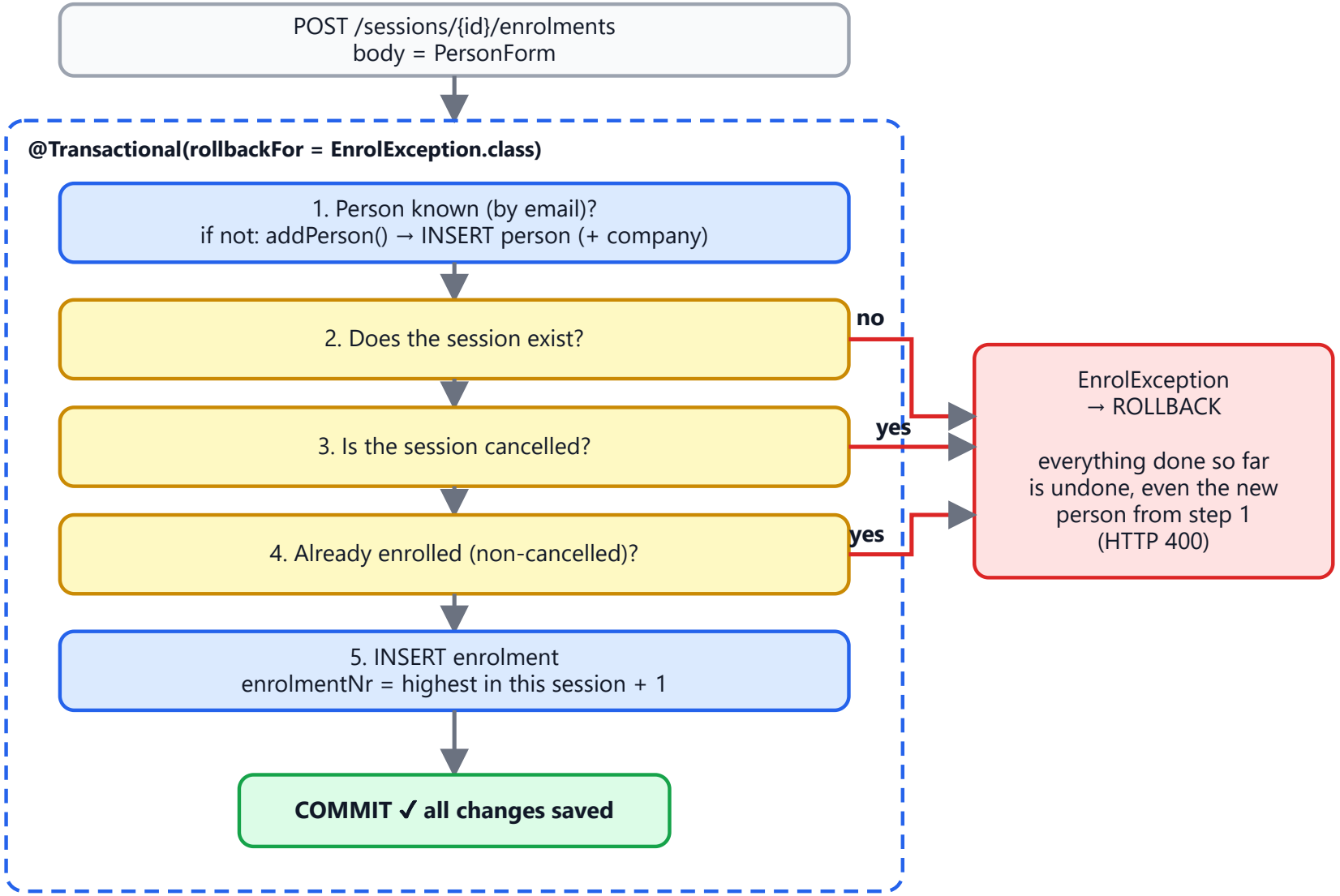
Adding a person (exercise 11.2)

The same service method handles all three cases from the exercise: new company, existing company, no company.



A.8 Enrolling for a session — transaction and rollback (exercise 13.5)

All steps run in **one transaction**. If any check fails, an `EnrolException` is thrown and the database is put back exactly as it was — including a person that was inserted in step 1.



Testing tip. A test method marked `@Transactional` is rolled back automatically after it finishes, so the test data never reaches the database and tests stay independent (no `@Order` needed). Tests that exercise the transaction of the service itself (cancel, enrol) must *not* be `@Transactional`, so they clean up after themselves.

A.9 Which exercise lives where?

Exercise	Task	Classes in `JpaSolution`
9.1–9.3	Dependencies, H2 settings, test the connection, create the DB	`pom.xml`, `application.properties`, `createH2DB.sql`, `DataSourceTest`
10.1–10.2	Map `Course`, replace the course repository, DML tests	`Course`, `CourseJpaRepository`, `AbisCourseService`, `CourseServiceTest`
11.2	Map `Person`, add person (new / existing / no company)	`Person`, `PersonJpaRepository`, `AbisPersonService`, `PersonServiceTest`
11.3, 12.3	Sessions of a course (JPQL, then native without cancelled)	`Session`, `SessionJpaRepository`, `AbisTrainingService`
11.1	Company + Address, find by name / name and town	`Company`, `Address`, `CompanyJpaRepository`, `CompanyRepositoryTest`
12.1	PersonDTO, PersonForm, PersonMapper	`dto/PersonDTO`, `form/PersonForm`, `mapper/PersonMapper`
12.2	SessionDTO as a record	`dto/SessionDTO`
12.4	Enrolment: composite key, named query, EnrolmentDTO	`Enrolment`, `EnrolmentId`, `EnrolmentJpaRepository`, `dto/EnrolmentDTO`
13.1	PersonAlreadyExistsException works again	`AbisPersonService.addPerson`, `ApiExceptionHandler`
13.2	Delete a session unless it has enrolments	`AbisTrainingService.deleteSession`, `SessionCanNotBeDeletedException`
13.3	Change a course price with a modifying native query	`CourseJpaRepository.changePrice`
13.4	Cancel a session with a modifying native query	`SessionJpaRepository.cancelSession`
13.5	Transactional `enrolForSession` with rollback	`AbisTrainingService.enrolForSession`, `TrainingServiceTest`

A.10 What changed compared to the starting project

Before	After
`MemoryCourseRepository` (hard-coded list)	`CourseJpaRepository` — an interface, Spring writes the implementation
`FilePersonRepository` (CSV file, loops)	`PersonJpaRepository` — derived queries
Empty `addCourse` / `updateCourse` / `deleteCourse`	Real DML, annotated `@Transactional`; add and update return the saved object
Controllers returned the model classes	Controllers return DTOs and accept forms; a mapper converts
Training service methods returned `null`	Sessions, enrolments, cancel, delete and enrol are implemented
No database	H2 file database, `spring.jpa.show-sql=true` shows every generated statement in the console

Run `mvnw test` to check the whole solution (45 tests). Start the app and call `http://localhost:8085/api/courses/7900` to see JSON coming out of the database.